Proof of Concept Technical Solution for the Marconi Law Firm, LLC. (WordPress Website)

Orlando, Florida

Robert Potter

Office Networks and Security Wizards LLC.

Table of Contents

Inventory

EQUIPMENT	OPERATIING SYSTEM	ADDITIONAL INFO	IP ADDRESS
Router/Custom Network	- (Firewall VM)	-Firewall VM	10.10.229.1
Docker	Rocky 8 (-Docker)	Ghost Container	10.10.229.11
NginX Reverse Proxy	Rocky 8 (-Nginx)	Reverse Proxy	10.10.229.10
WordPress	Ubuntu	LAMP Stack running WordPress	10.10.229.12

Custom Network

NETWORK NAME	SUBNET IP	SUBNET MASK	DNS	GATEWAY
ITE229	10.10.229.0	255.255.255.0	10.10.229.1	10.10.229.1

IDs and Passwords

ACCOUNT	USER ID	PASSWORD
Rocky8-Docker Root User	root	Fullsail1!
Rocky8-Nginx Root User	root	Fullsail1!
Ubuntu Root User	Root	Fullsail1!
MySQL Root User	root@localhost	Fullsail1!
MySQL WordPress User	WordPressUser	Fullsail2!
WordPress Admin	admin	Fullsail1!

Preface

This document will serve as proof of concept to Mr. Marconi for creating his WordPress website for his law firm and as audit documentation.

The purpose of audit documentation is to provide a comprehensive record of the organization's information technology infrastructure and security controls and processes. It plays a crucial role in providing transparency, accountability, and QA/QC regarding an organization's cybersecurity controls and practices. It enables organizations to demonstrate compliance, identify areas for improvement, and make informed decisions to strengthen their overall organizational cybersecurity.

Audit documentation serves several important purposes:

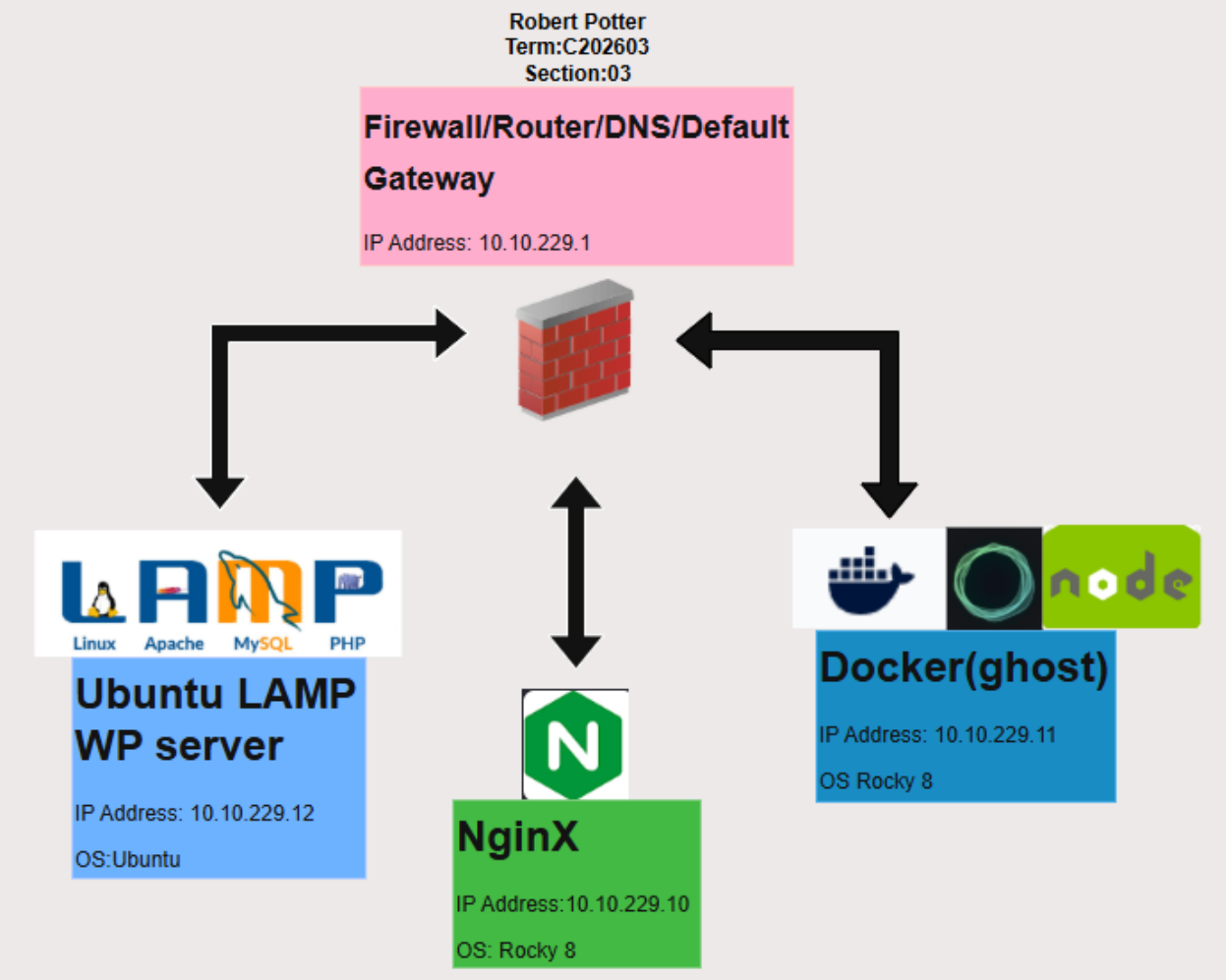
- Compliance: Evidence that an organization has undergone a thorough examination of its systems. It helps validate that the organization has implemented appropriate controls to protect its information systems and sensitive data.

- Validation: Verification of the effectiveness and adequacy of cybersecurity controls. It provides detailed information about the design, implementation, and operation of these controls, enabling reviewers to assess their reliability and identify any gaps or weaknesses.

- Records Maintenance: Historical record of cybersecurity audits conducted over time. It enables organizations to track their progress, identify trends, and evaluate the effectiveness actions taken. It also serves as reference for future audits and allows auditors to understand the current cybersecurity implemented and facilitates a more targeted approach to future cybersecurity updates and audits.

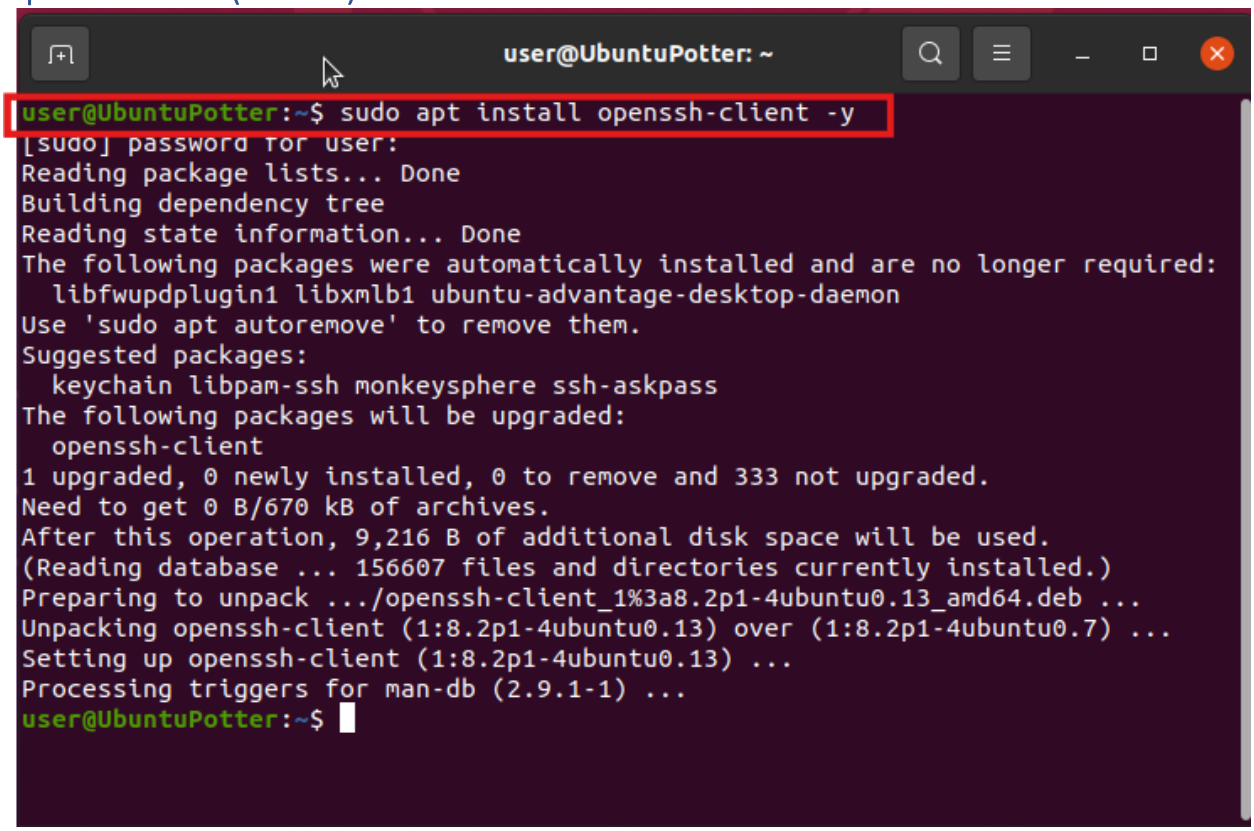
- Decision-making Support: Valuable insights and information that can support decision-making processes. It allows management to make informed decisions about allocating resources, prioritizing cybersecurity investments, and addressing identified risks and vulnerabilities.

Network Topology Diagram



SSH Install & Access

OpenSSH Install (Ubuntu)

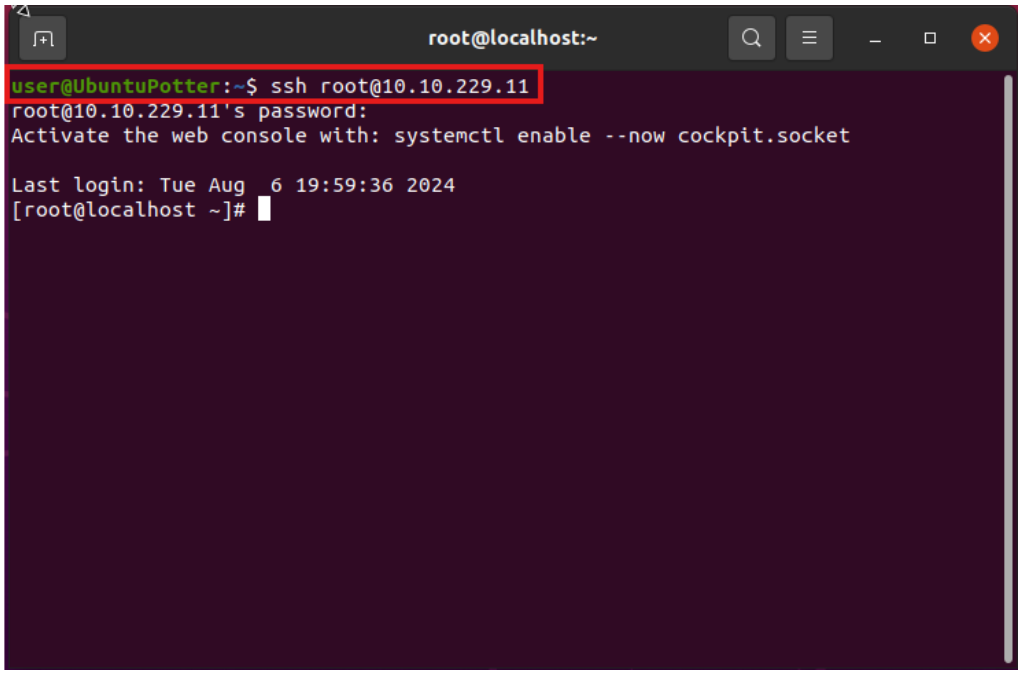


To Install open SSH on Your Ubuntu Machine with Logged into our user Account. We then went to Terminal in the “Activities” Button in the upper right hand side of the Desktop. Once in the Terminal we entered the Command “**sudo apt install openssh-client -y**” you are then prompted to enter your password. Once entered correctly the system will install the package or confirm that the package is already installed.

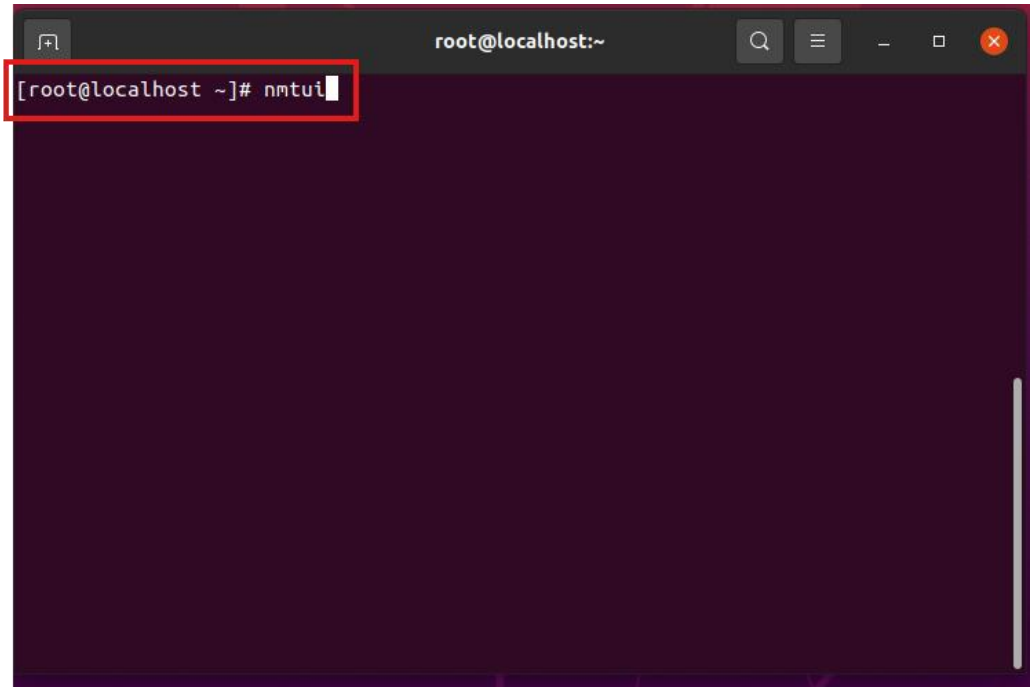
Node.js Application (Ghost) on Docker

Update Rocky8-Docker Host Name

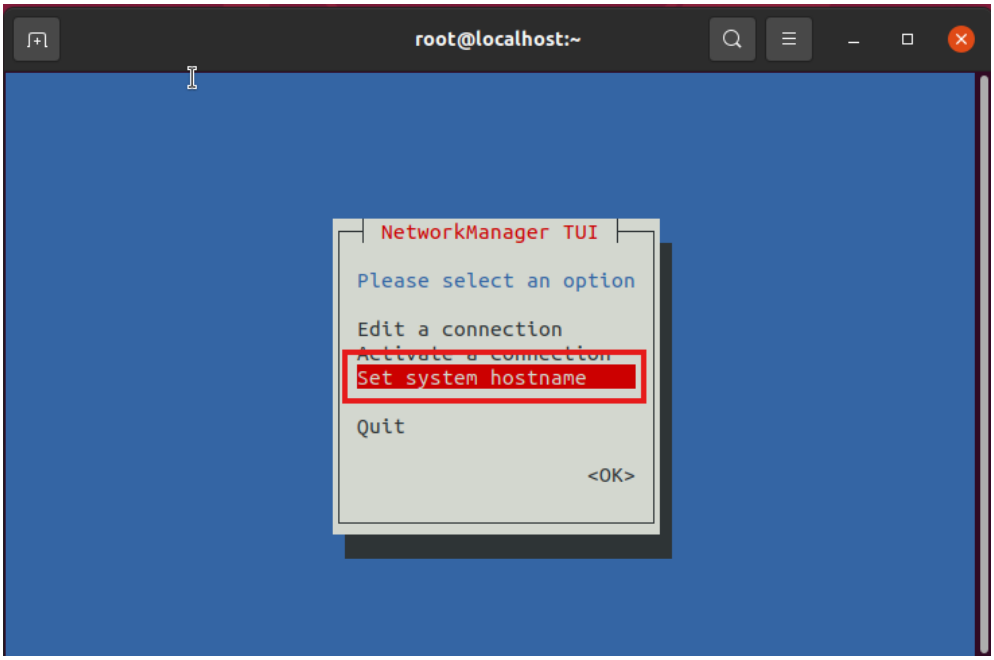
First We will SSH into The Rocky 8 Docker Host using the command "**ssh root@10.10.229.11**" Followed by the password:



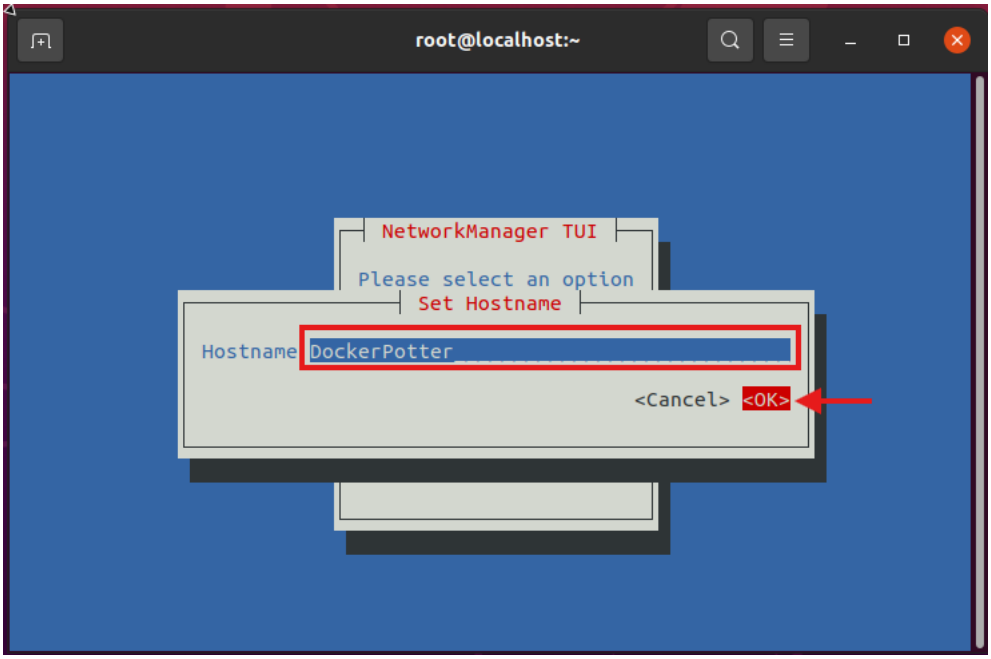
We will then use the "**nmtui**" command to set the host name.



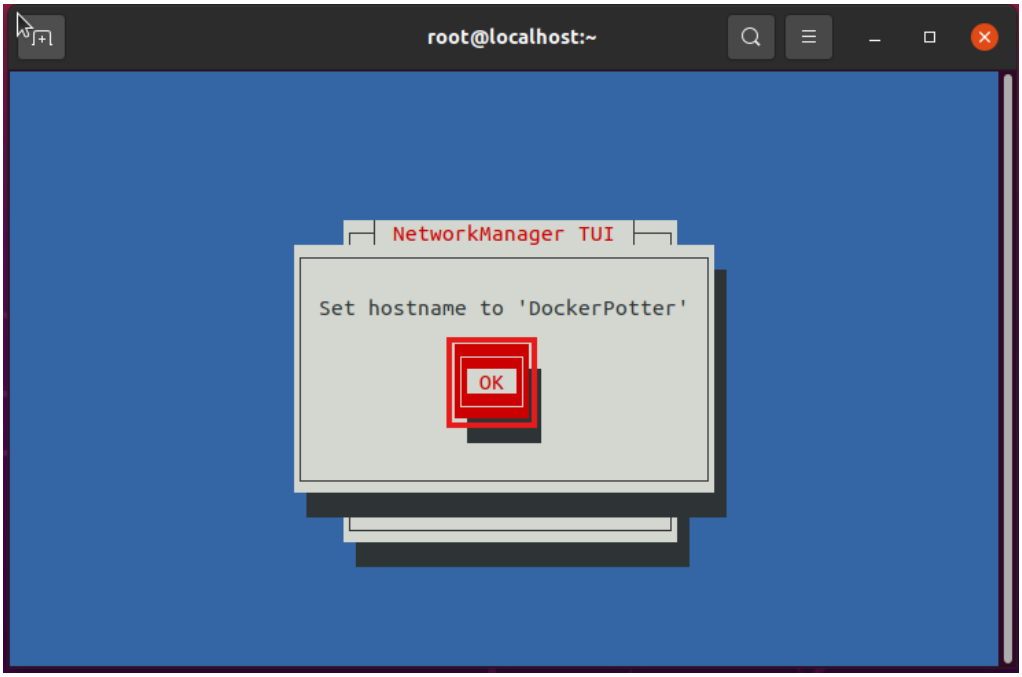
Select “Set system hostname”



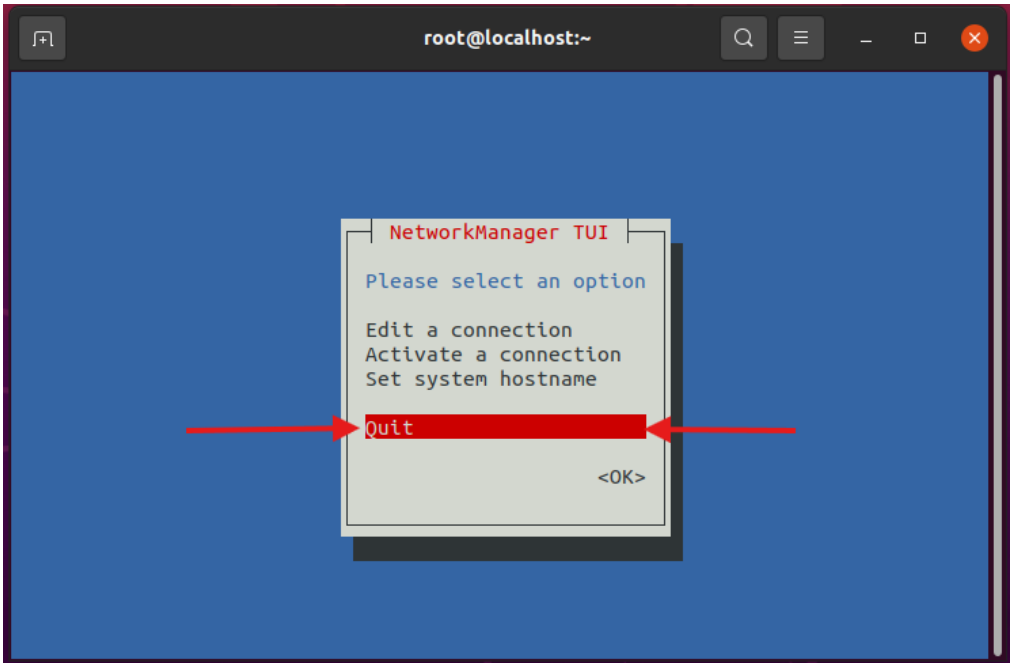
Then Enter the desired hostname and Select “ok”



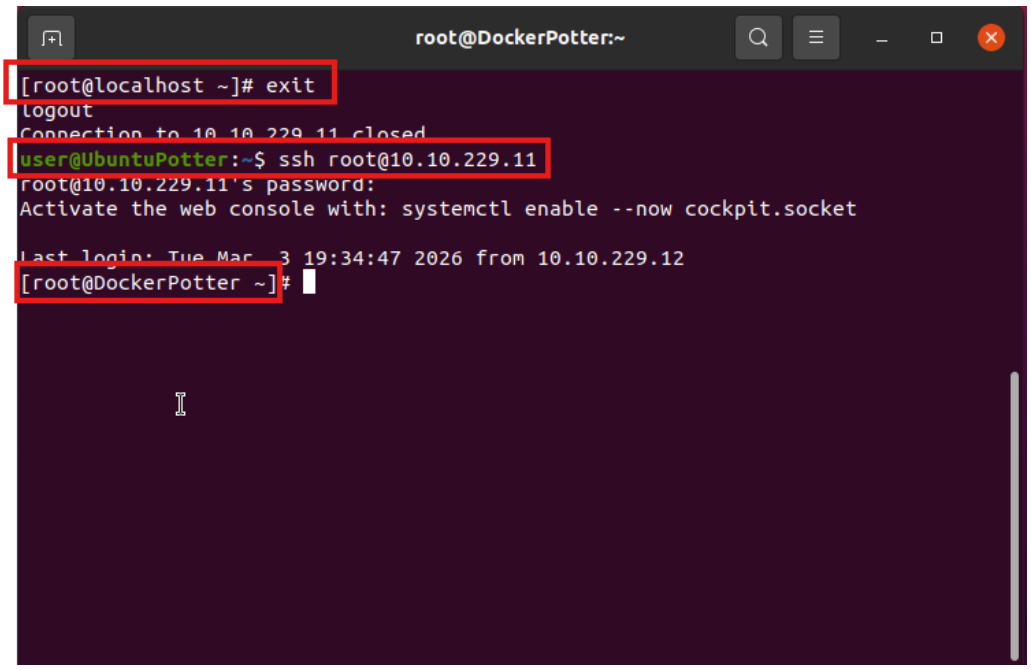
Select “OK”



Select “Quit”

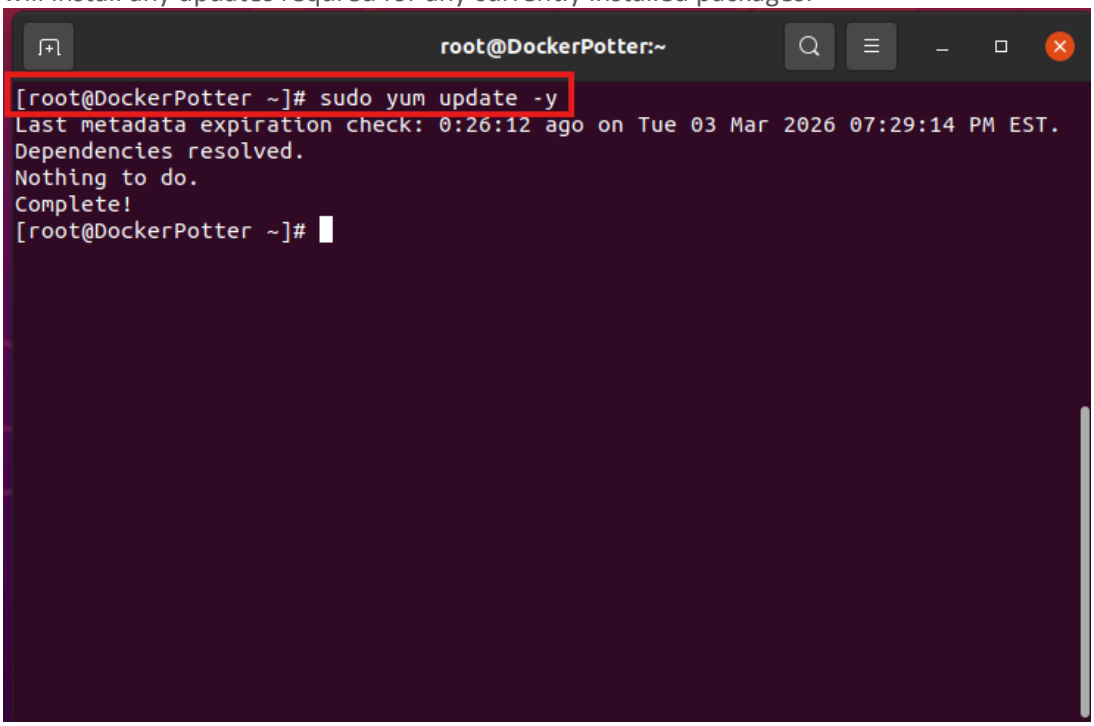


Use the command “**exit**” and Reconnect to see the new Host name.



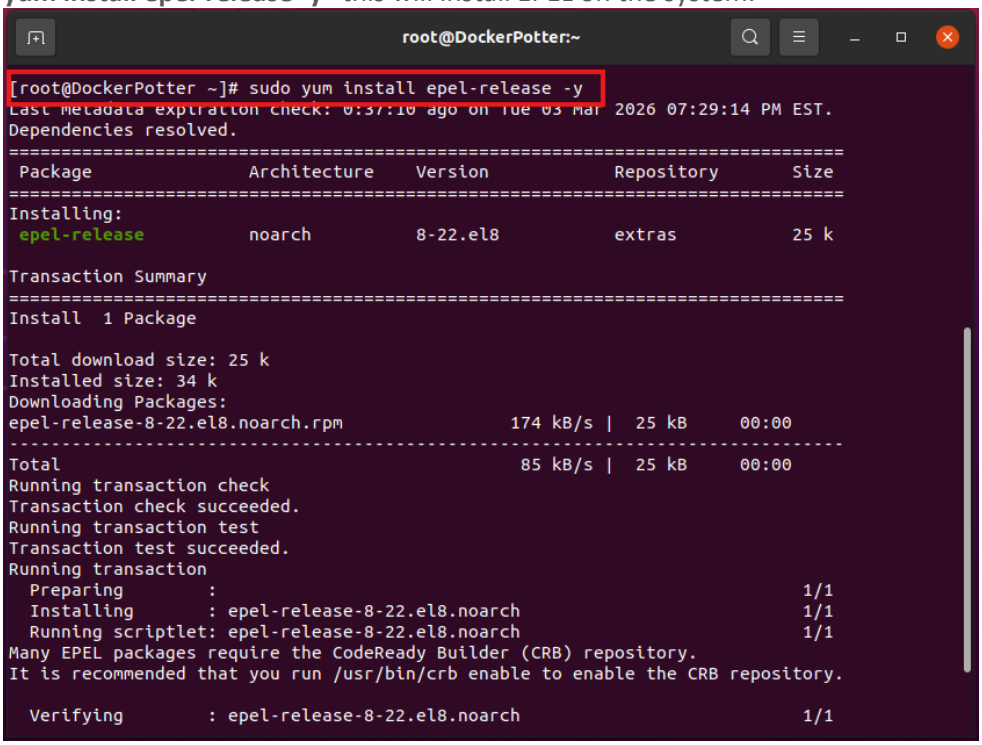
Update Rocky8-Docker

To run the updates on our Rocky8-Docker System we simply input the command "**sudo yum update -y**" This will install any updates requred for any currently installed packages:



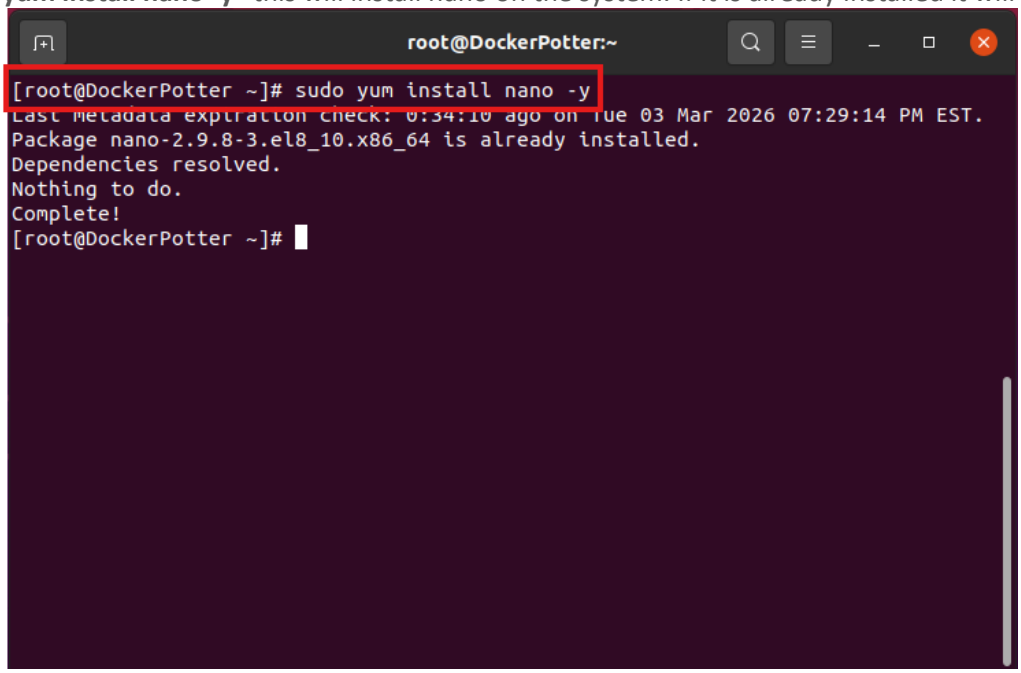
Install EPEL Packages

We now want to install EPEL packages on the docker system so to do this we will use the command "**sudo yum install epel-release -y**" this will install EPEL on the system.



Install Nano Editor

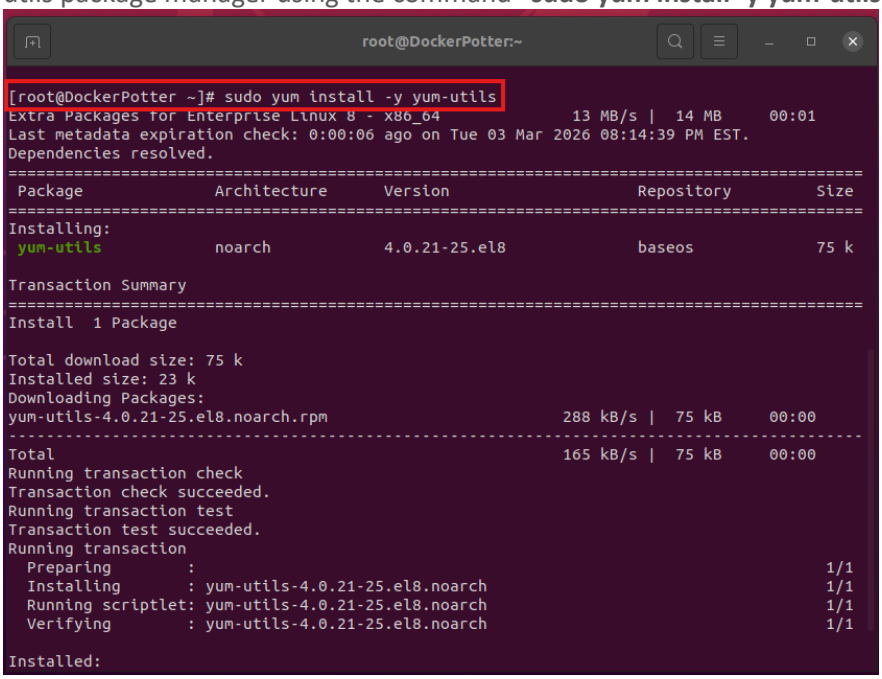
We now want to ensure nano is installed on the docker system so to do this we will use the command "**sudo yum install nano -y**" this will install nano on the system. If it is already installed it will show like this:



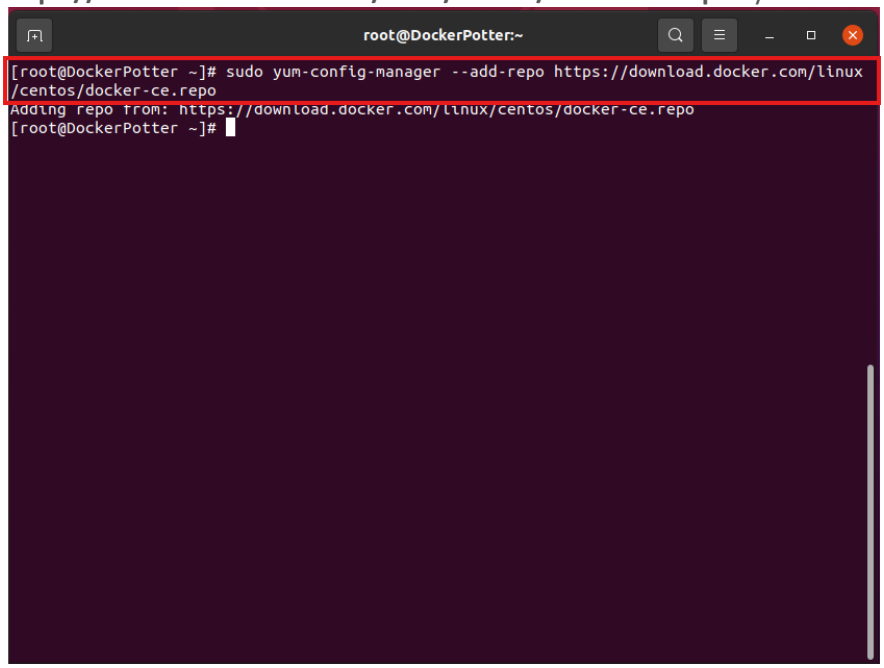
Docker CE

Set up stable repository

We now want to setup the stable repository for Docker. In order to do this we first need to install the yum-utils package manager using the command "**sudo yum install -y yum-utils**":

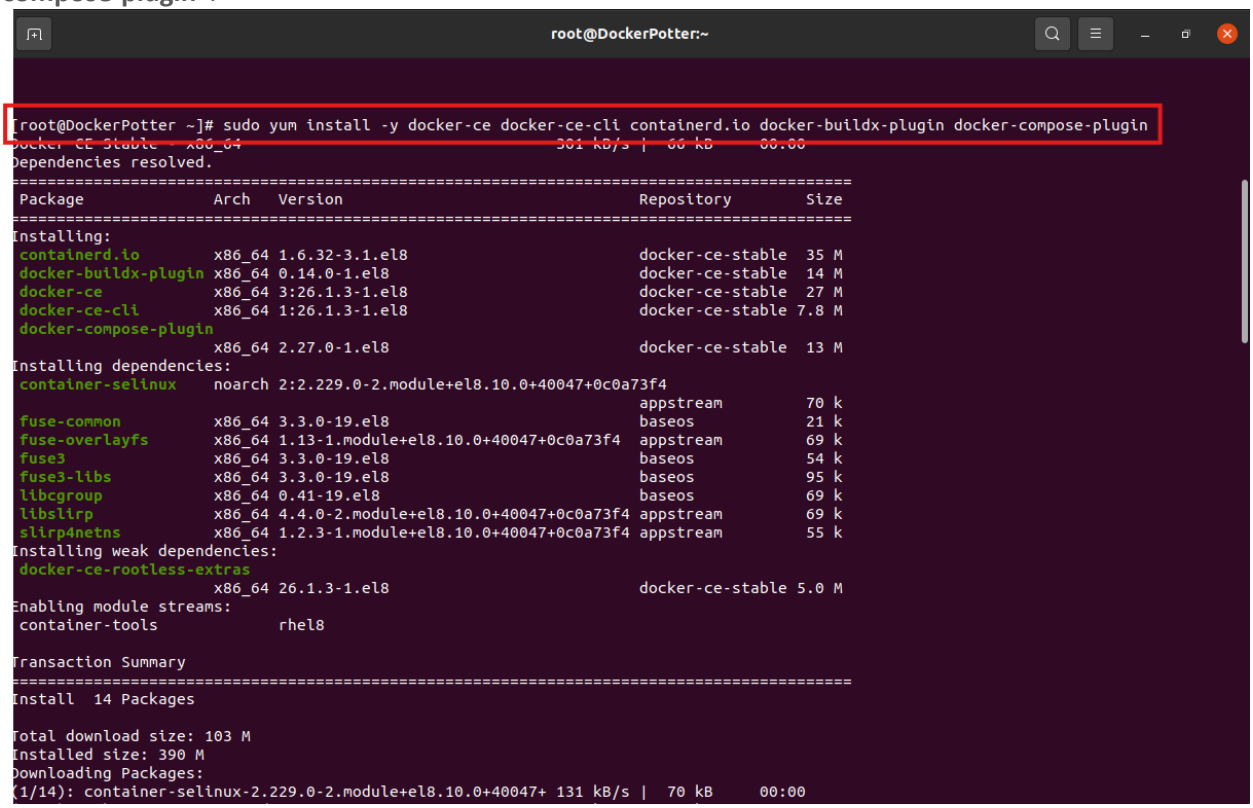


Once this is installed, we want to add the Repo using this command "**sudo yum-config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo**" you should get an output like below:



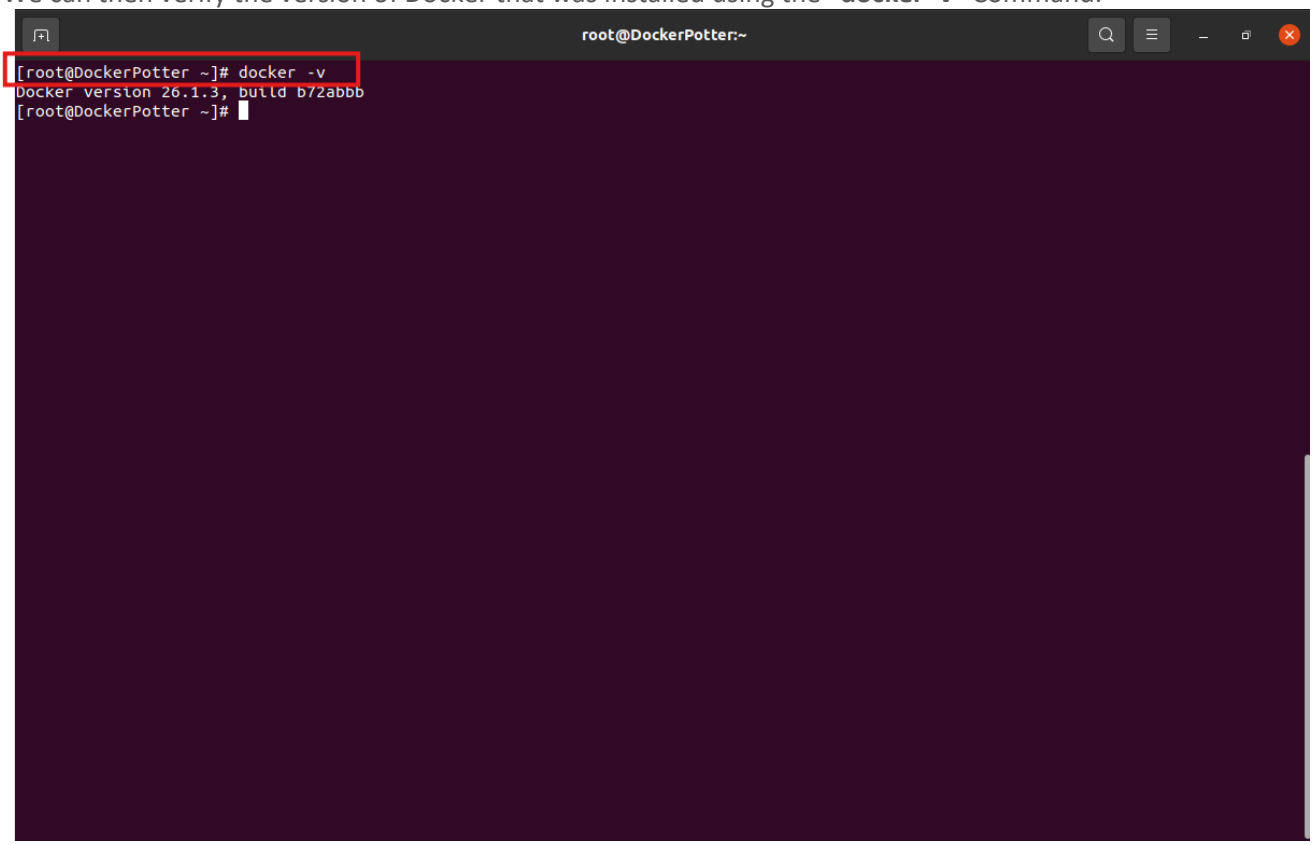
Install Docker CE

Now we will install Docker CE, We will use the following command to download docker and some of it dependicys "**sudo yem install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin**":



Verify Docker version

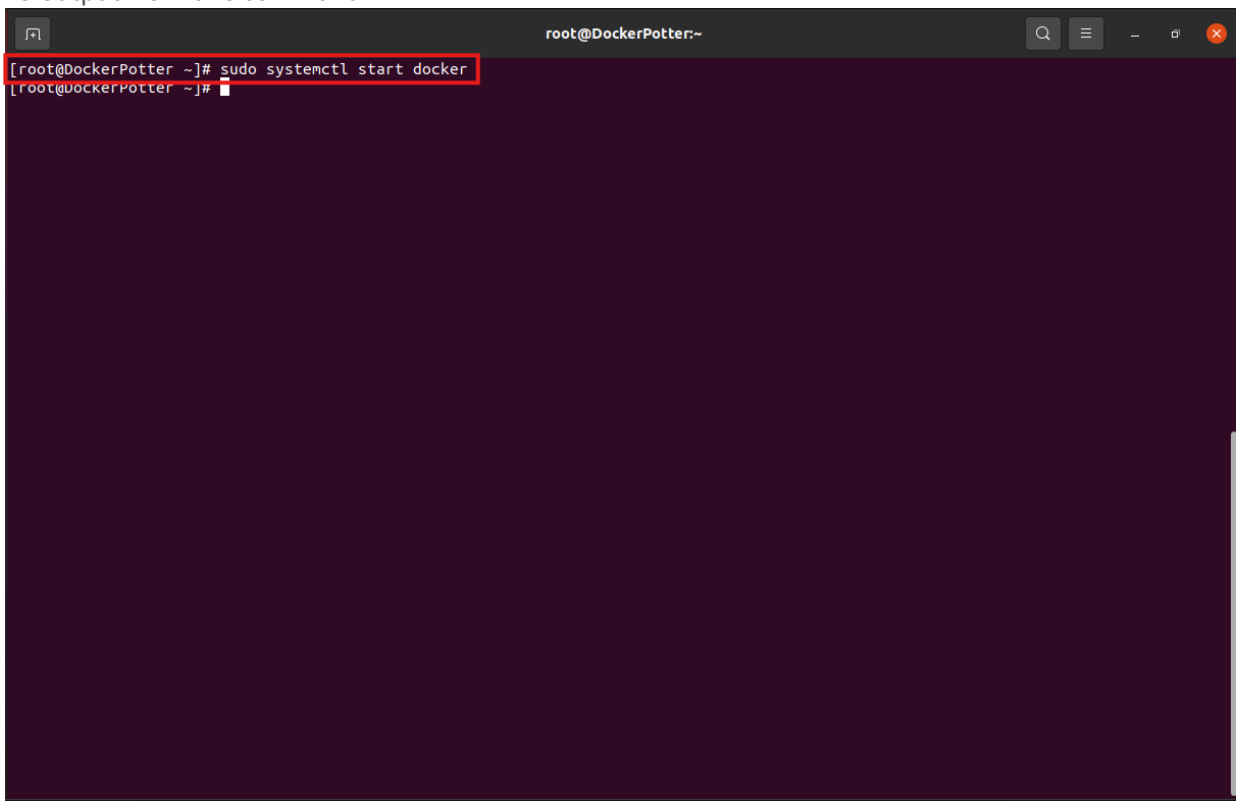
We can then verify the version of Docker that was installed using the "**docker -v**" Command:



Initialize Docker

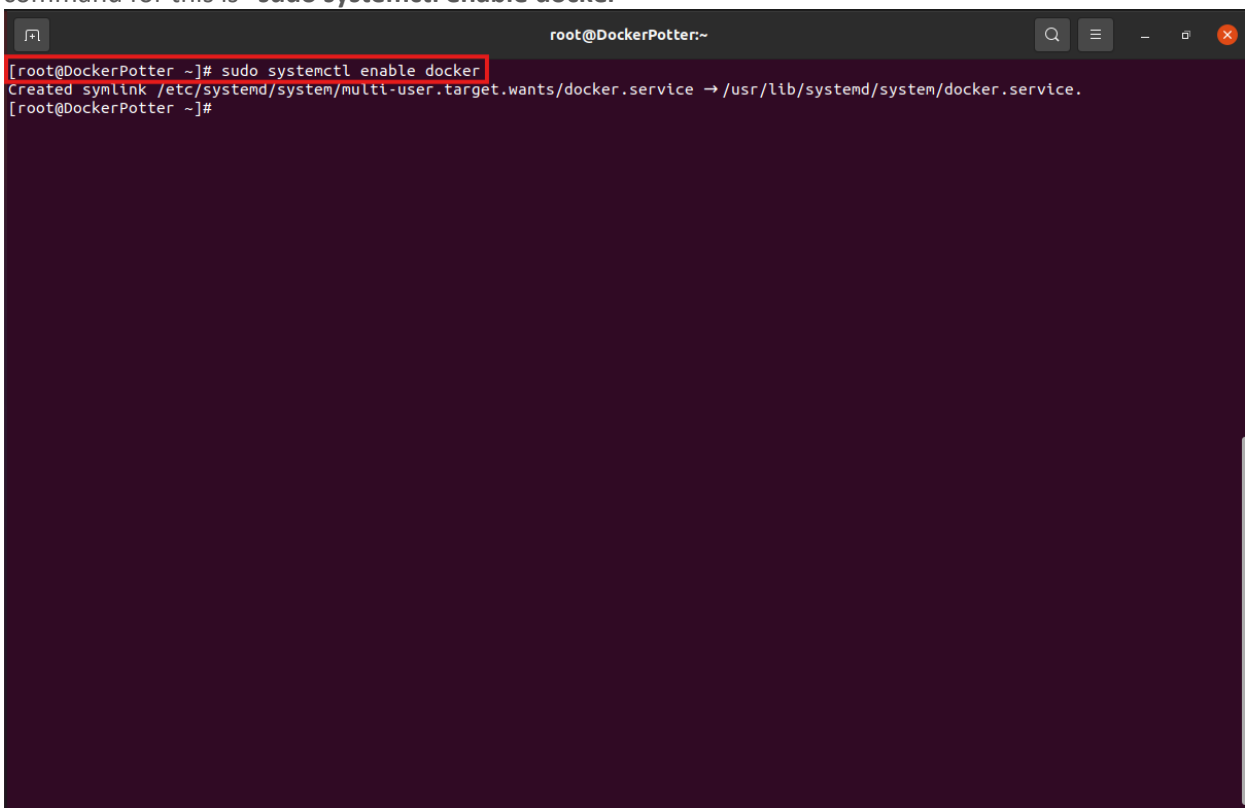
Start Docker

We now need to start the Docker service using the command "**sudo systemctl start docker**" Note: There is no output from this command.



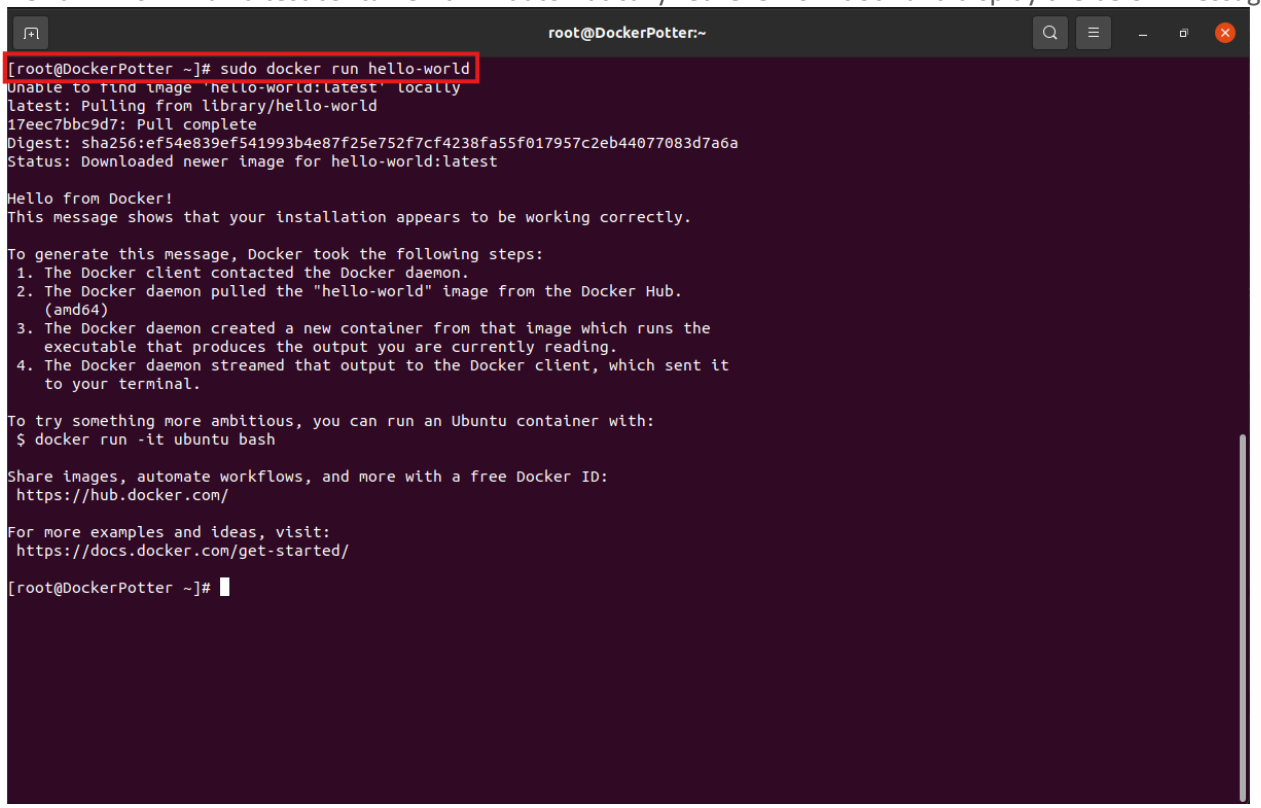
Enable Docker

After that, we need to enable Docker, This will automatically start docker when the system boots up, the command for this is "**sudo systemctl enable docker**"



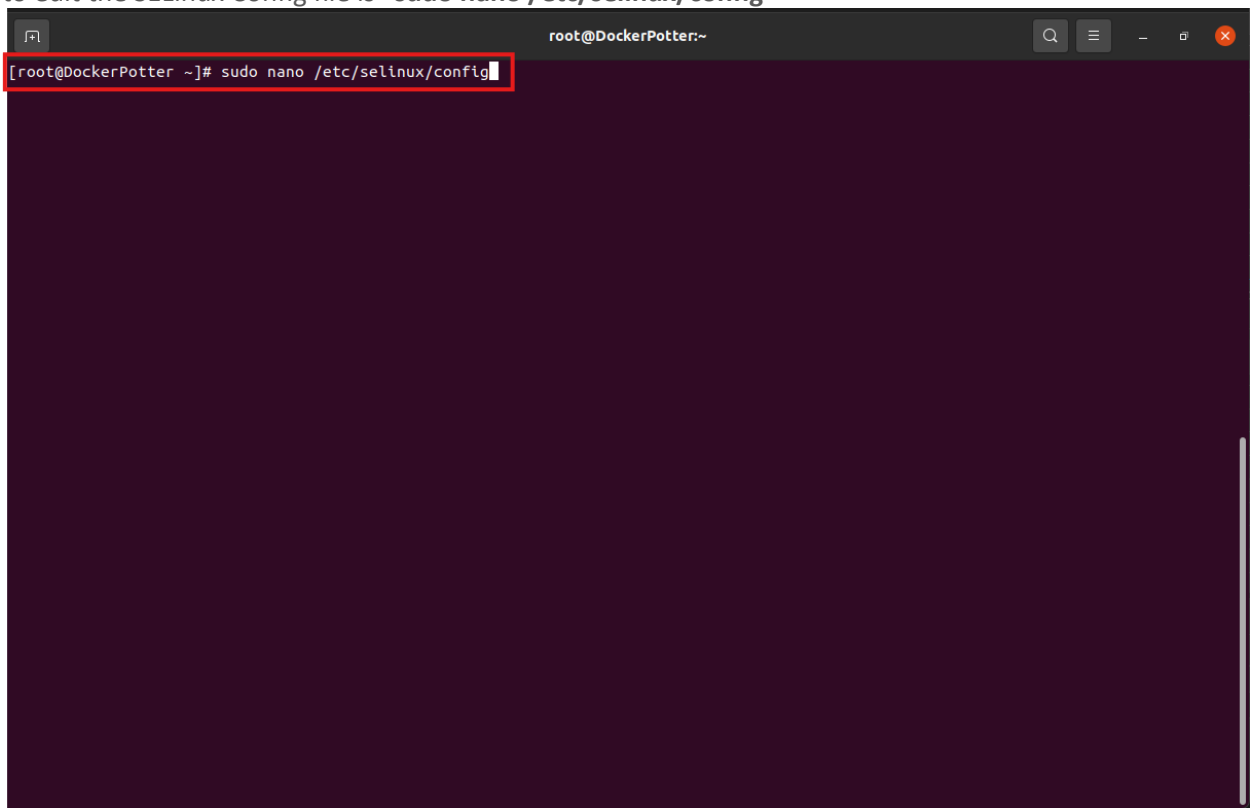
Test Docker (hello-world)

We will now test that our Docker install is working correctly by using the command "**sudo docker run hello-world**". This wil run a test container it will automatically retrieve from dock and display the below message:

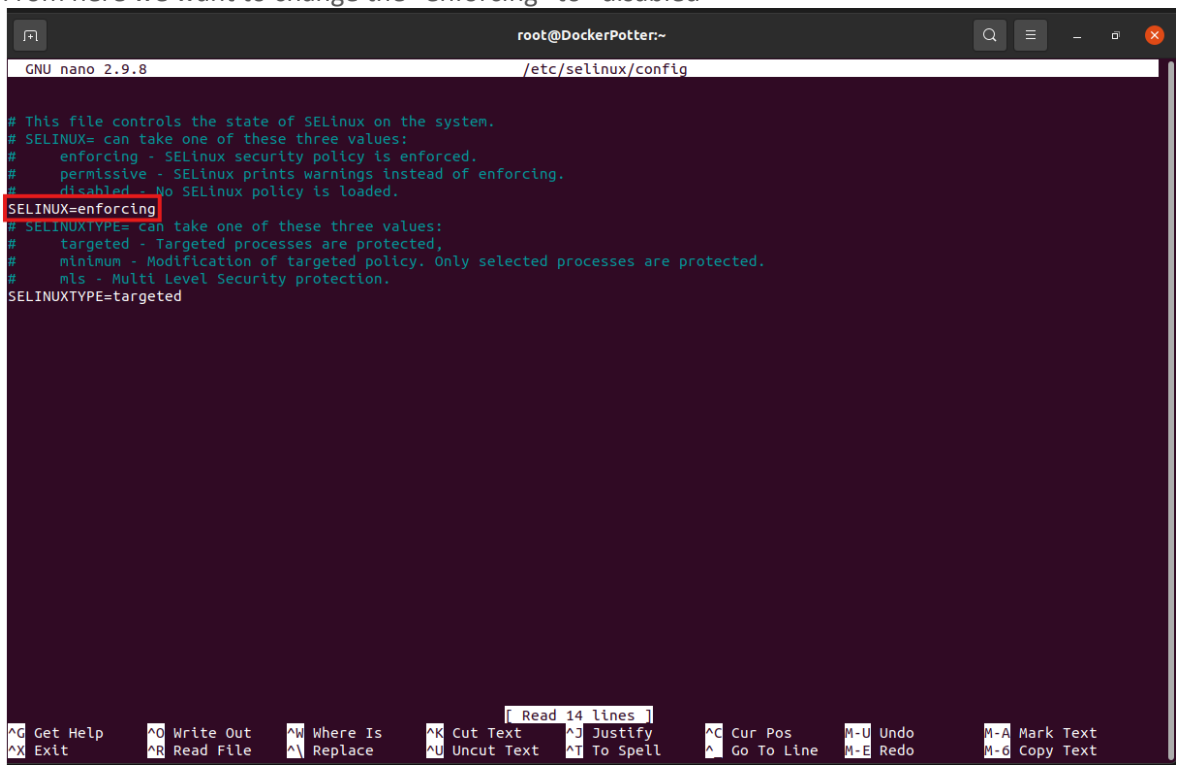


Disable SELinux (/etc/selinux.config)

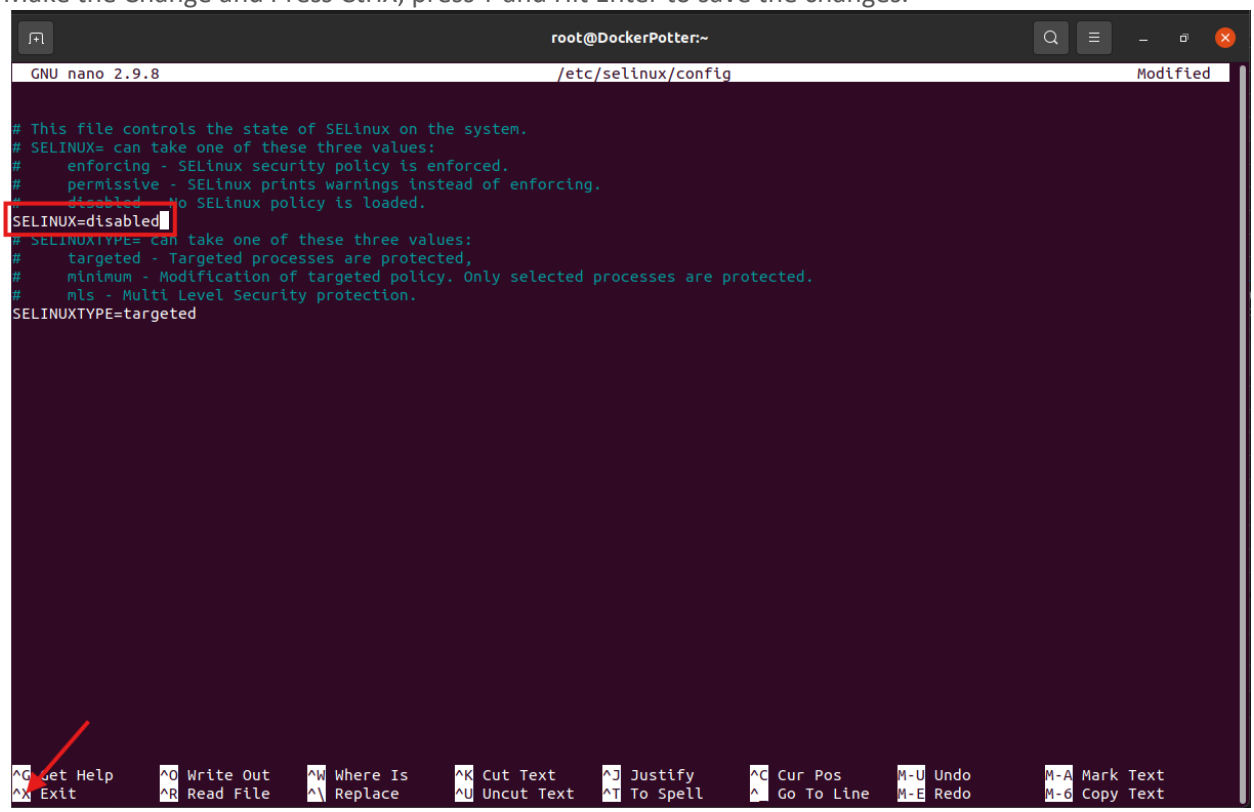
We will now be disabling SELinux in this enviroment so that we done need to login and disable it every time we want to make a change in the future. I can only recommend this in a closed environment. The command to edit the SELinux Config file is "**sudo nano /etc/selinux/config**"



From here we want to change the “enforcing” to “disabled”

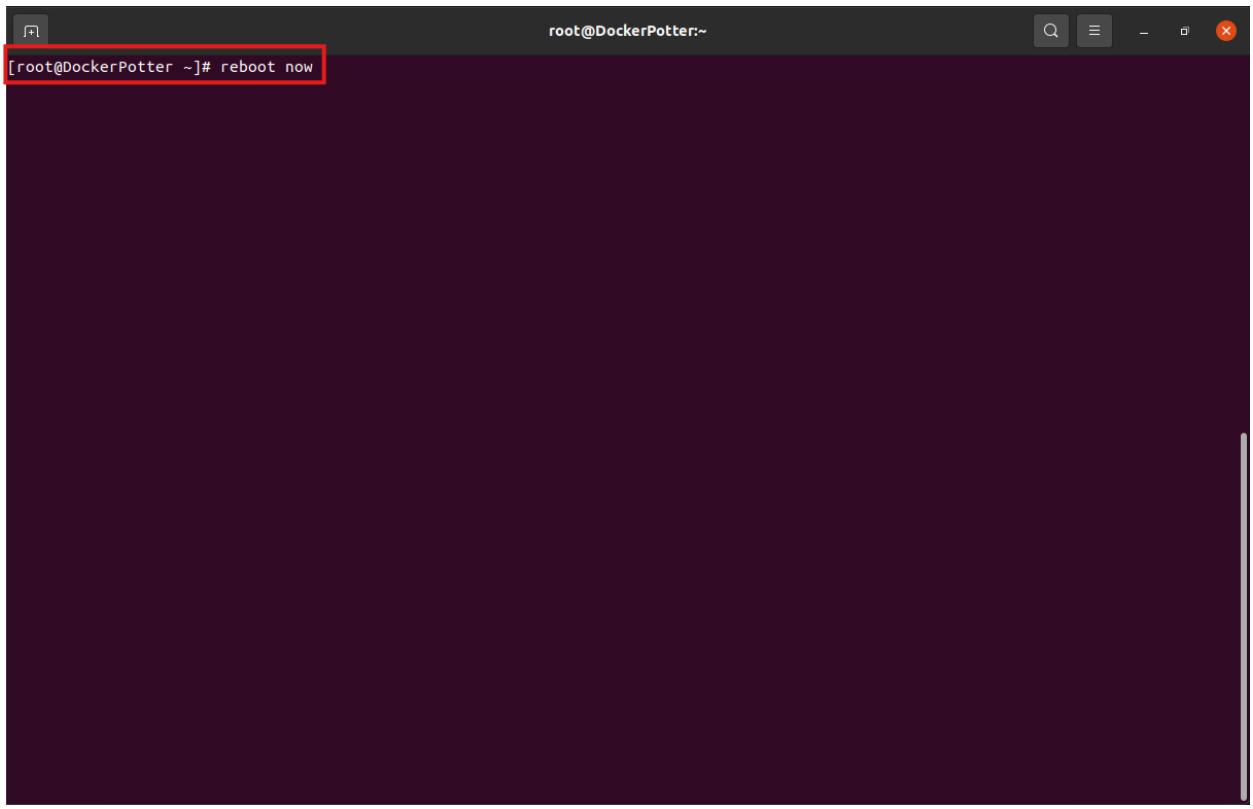


Make the Change and Press CtrlX, press Y and Hit Enter to save the changes.



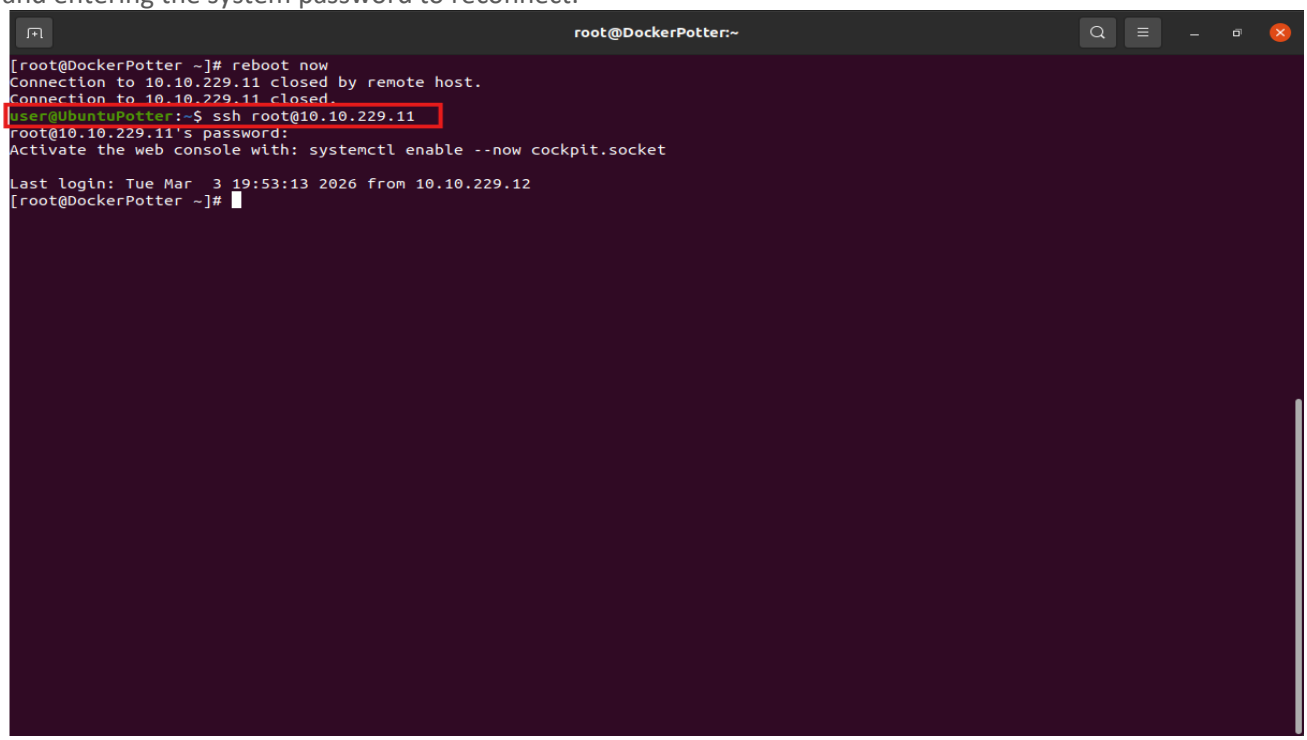
Reboot VM

We now want to reboot to make sure the SELinux changes save correctly, to do this we will use the "**Reboot now**" command:

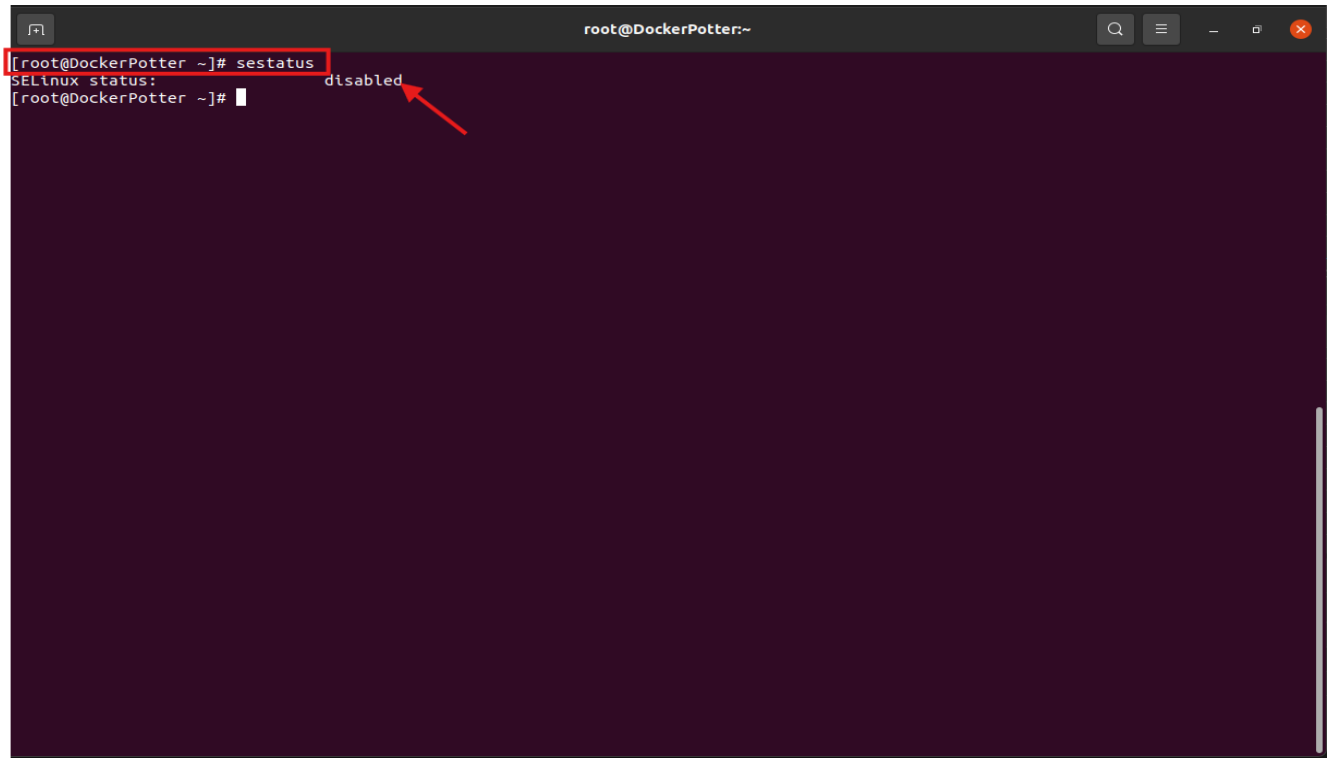


Confirm SELinux Status

After Waiting a moment for the system to reboot with will reconnect using the "**ssh root@10.10.229.11**" and entering the system password to reconnect:

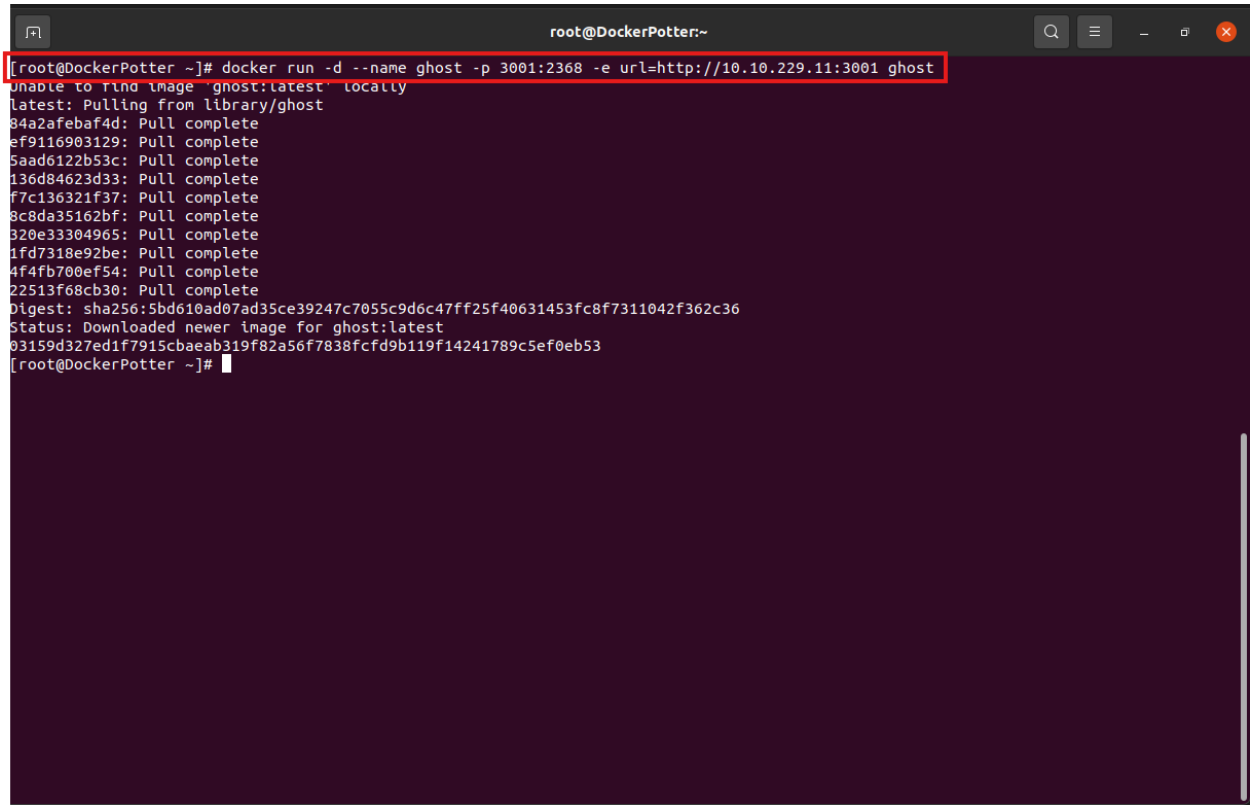


And to confirm That the SELinux is disable we will use the command "**sestatus**" the output should look like this:



Install Ghost Docker Container

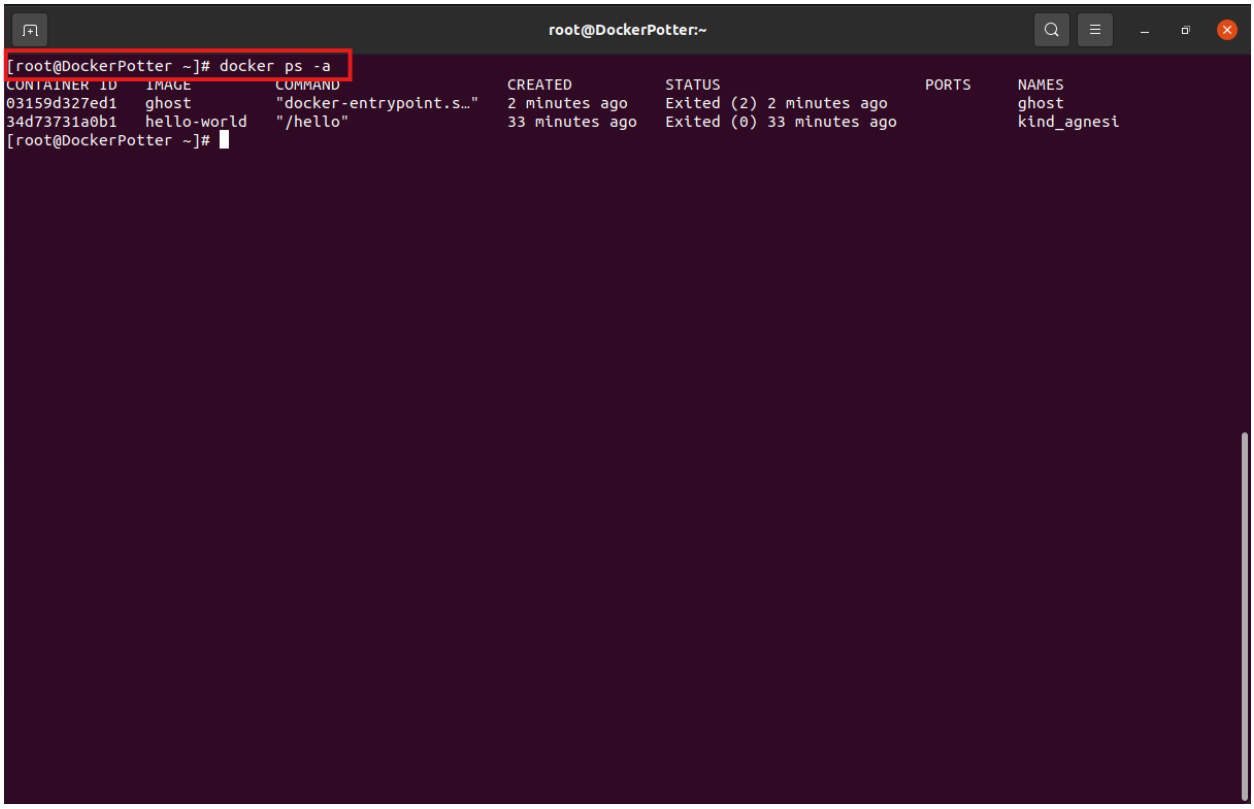
We will now create,start and name a new ghost conatiner to our docker setup. The command for this is "**docker run -d --name ghost -p 3001:2368 -e url=http://10.10.229.11:3001 ghost**" you should get an output that looks like this:



Check for Ghost Container (docker ps)

Ghost Container ID 03159d327ed1

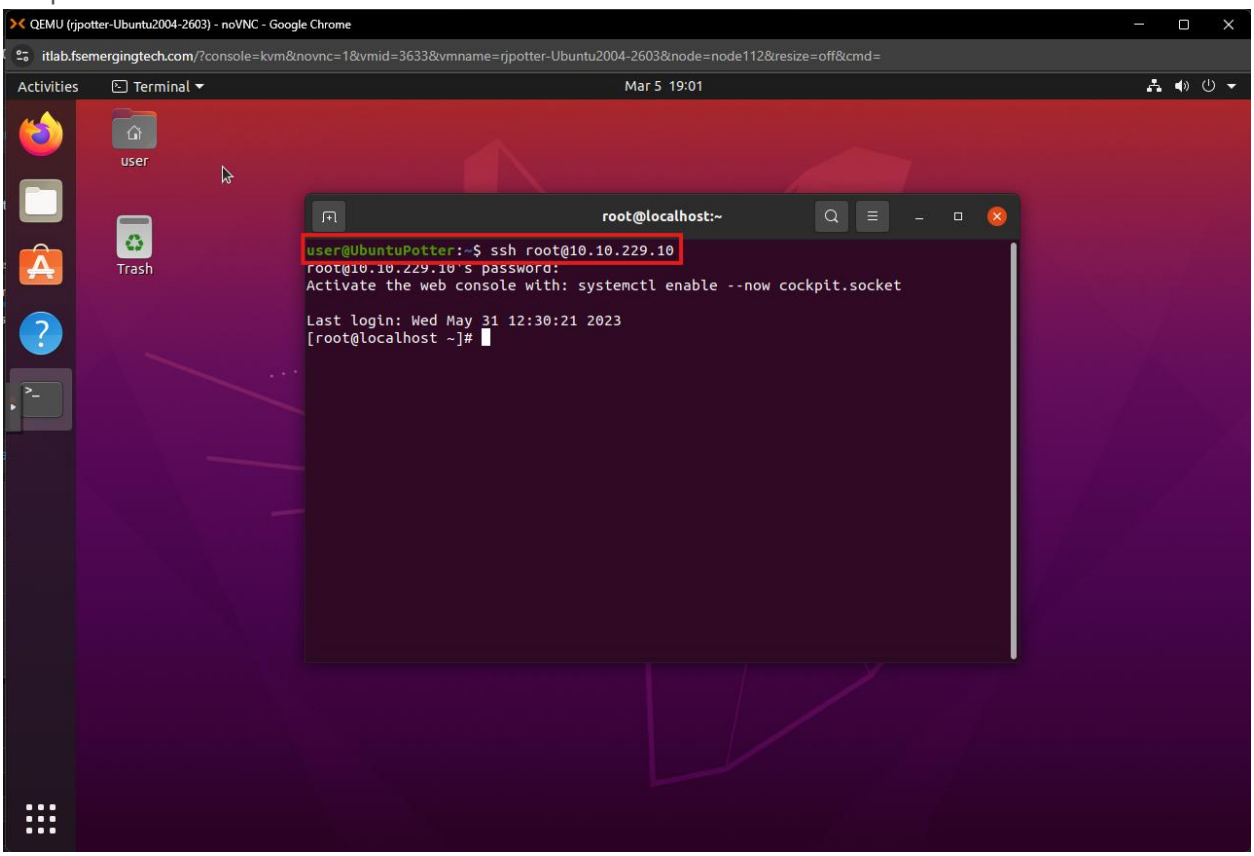
We now want to see our new container and confirm the id of it, in order to do this we will use the following command "**docker ps -a**" this command lists all containers.



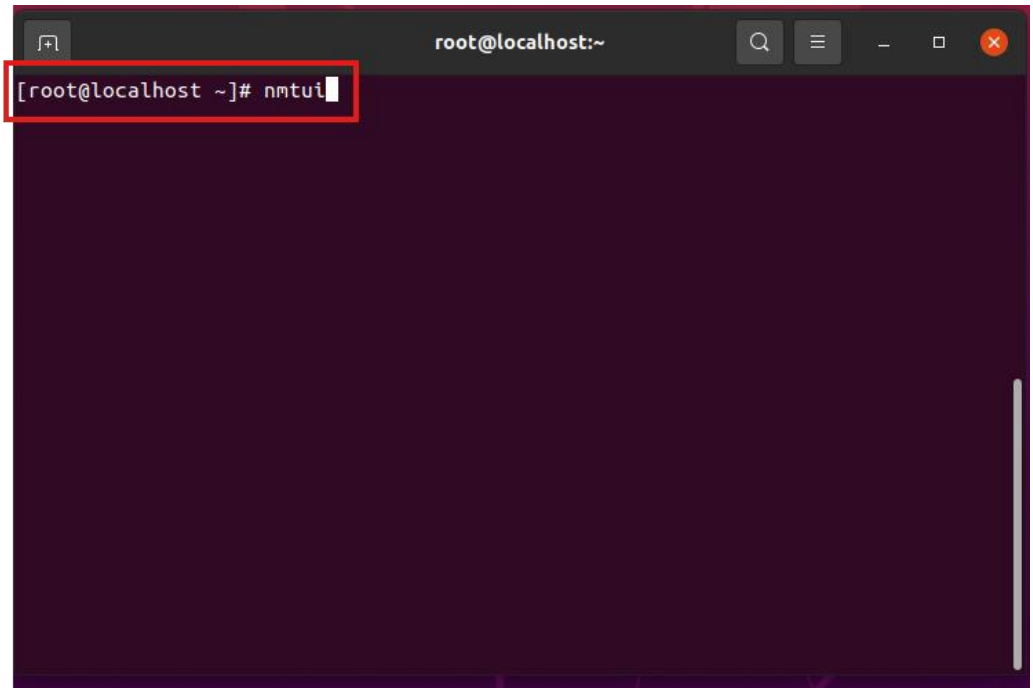
NginX Reverse Proxy

Update Rocky8-Nginx Host Name

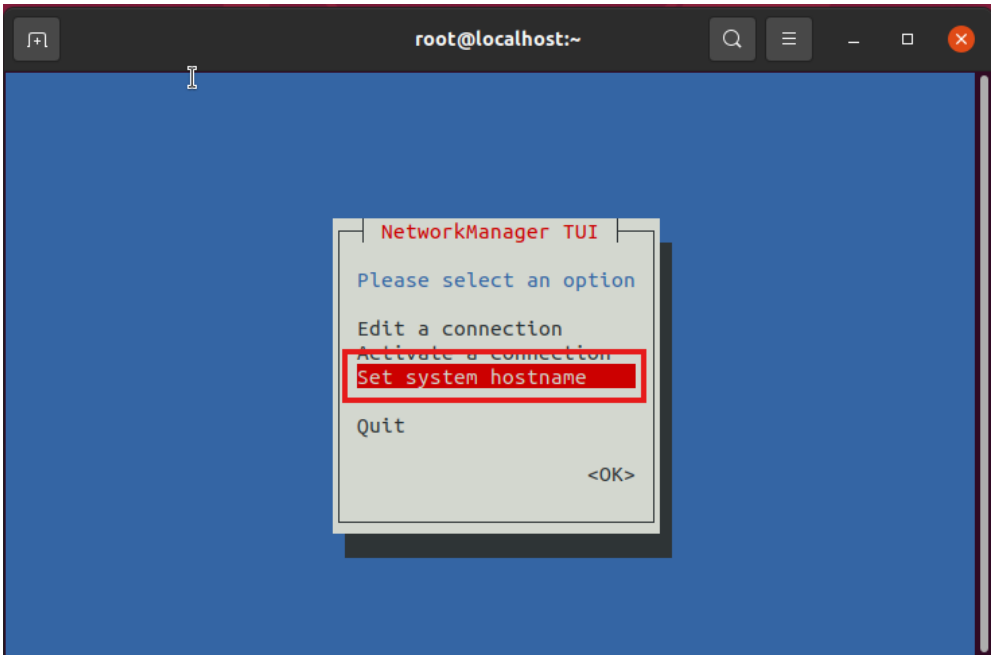
First We will SSH into The Rocky 8 NginX Host using the command "**ssh root@10.10.229.10**" Followed by the password:



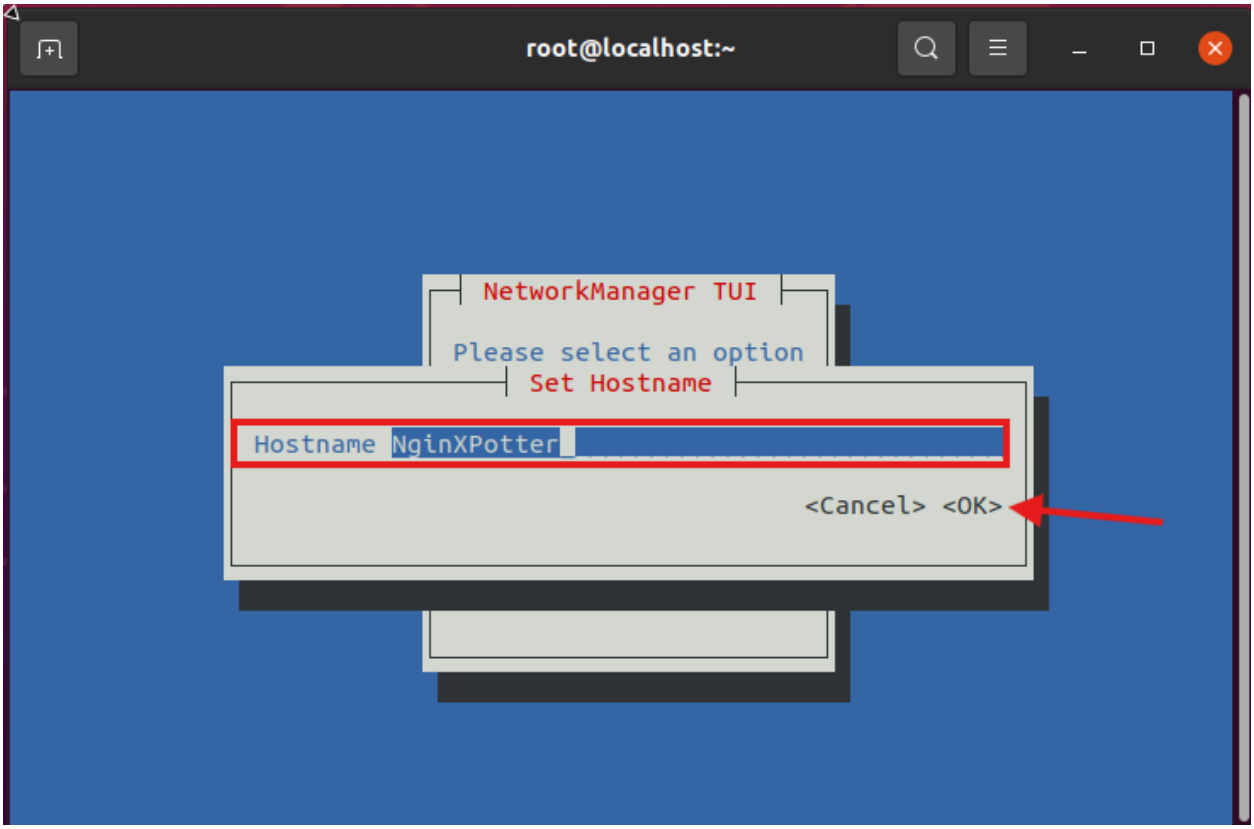
We will then use the "**nmtui**" command to set the host name.



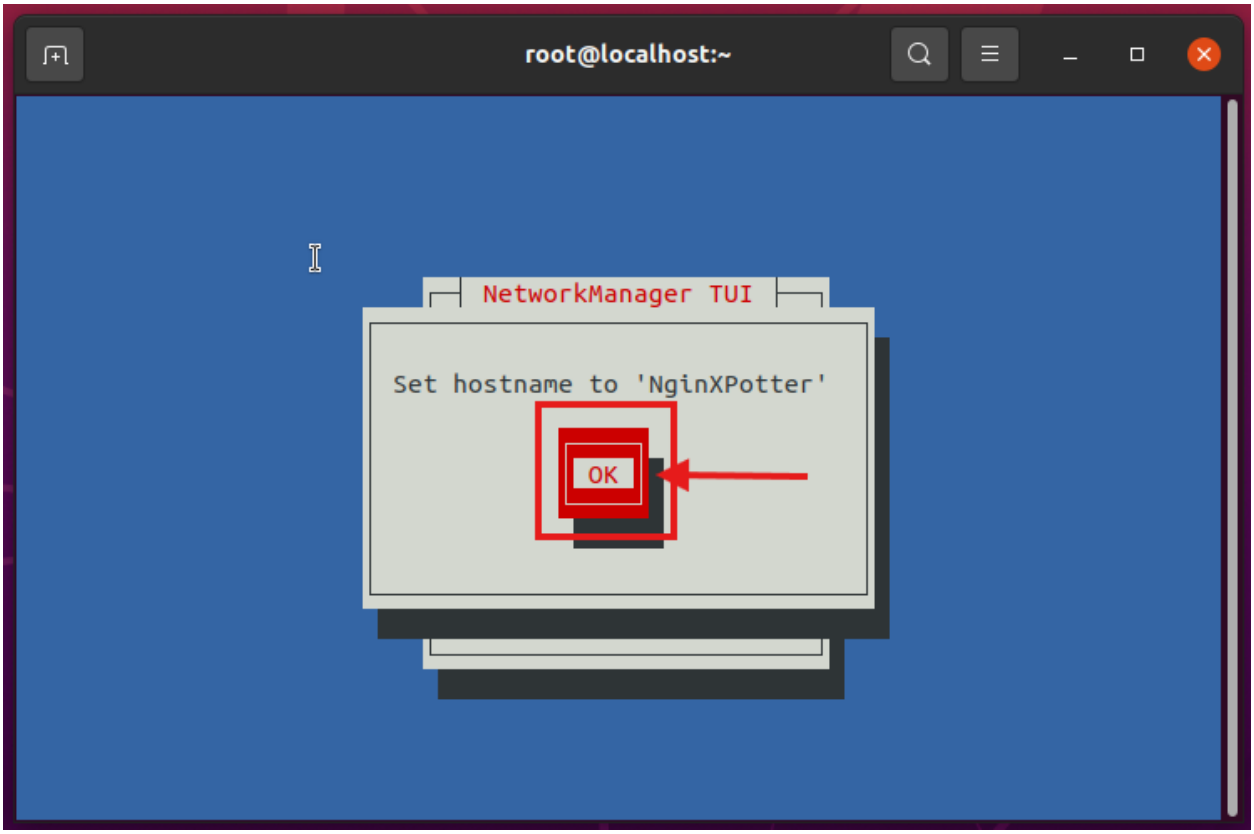
Select "Set system hostname"



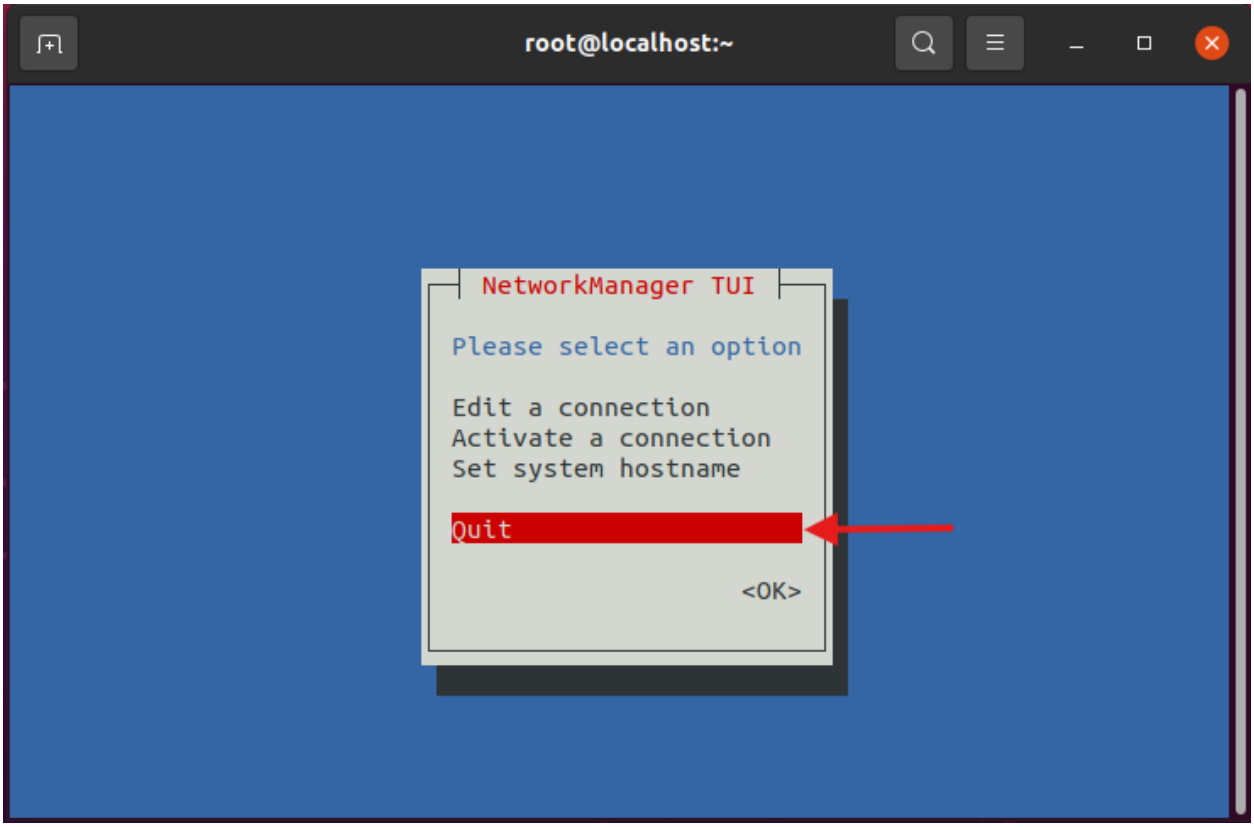
Then Enter the desired hostname and Select “ok”



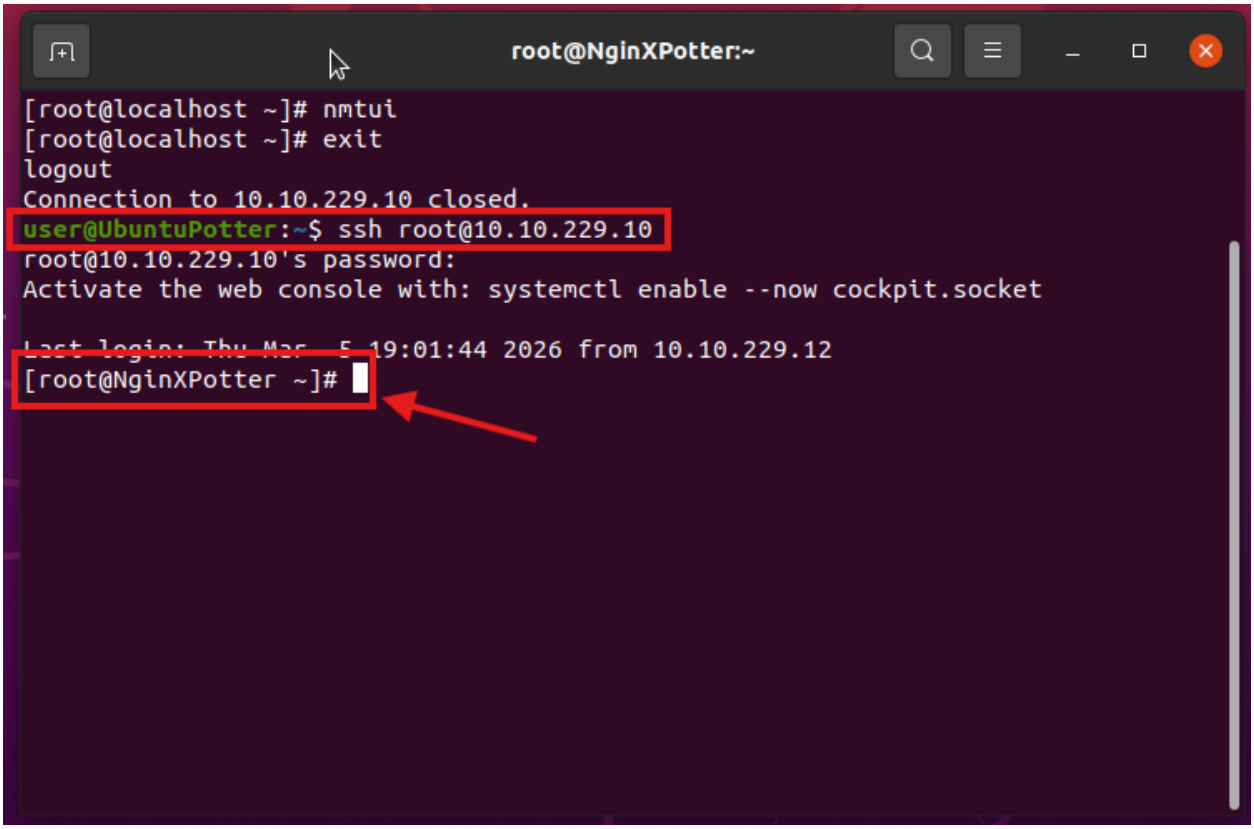
Click Ok:



Select Quit:

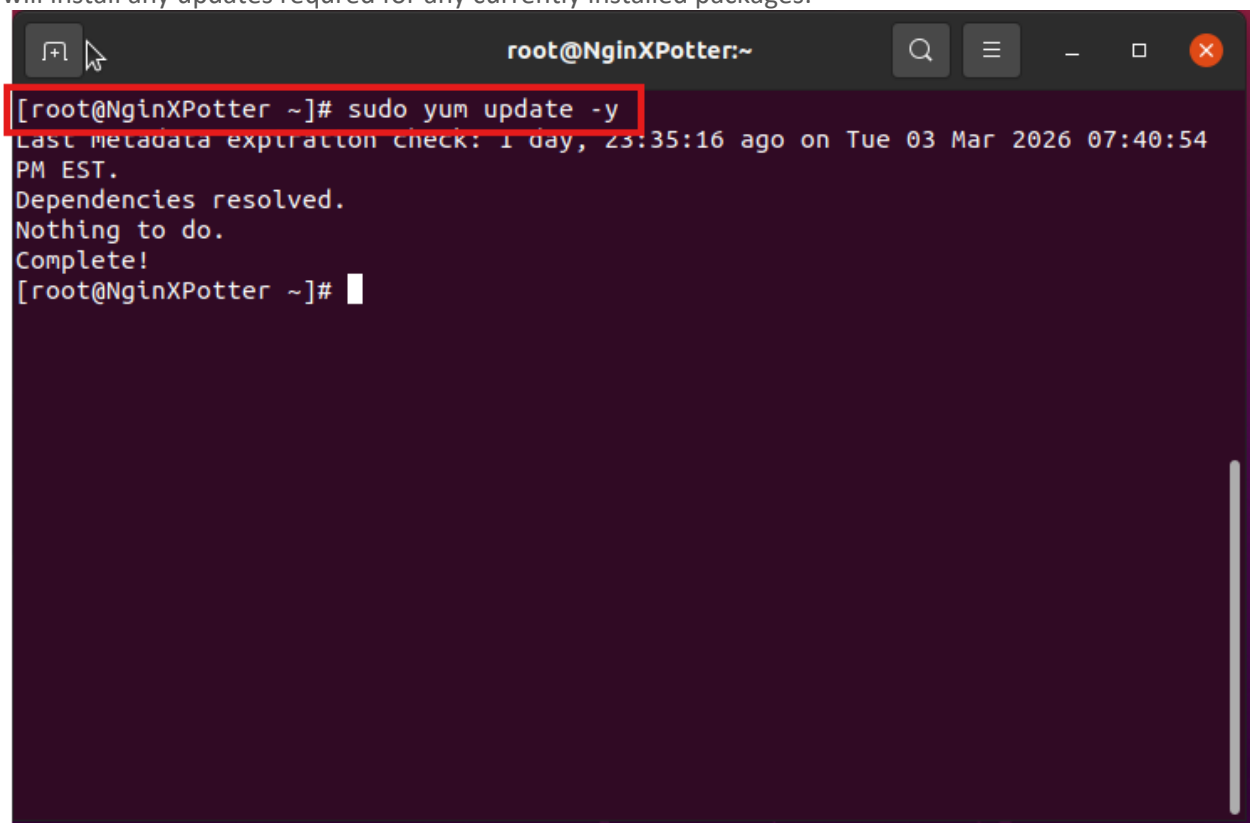

root@localhost:~
NetworkManager TUI
Please select an option
Edit a connection
Activate a connection
Set system hostname
Quit
<OK>

Then type “Exit” and relogin Using the “ssh 10.10.229.10” and Reenter the password to see the new Hostname Like Below:



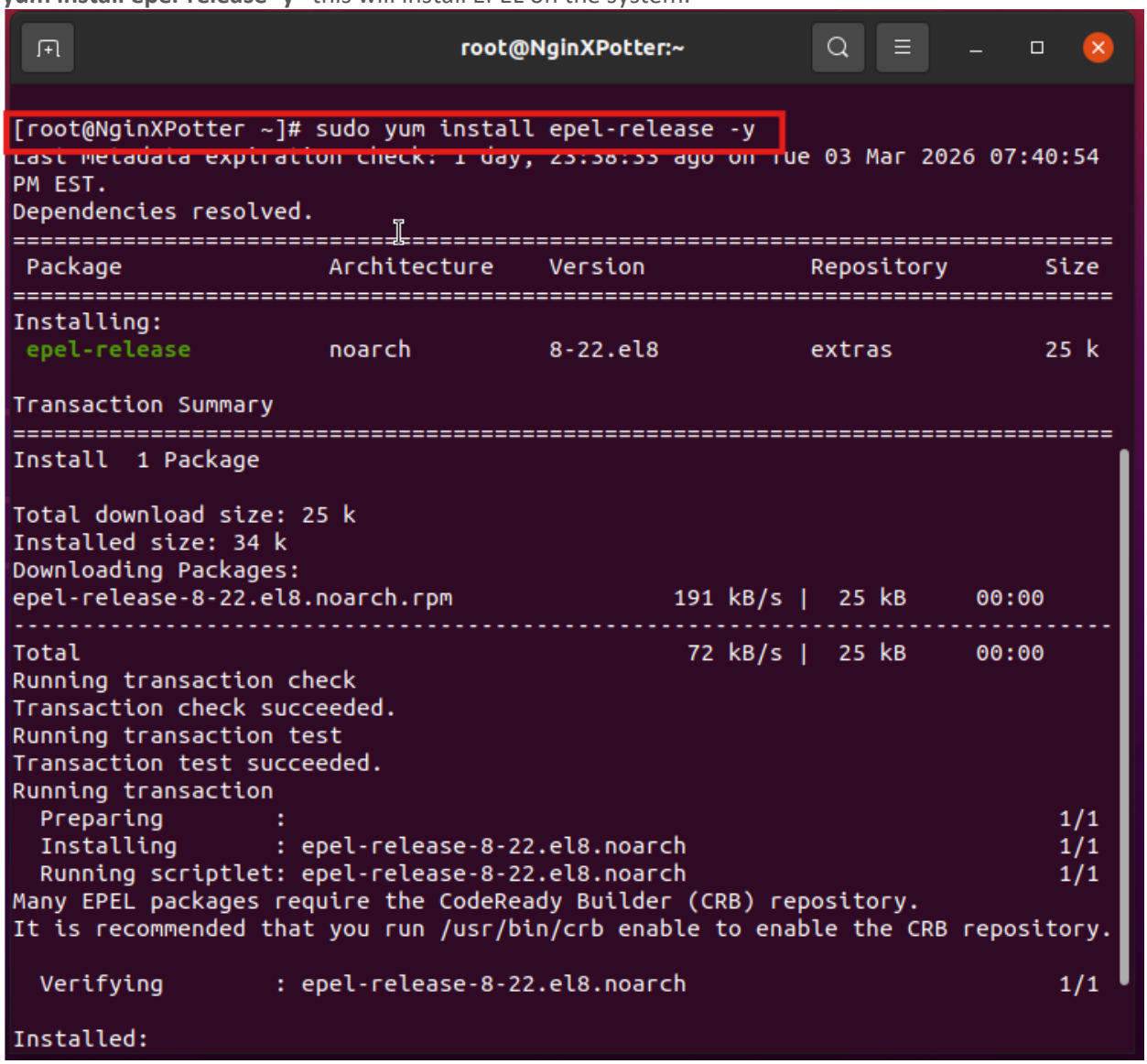
Update Rocky8-Nginx

To run the updates on our Rocky8-NginX System we simply input the command "**sudo yum update -y**" This will install any updates requred for any currently installed packages:



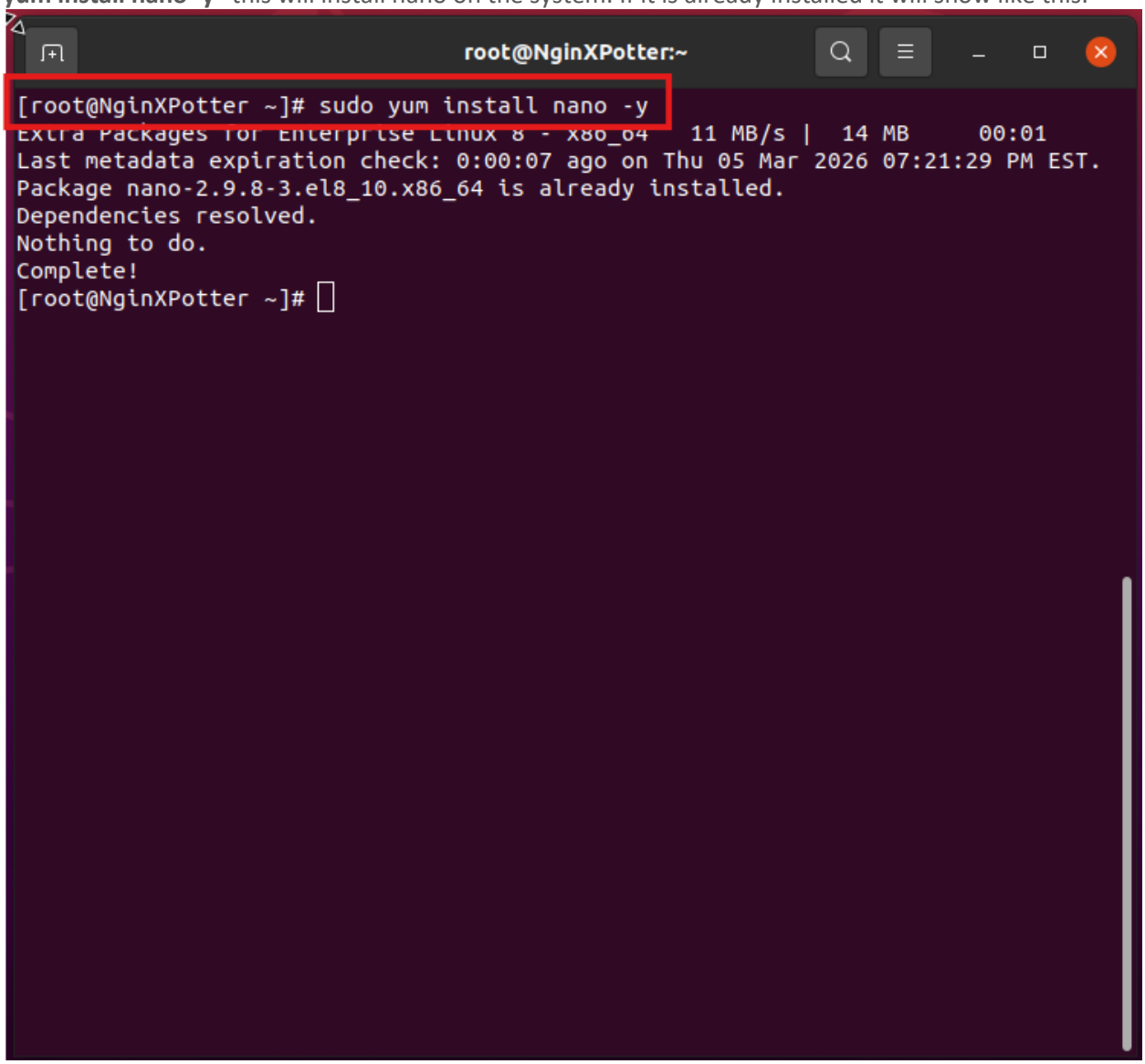
Install EPEL Packages

We now want to install EPEL packages on the NginX system so to do this we will use the command "**sudo yum install epel-release -y**" this will install EPEL on the system.



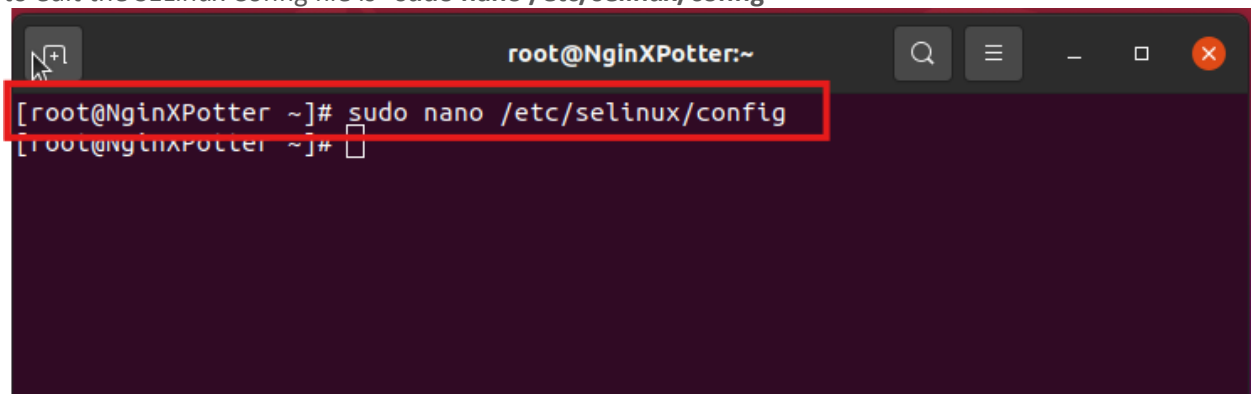
Install Nano Editor

We now want to ensure nano is installed on the NginX system so to do this we will use the command "**sudo yum install nano -y**" this will install nano on the system. If it is already installed it will show like this:

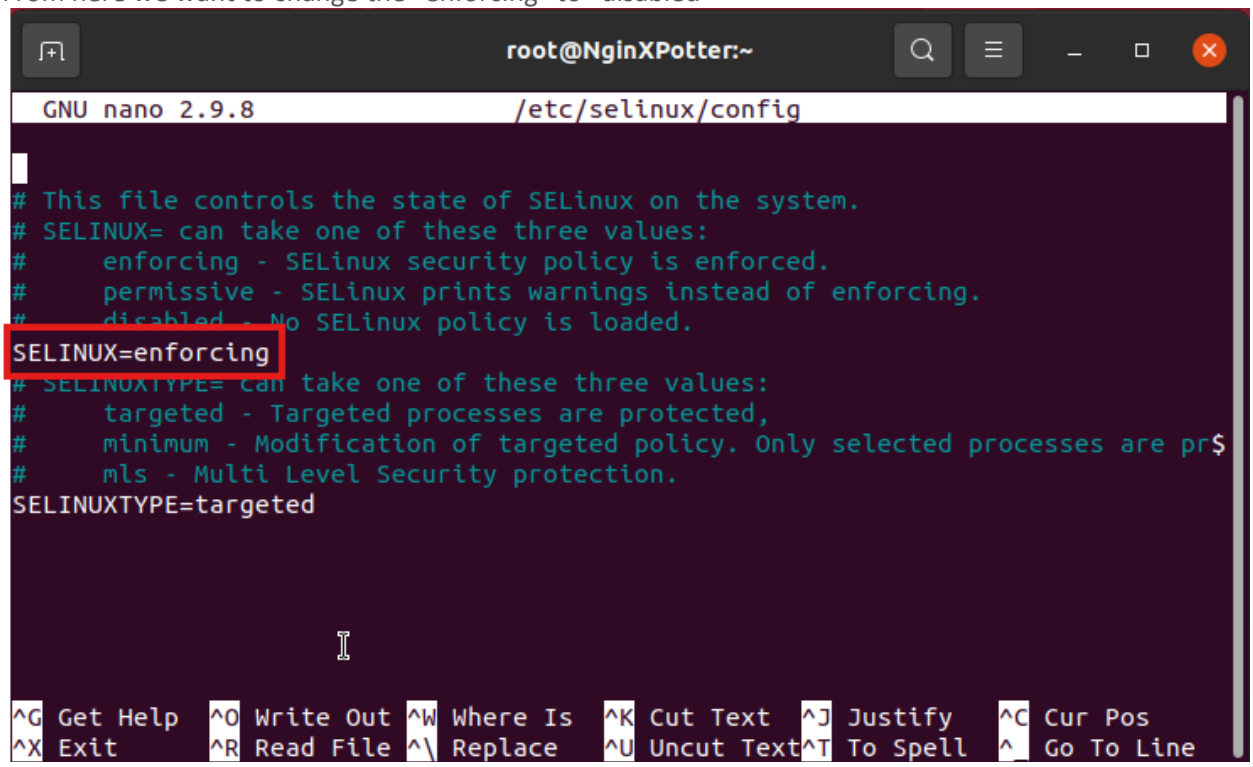


Disable SELinux

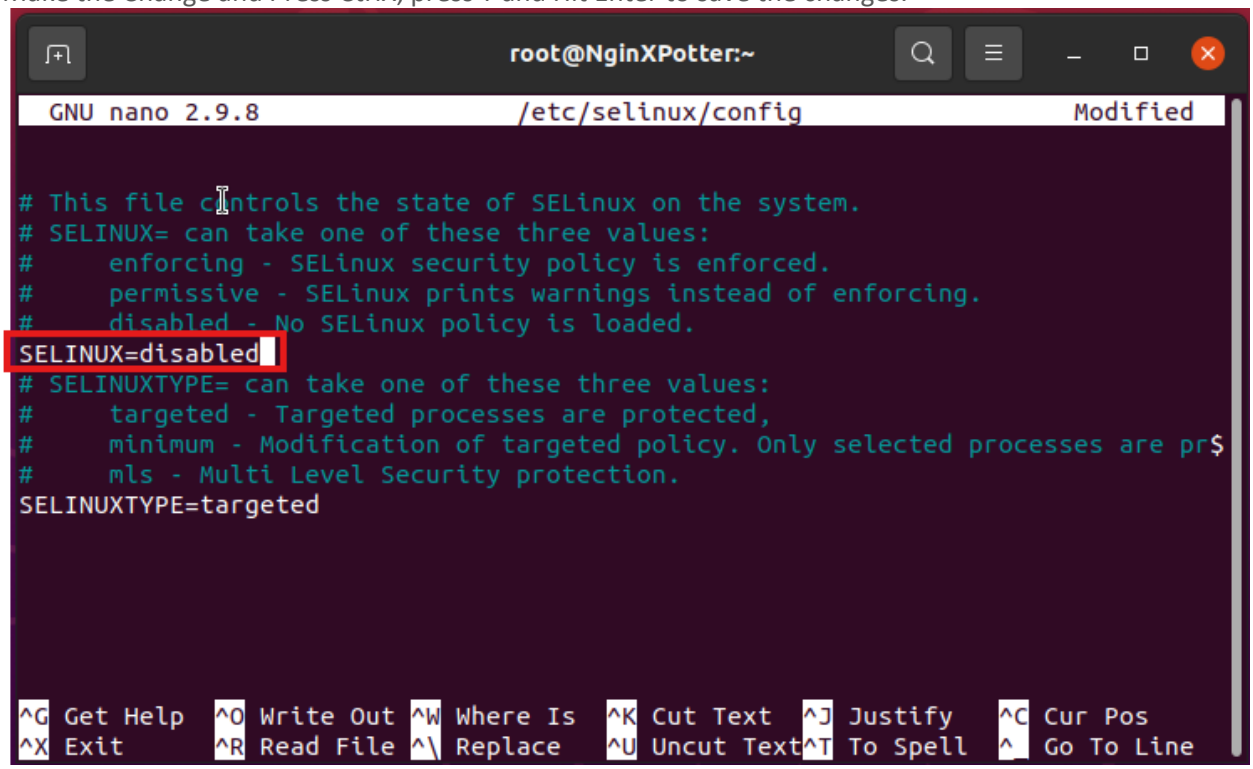
We will now be disabling SELinux in this enviroment so that we done need to login and disable it every time we want to make a change in the future. I can only recommend this in a closed environment. The command to edit the SELinux Config file is "**sudo nano /etc/selinux/config**"



From here we want to change the "enforcing" to "disabled"

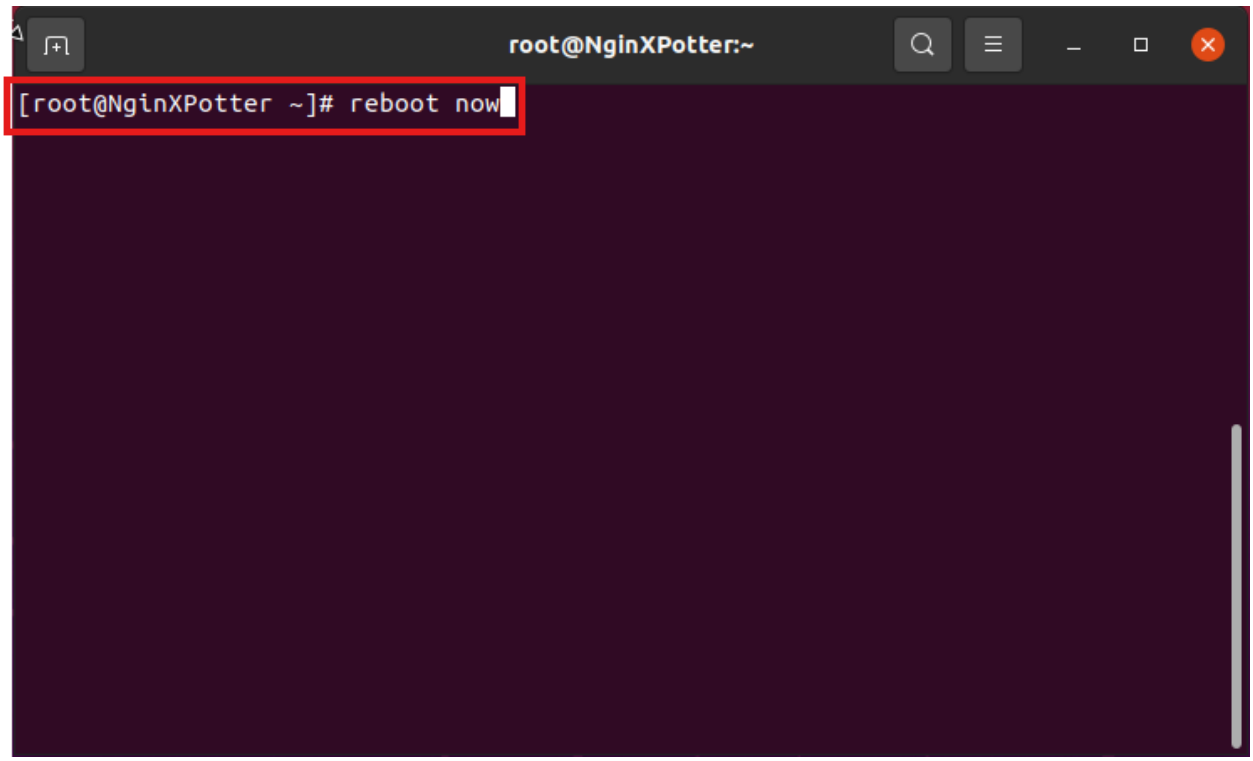


Make the Change and Press CtrlX, press Y and Hit Enter to save the changes.



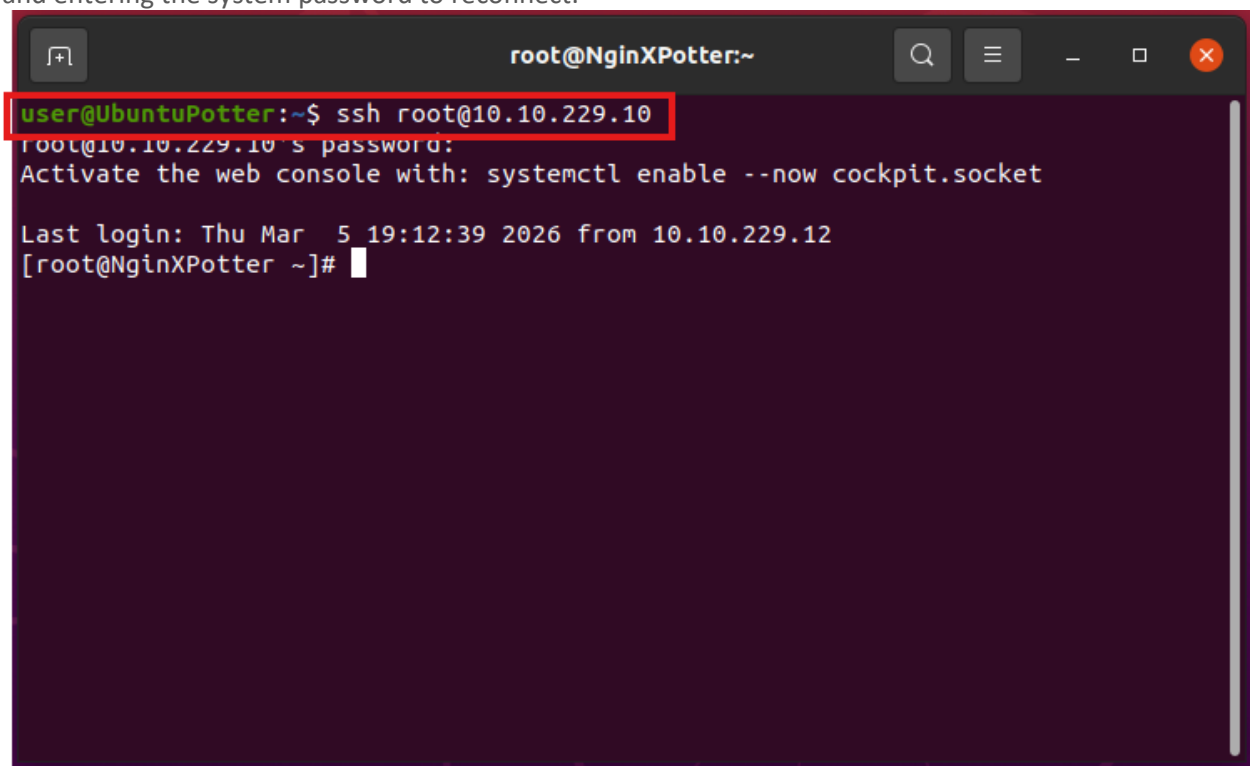
Reboot VM

We now want to reboot to make sure the SELinux changes save correctly, to do this we will use the "**Reboot now**" command:

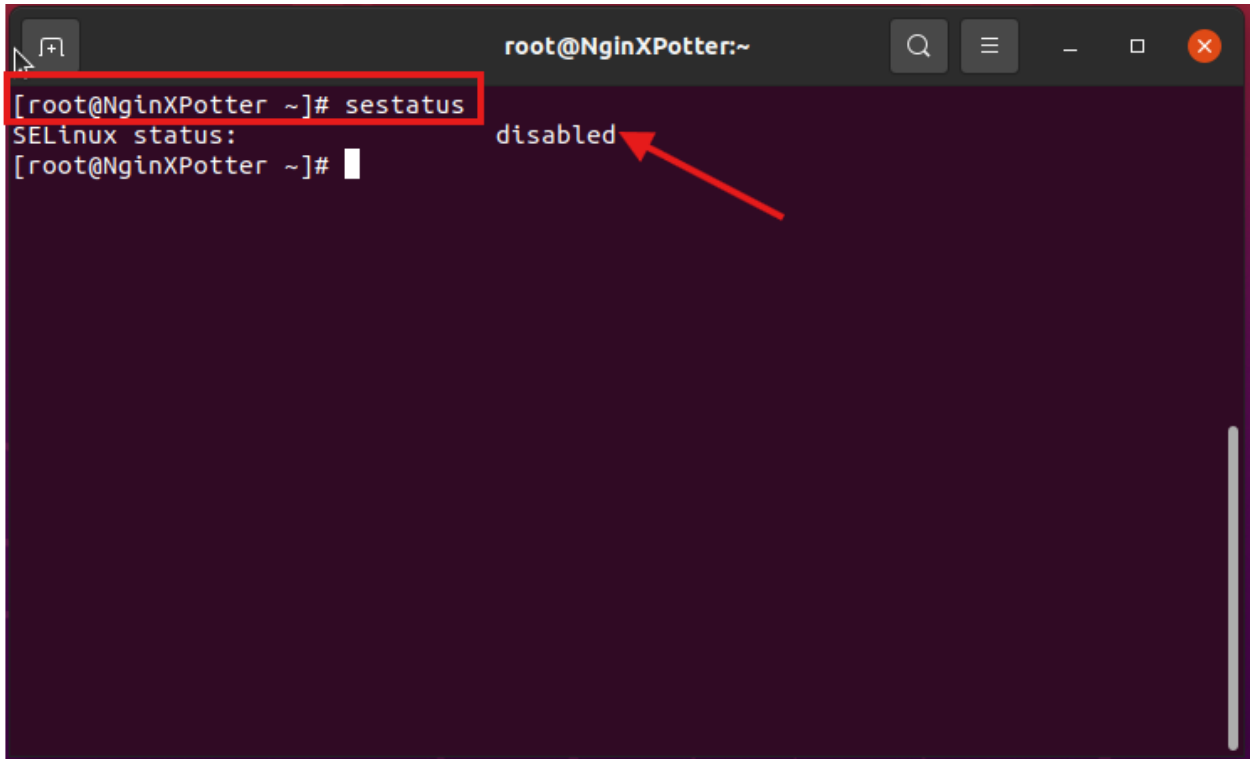


Confirm SELinux Status

After Waiting a moment for the system to reboot with will reconnect using the "**ssh root@10.10.229.10**" and entering the system password to reconnect:



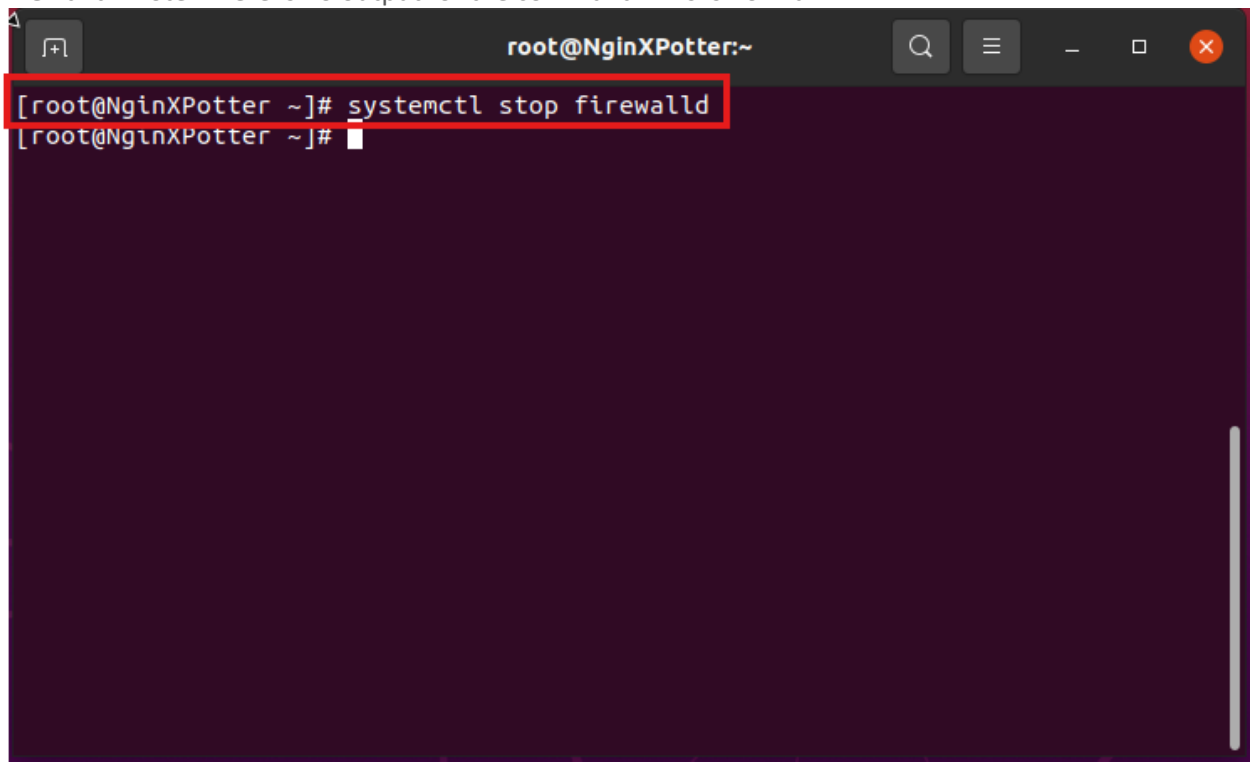
And to confirm That the SELinux is disable we will use the command "**sestatus**" the output should look like this:



Rocky Firewall

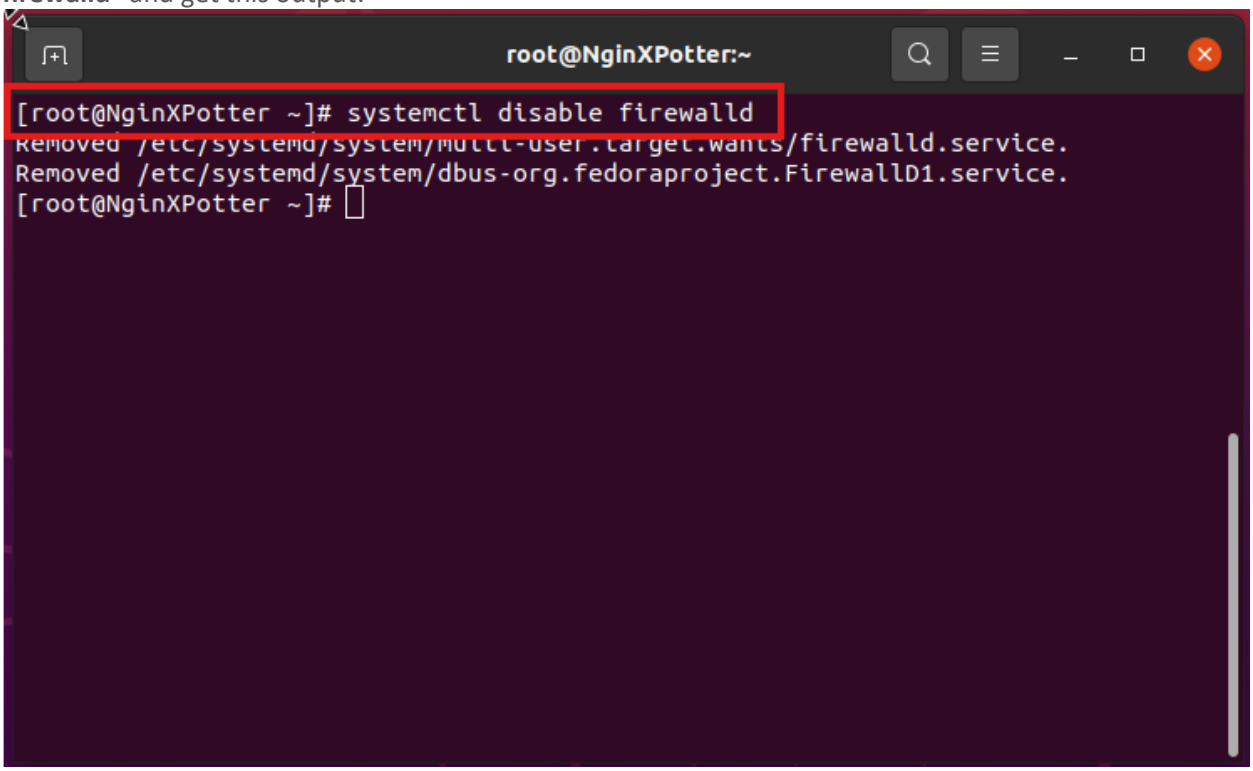
Stop Firewall

We now need to Stop the firewall on the NginX to do this we will enter the command "**systemctl stop firewalld**" Note: There is no output for the command. This is normal.



Disable Firewall

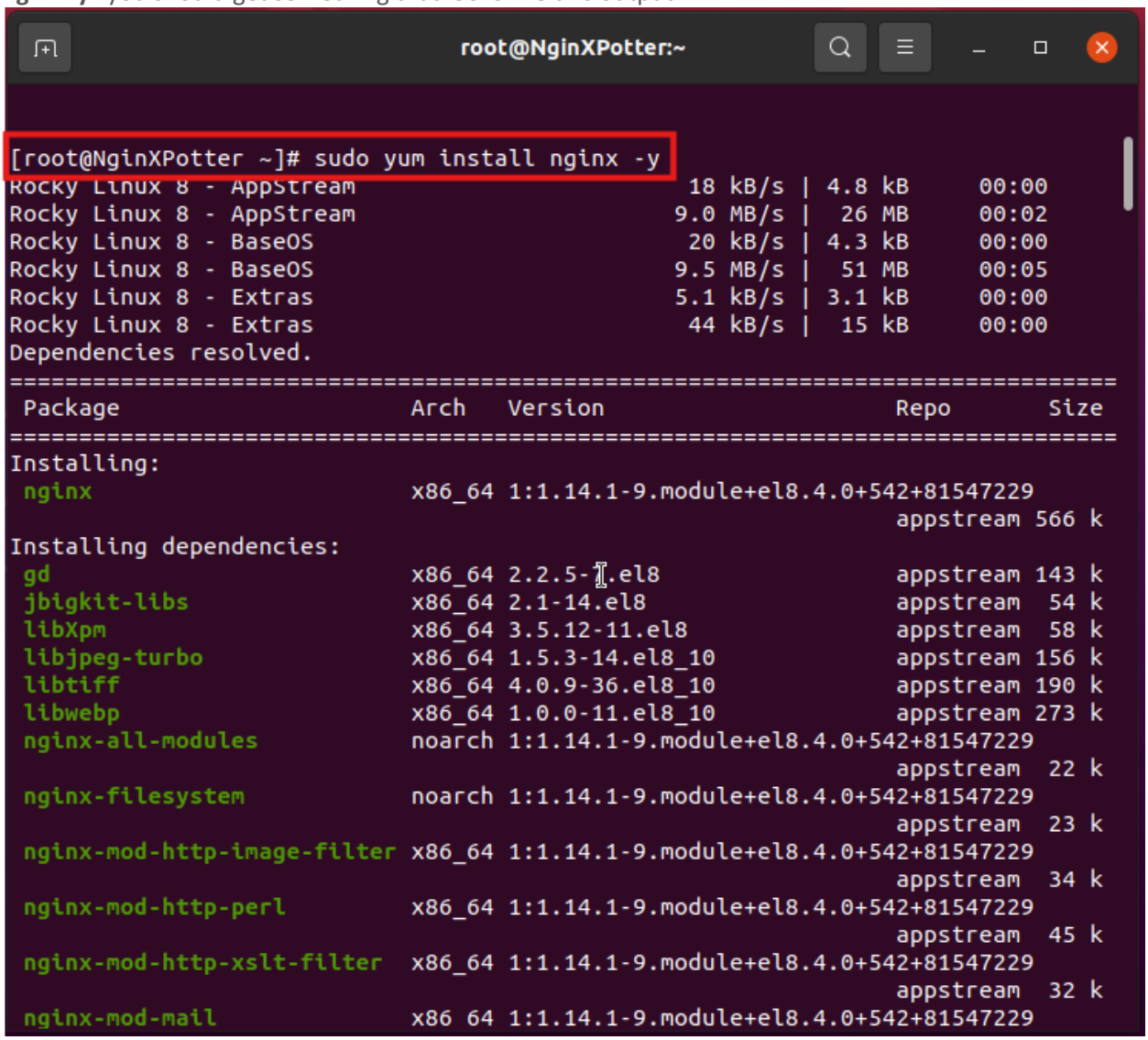
We now need to disable the firewall on the NginX to do this we will enter the command "**systemctl disable firewalld**" and get this output:



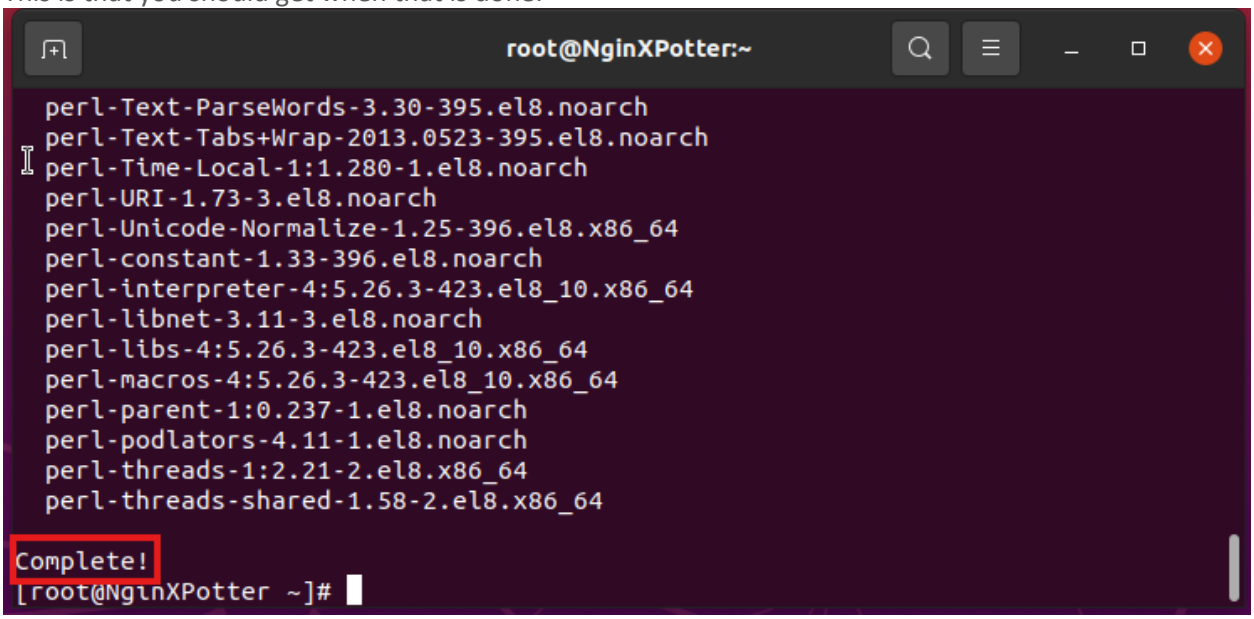
NginX

Install NginX

We will now install NginX on this System we will do this by entering the command "**sudo yum install nginx -y**" you should get something that looks like this output:

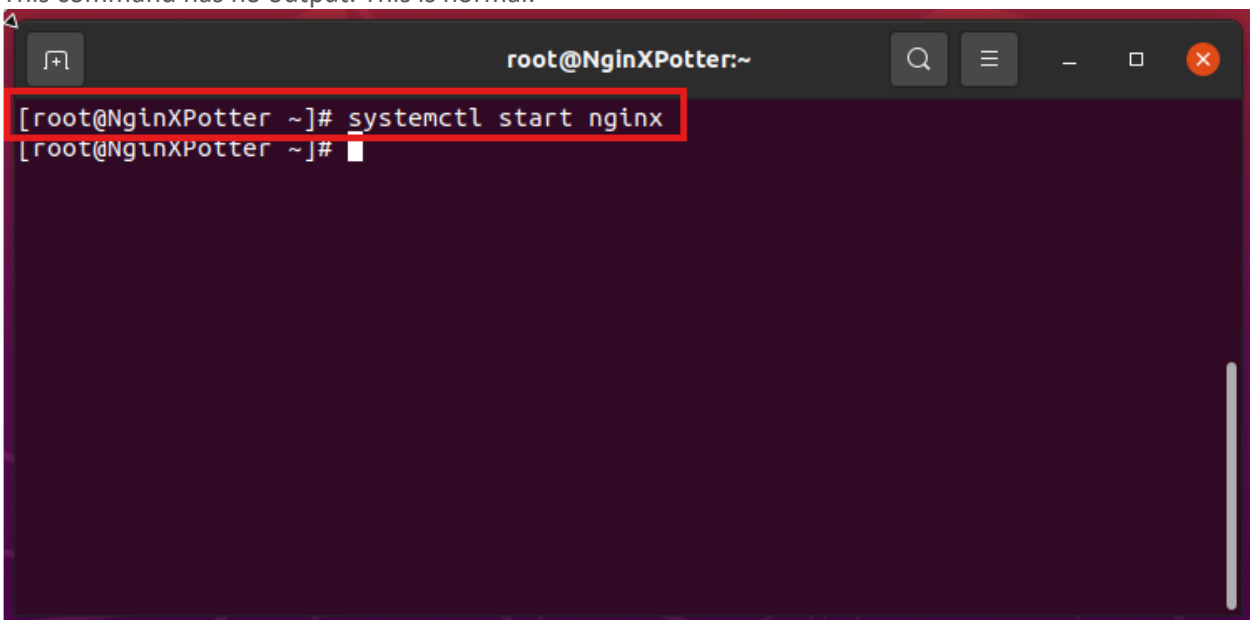


This is that you should get when that is done:



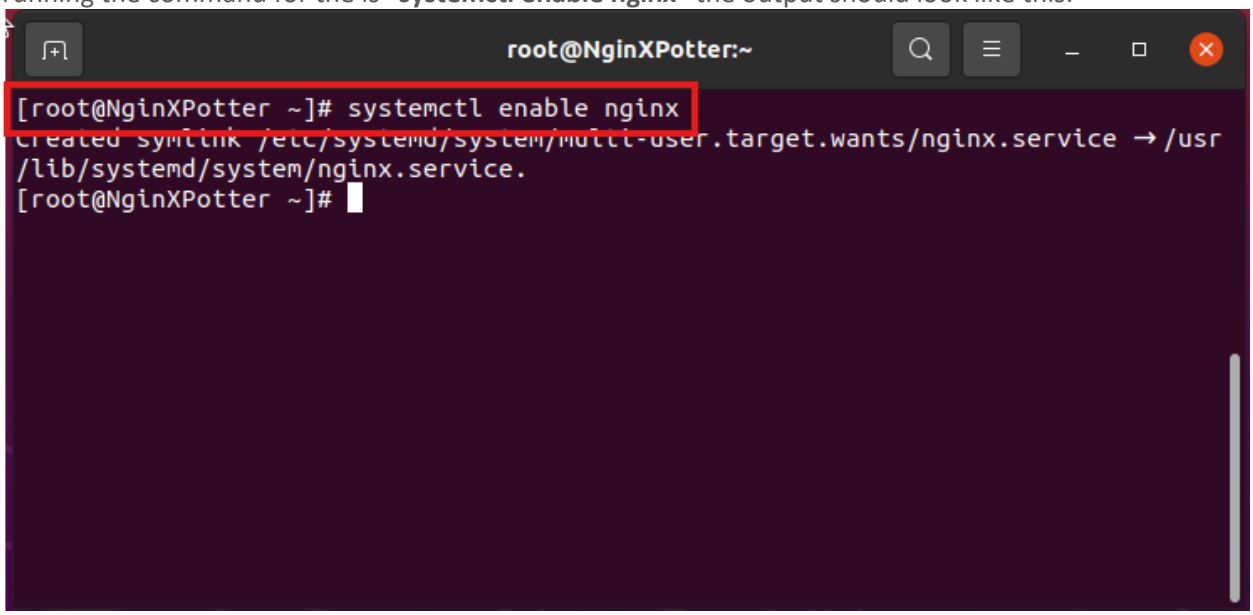
Start NginX

Once The installation is complete we need to start NginX we do this using the "**systemctl start nginx**" Note: This command has no output. This is normal.



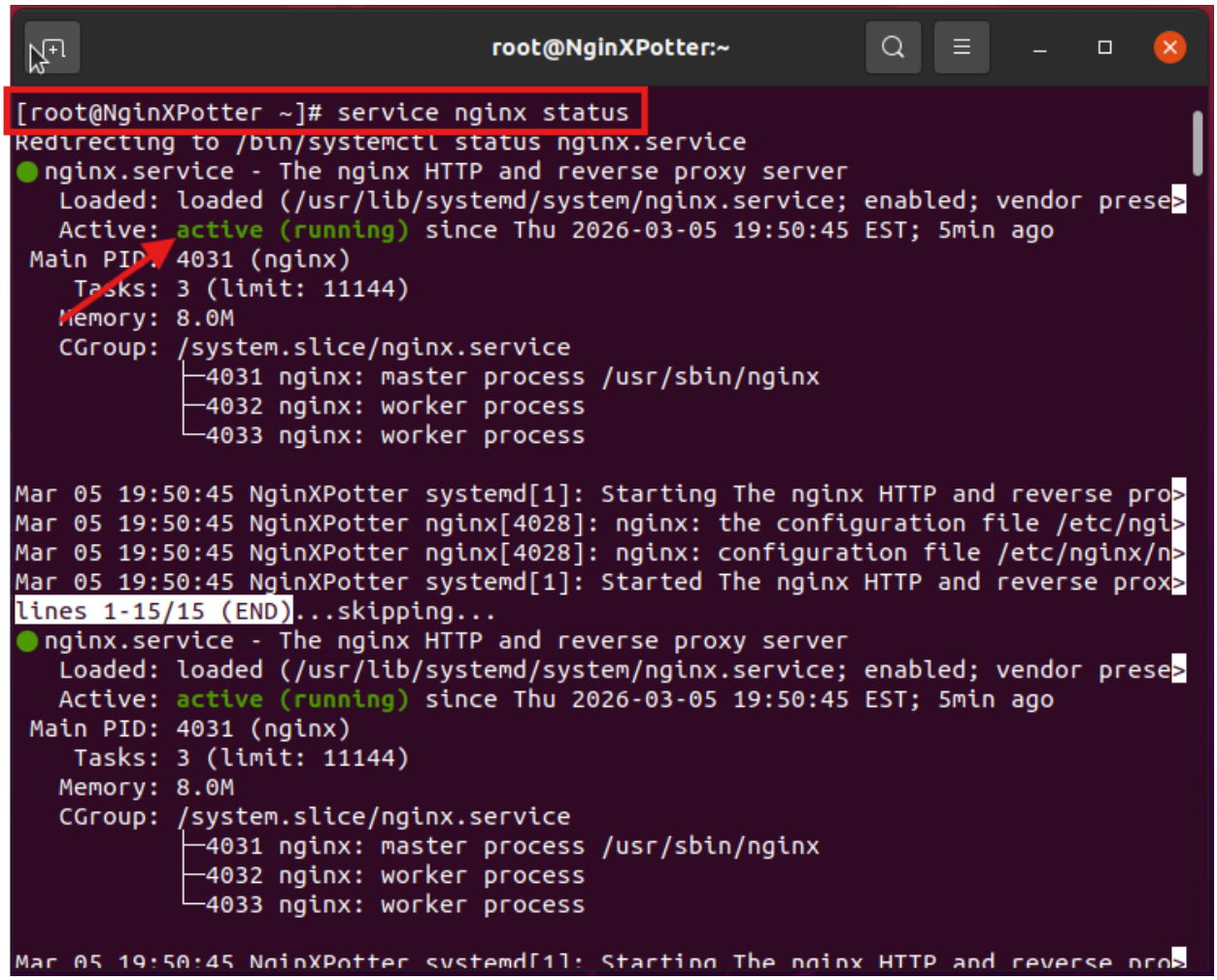
Enable NginX

We now need to enable NginX on the system so that it starts automatically whenever the system is running the command for the is "**systemctl enable nginx**" the output should look like this:



Confirm NginX Status

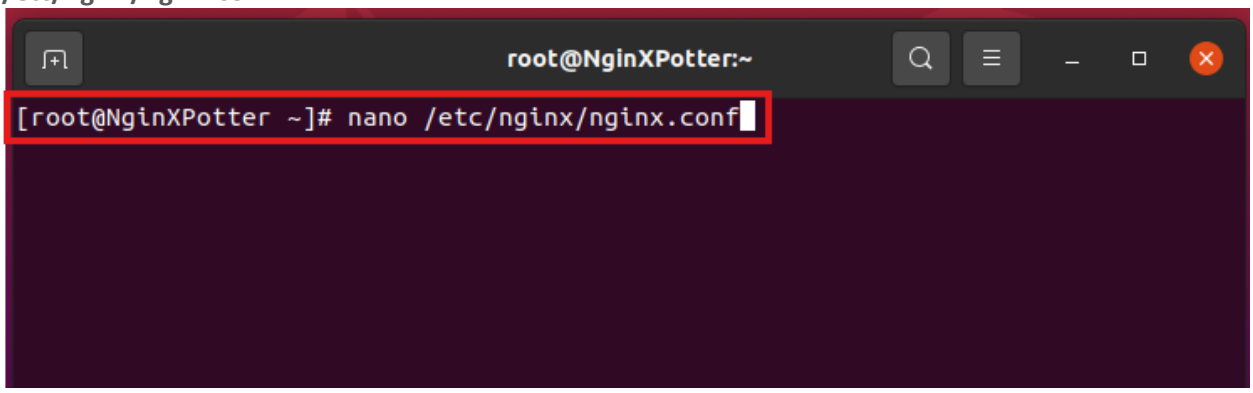
We now want to ensure nginx is running we will use the command "**service nginx status**" you should see this:



Reverse Proxy for Ghost Site

Edit NginX configuration file

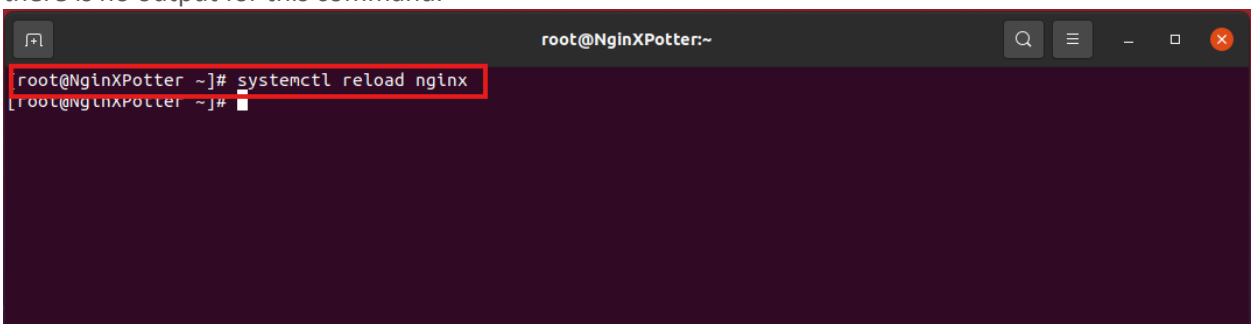
We now need to edit the configuration file for nginx, we will use the command "**nano /etc/nginx/nginx.conf**"



The Configurations Changes that need to be made are in Appendix A

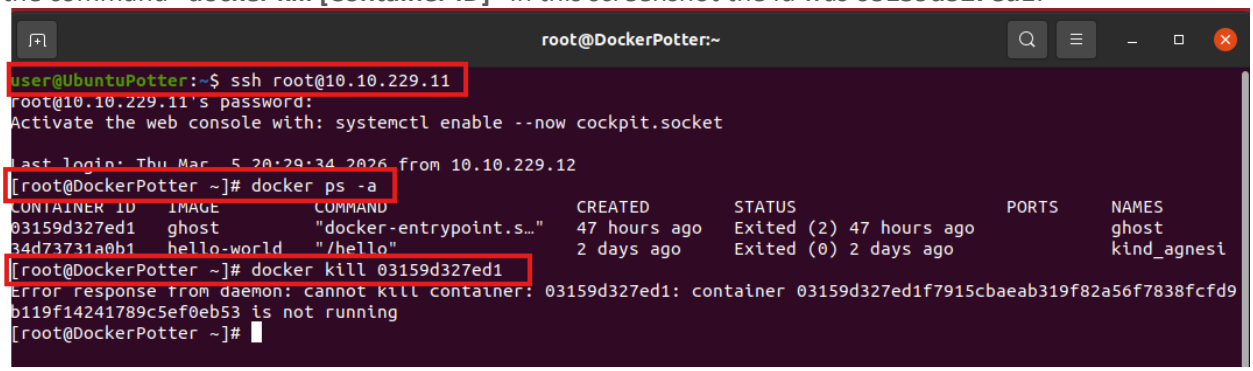
Reload NginX service

After the changes are made we need to reload nginx using the command "**systemctl reload nginx**" Note: there is no output for this command.



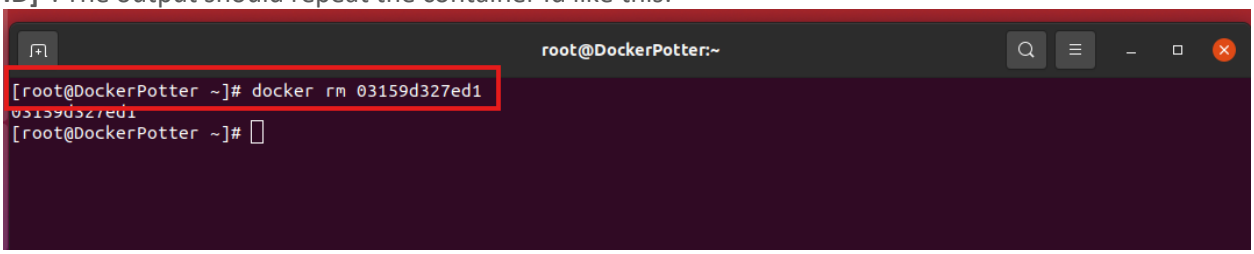
Terminate Docker

We now want to move back to our Docker system. Use the command "**exit**" to log out of the NginX system and log back into the docker with the command "**ssh root@10.10.229.11**". Once logged in, Use the command "**docker ps -a**" this will give you the container ID you will need for the next command. Use the command "**docker kill [Container ID]**" in this screenshot the Id was **03159d327ed1**:



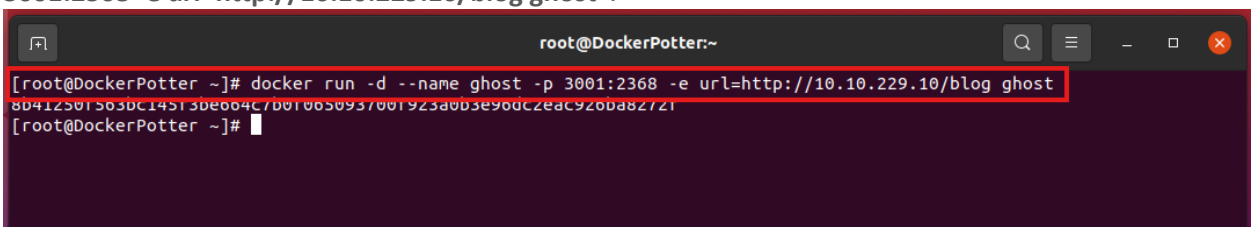
Delete First Ghost Container

We then want to remove the Ghost container we just stoped the command for this is "**docker rm [Container ID]**". The output should repeat the container Id like this:



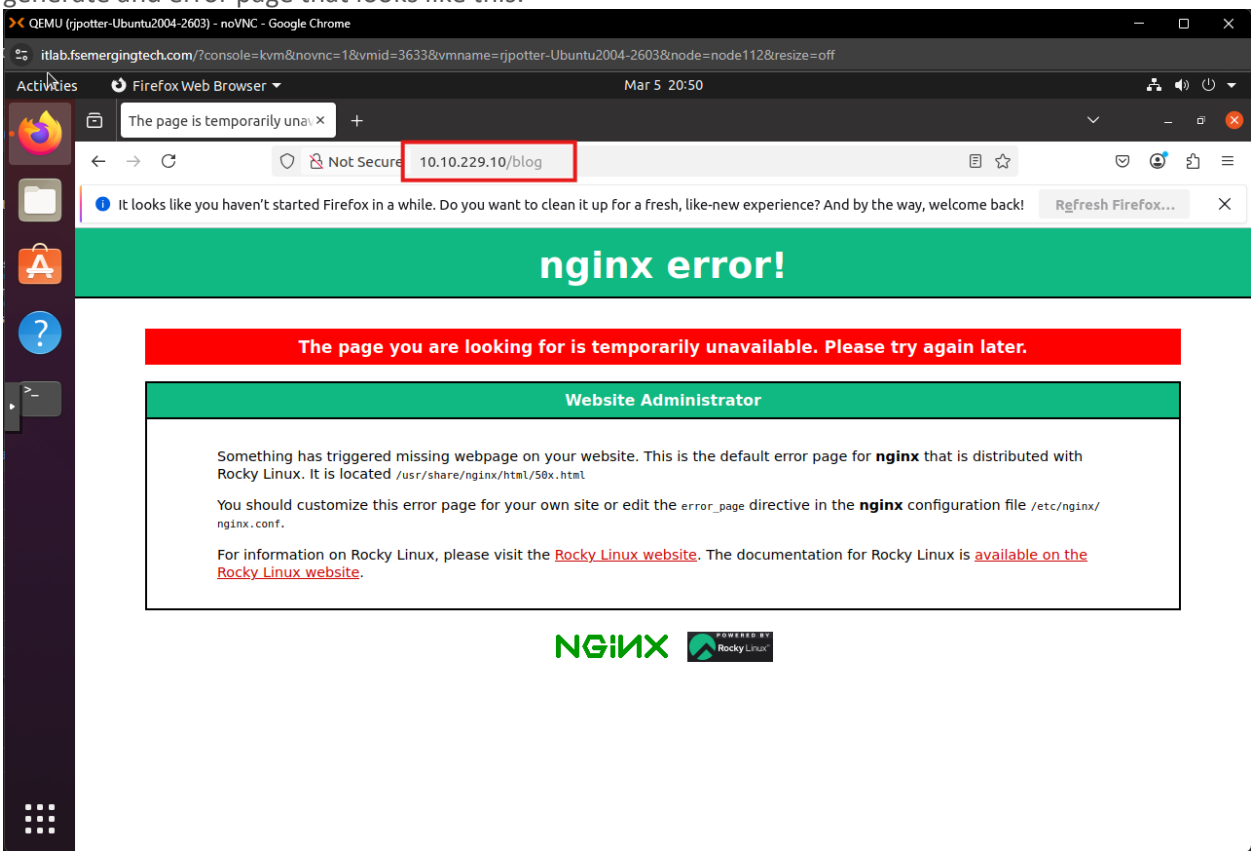
Create New Ghost Container

We will then Create a new ghost container using the following command "**docker run -d --name ghost -p 3001:2368 -e url=http://10.10.229.10/blog ghost**".



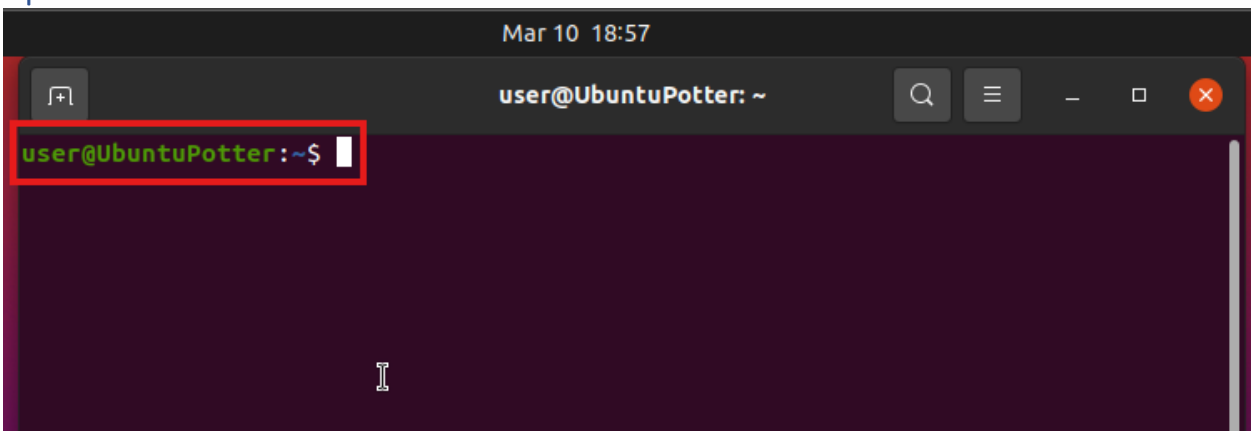
Browse to Ghost from Firefox on Ubuntu (10.10.229.10/blog)

We now want to test that our nginx config worked correctly and that the gost contained is created and running we will do this by opening firefox and navigating to the address “10.10.229.10/blog” This should generate and error page that looks like this:



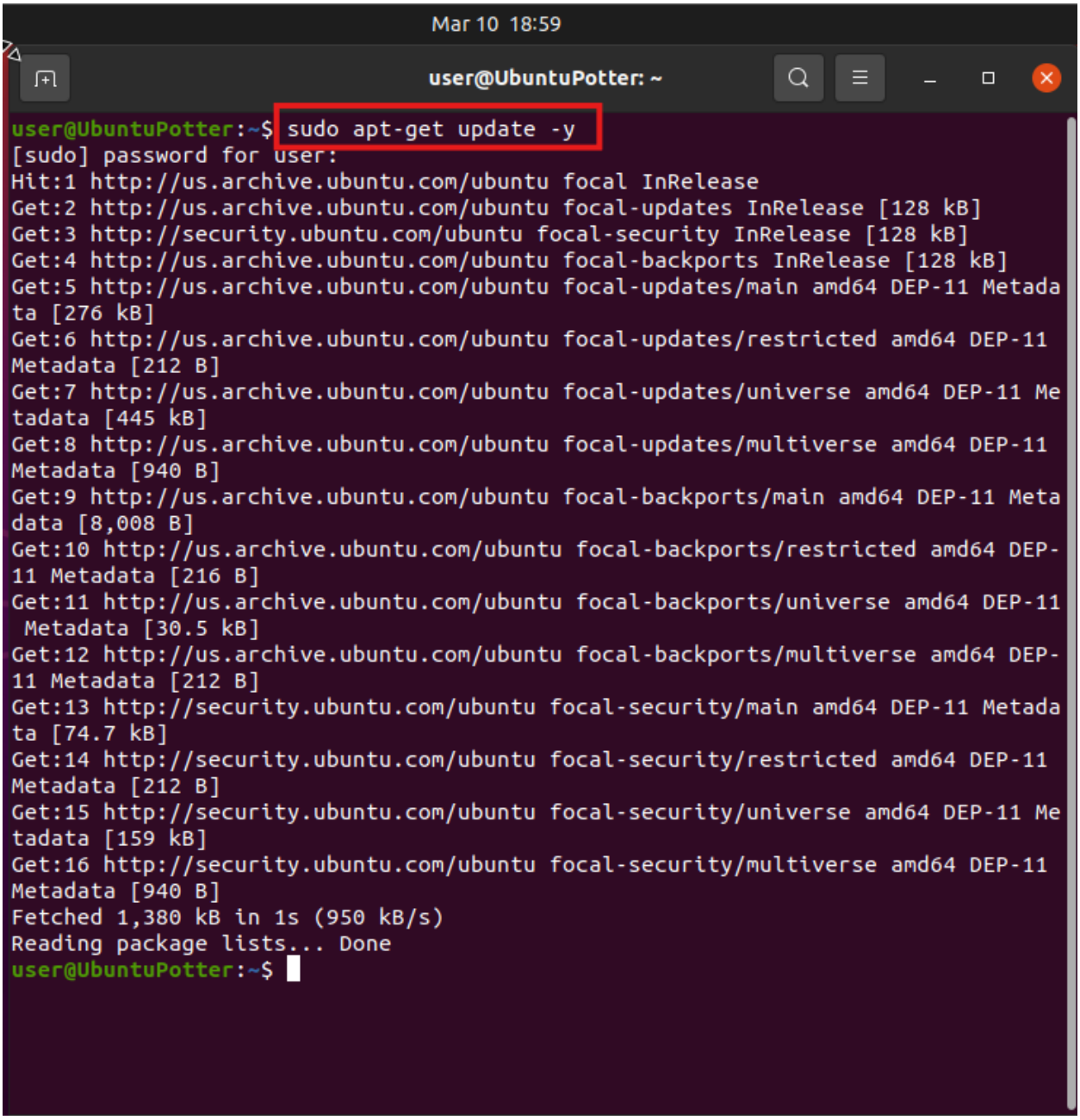
WordPress on Ubuntu - LAMP Stack

Update Ubuntu Host Name



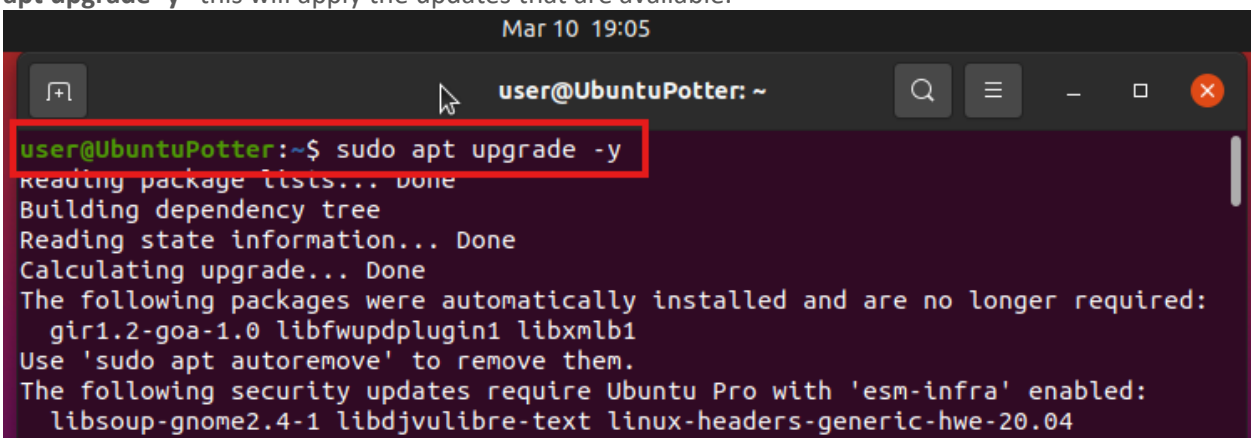
Update Ubuntu

So before we start our installation of the LAMP stack we first want to make sure that our operating system is up to date. We will do this by using the command "**sudo apt-get update -y**". The output may look like below:



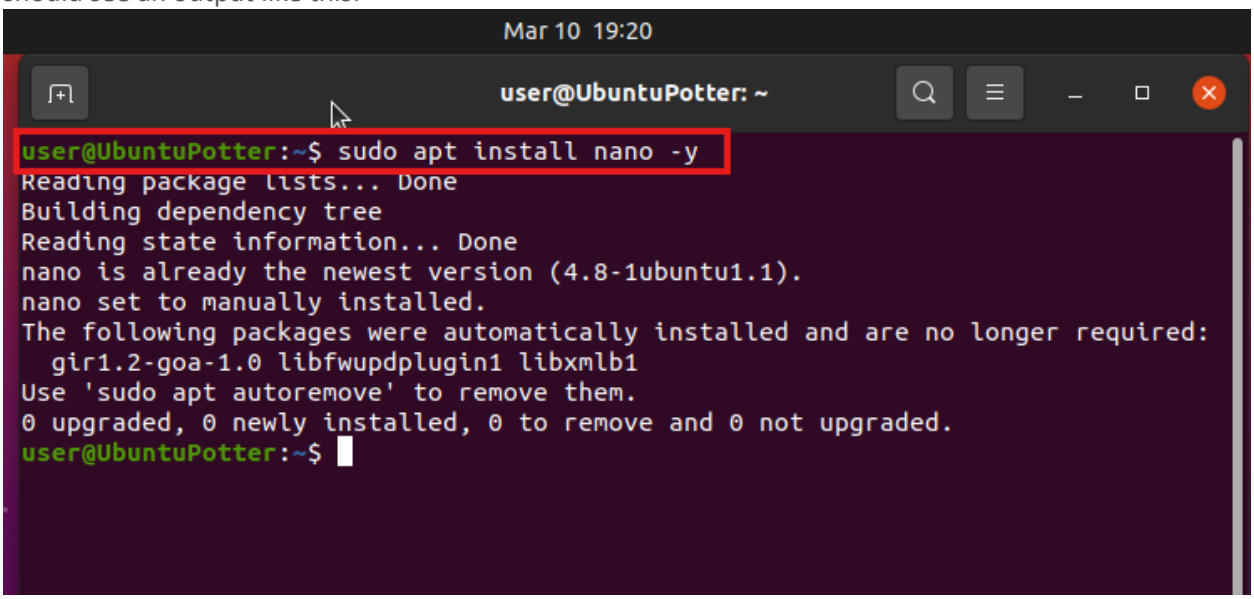
Upgrade Ubuntu

After the above command in order to apply the updates it may have found we will use the command "**sudo apt upgrade -y**" this will apply the updates that are available.



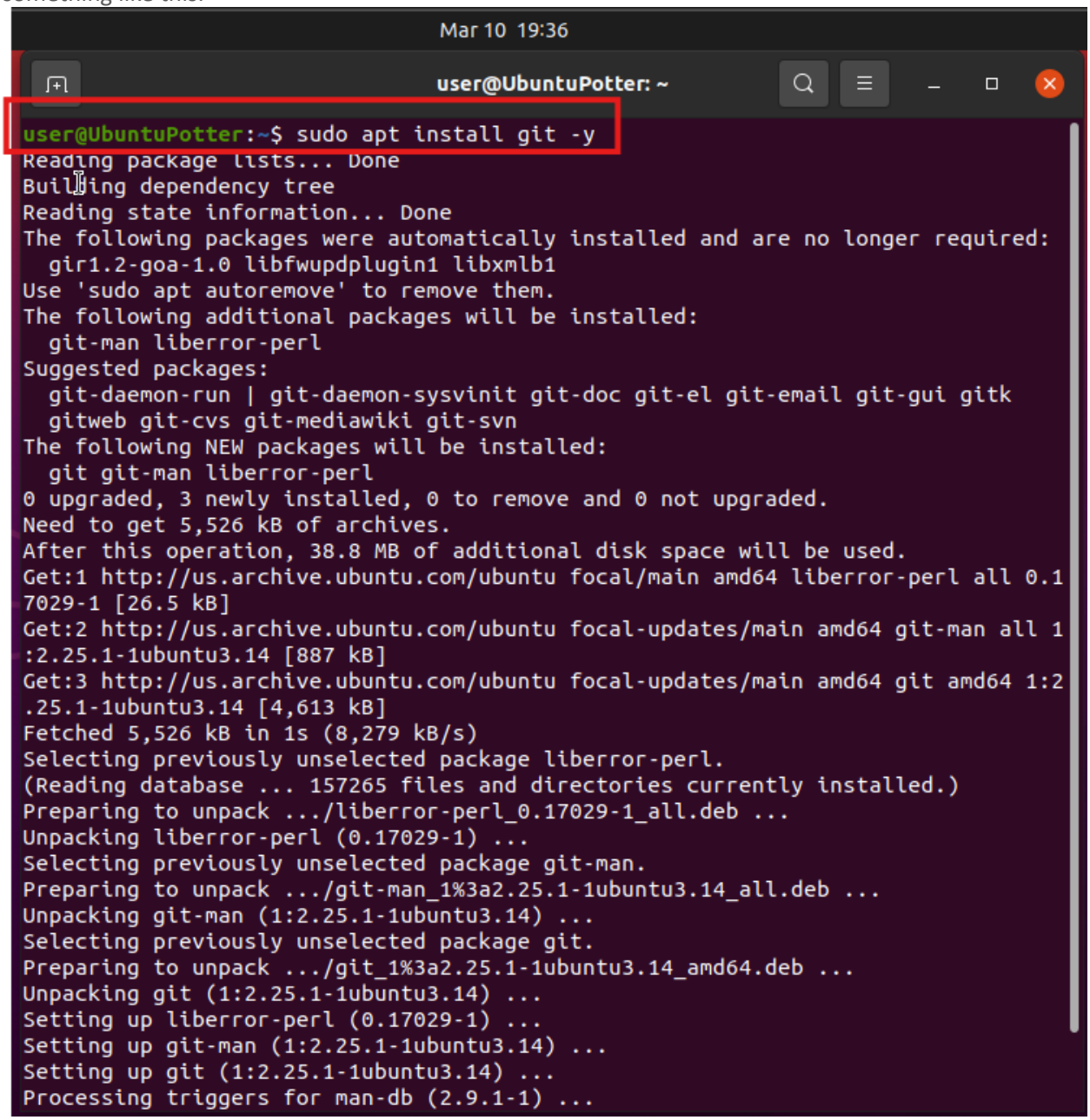
Install Nano Editor

Next, We will be installing nano which is a command line text editor that we will be using later to edit some configuration file we need to change later we will do this with the command "**sudo apt install nano -y**". You should see an output like this:



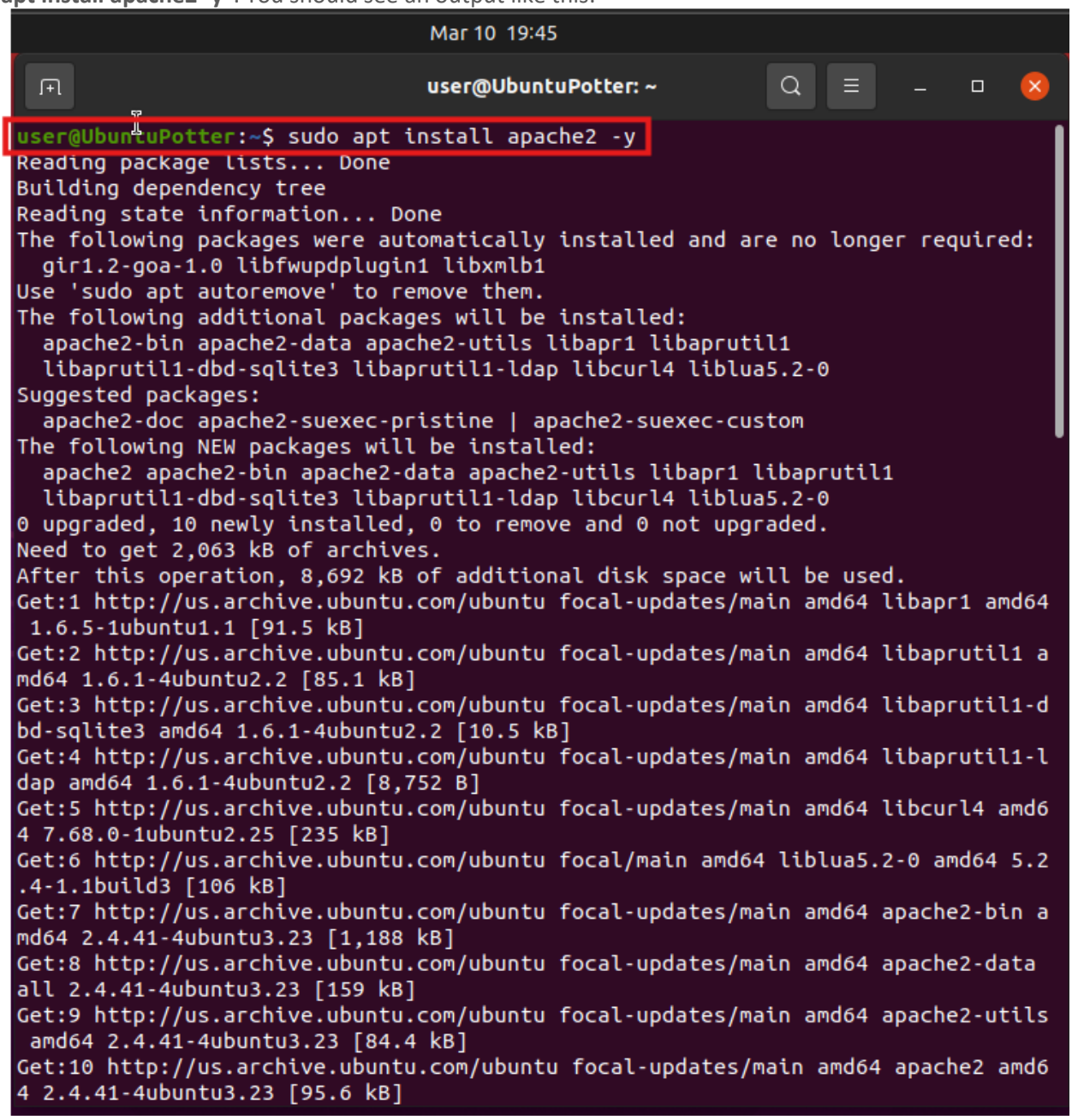
Install Git

Next, We will be installing Git we will be using this later in order to download templates and some later steps in this project. The command we will use for this is "**sudo apt install git -y**", the output should look something like this:



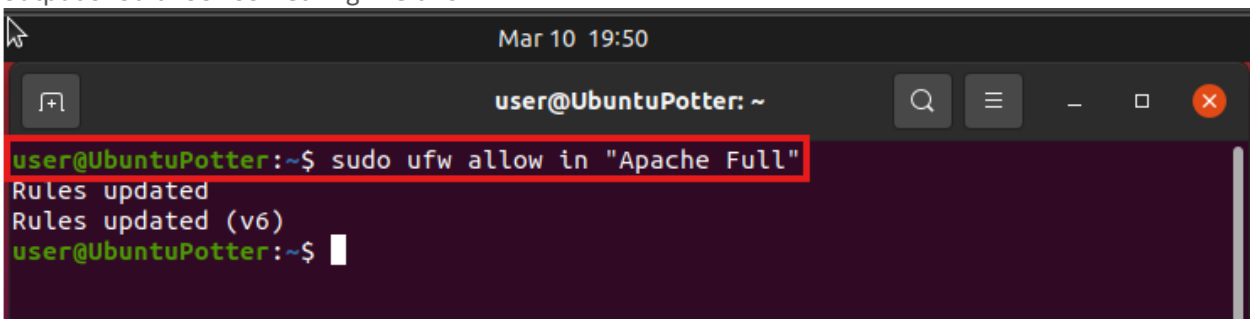
Install Apache2

Our next step is to install Apache2 on our Unbuntu machine, we will do this using the command "**sudo apt install apache2 -y**". You should see an output like this:



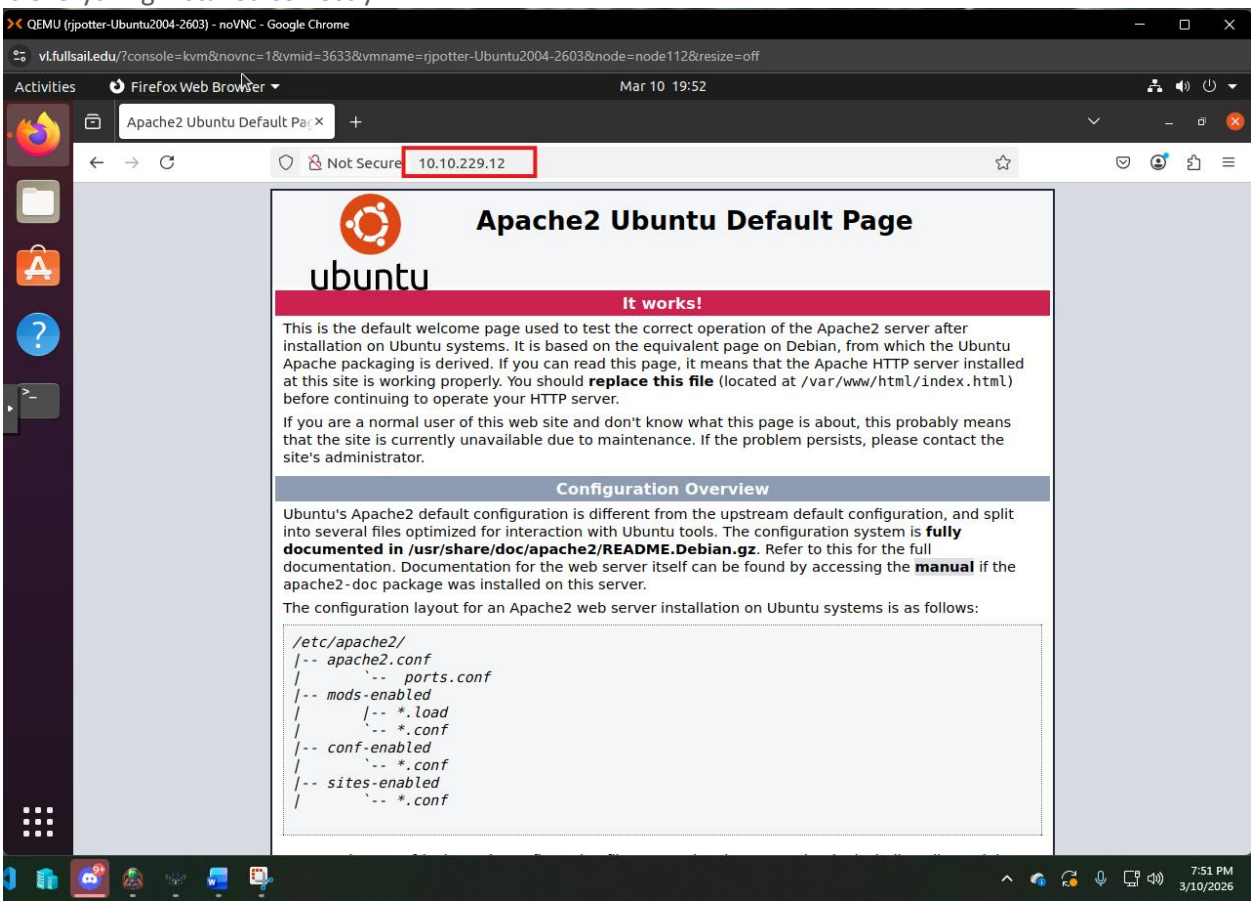
Open Firewall Ports 80 and 443

We now need to open Ports 80 and 443 for web traffic to be able to get to our web server, in order to do this we will use the command "**sudo ufw allow in "Apache Full"**". This adds Rules to our firewall and the output should look something like this:



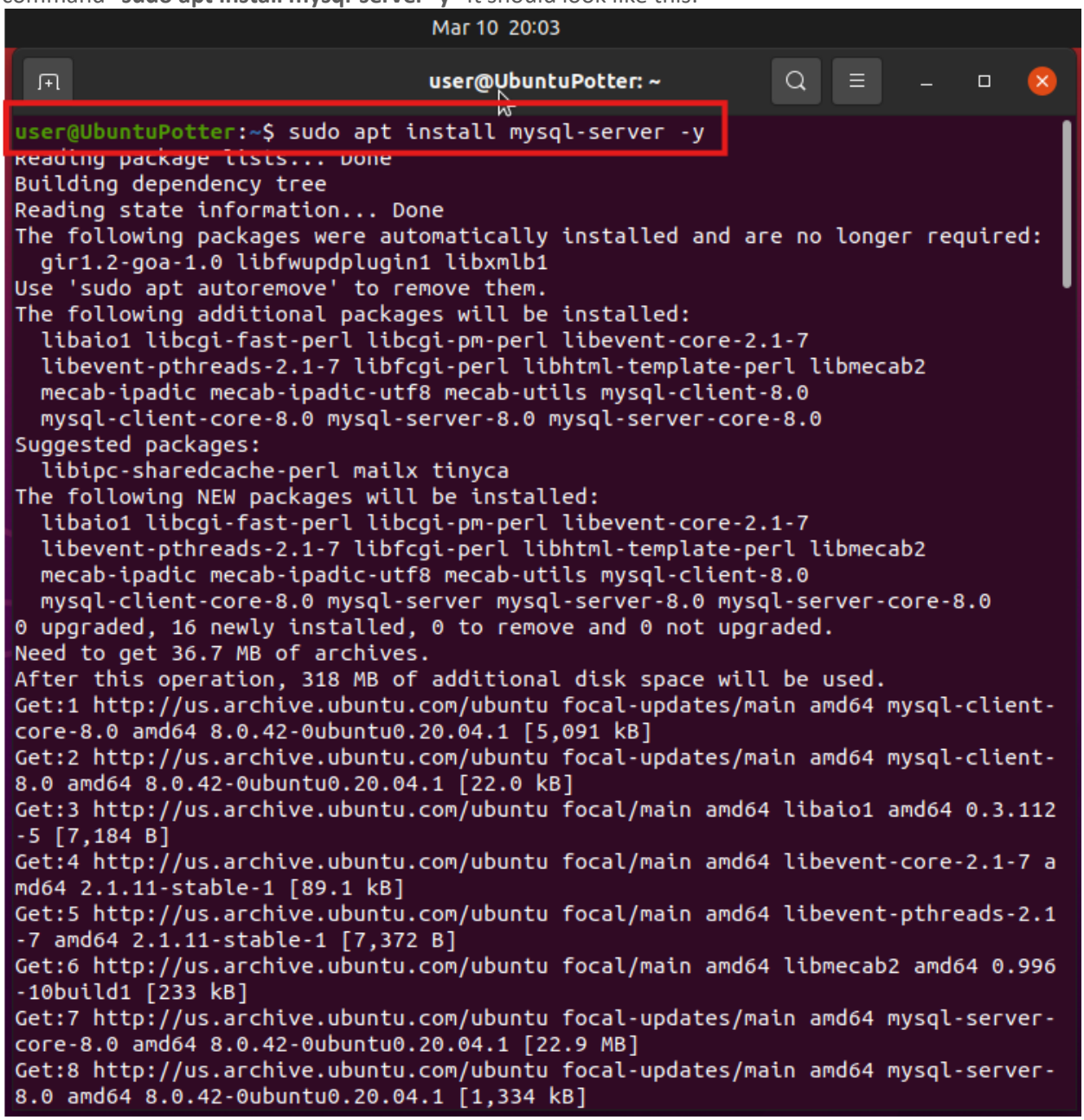
Browse to Apache2 Ubuntu Default Page

Now, we will test to make sure that our firewall rule applied correctly and that we can access our Apache2 install. To do this open the Firefox Brower and navigate to **http://10.10.229.12:80**. You should see this page is everything installed correctly:



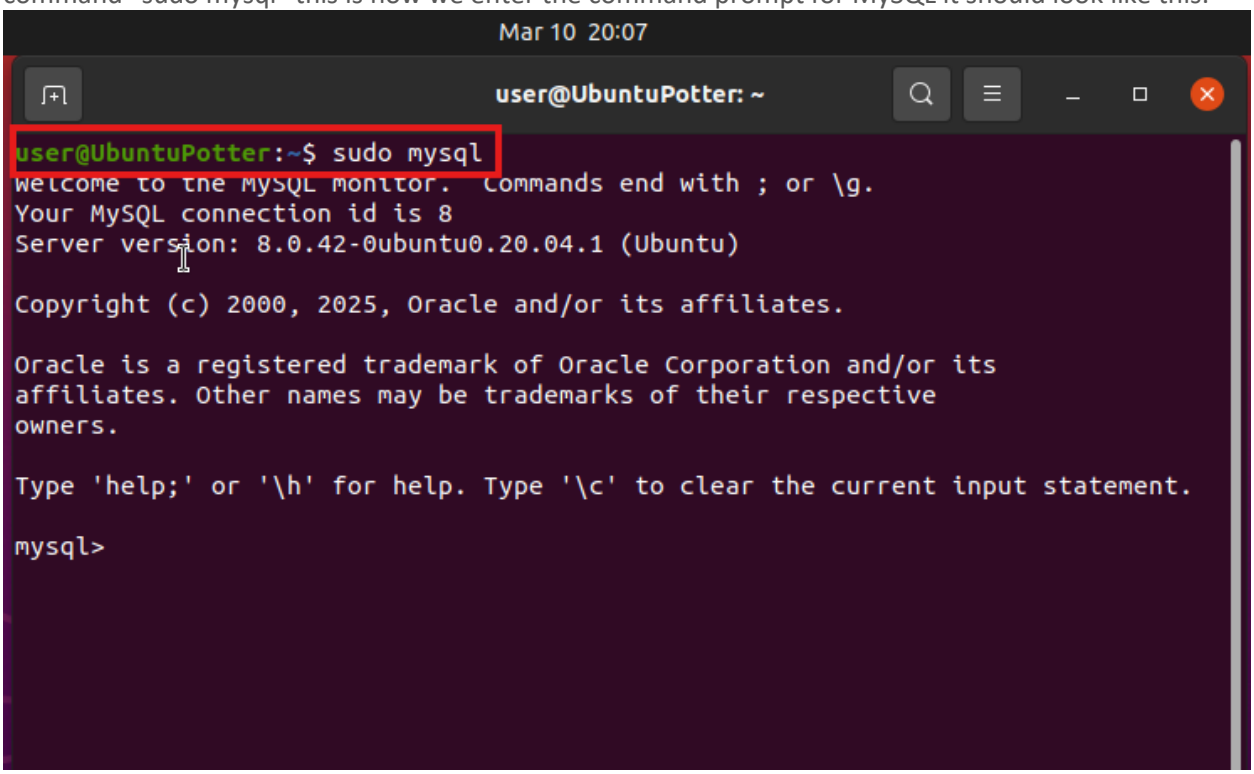
Install MySQL

Our next step is to install the MySql Server Database on out Lamp Stack. We will start this ny entering the command "**sudo apt install mysql-server -y**" It should look like this:

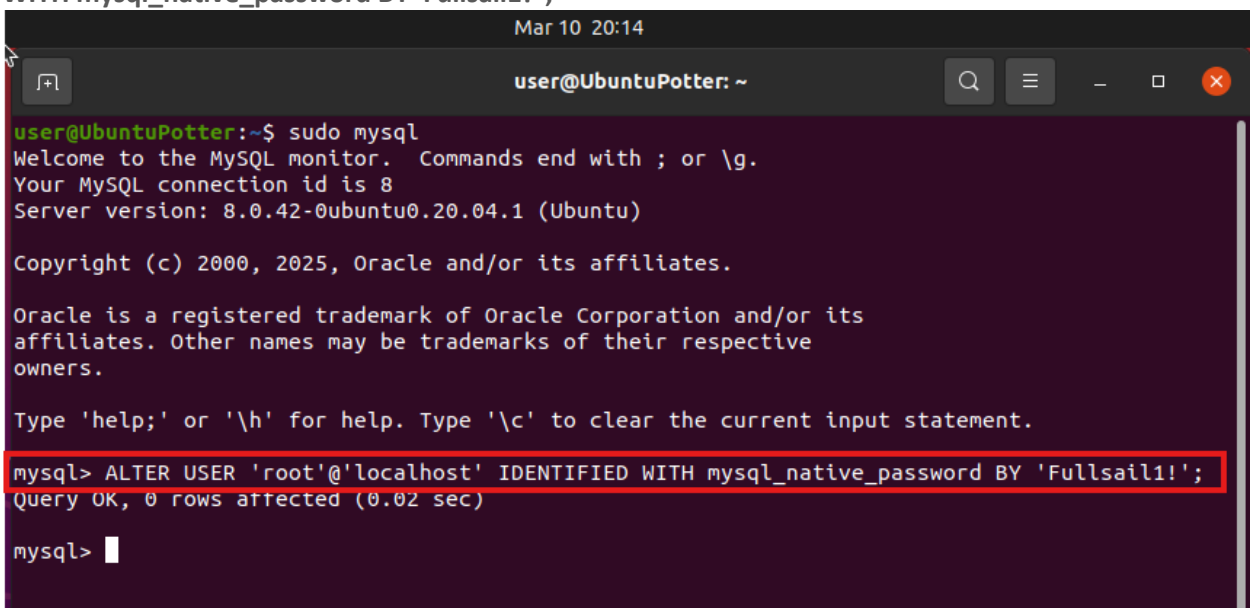


Alter root user password (root@localhost)

Once this is installed, we and to login and change the root user password, the first step in this process is the command “sudo mysql” this is how we enter the command prompt for MySQL it should look like this:

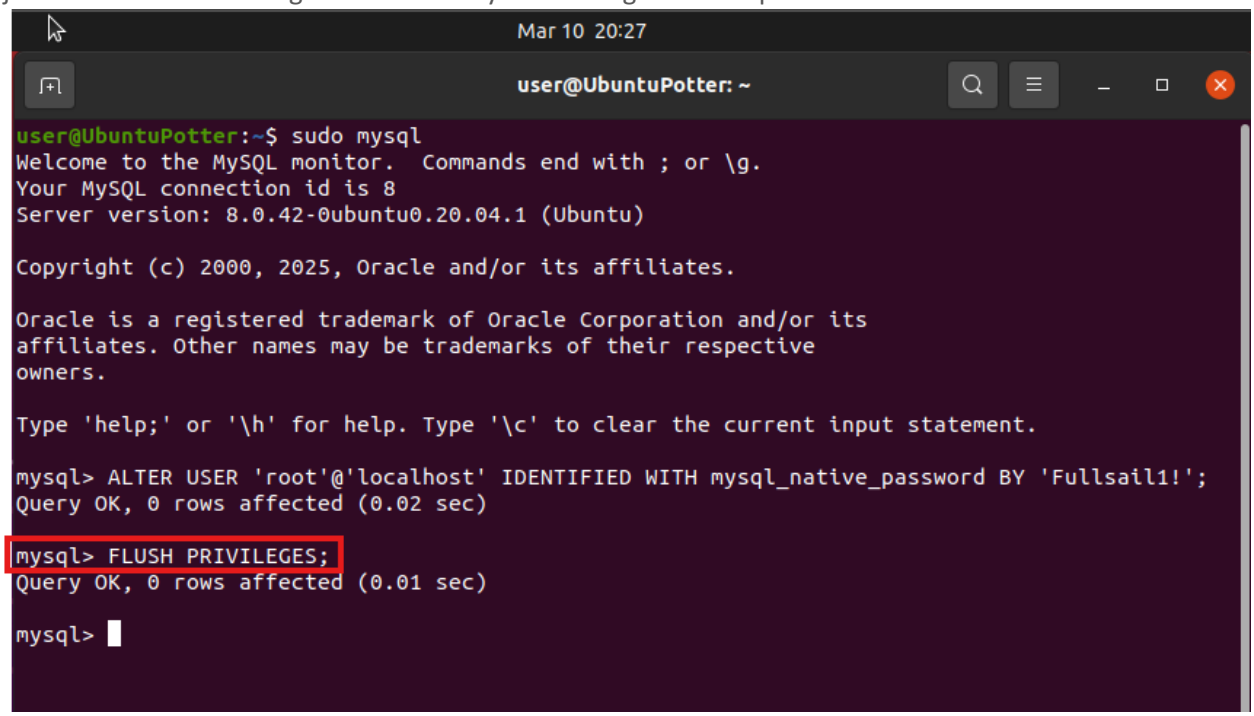


We will then enter this command to change the Root password “**ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'Fullsail1!' ;**”



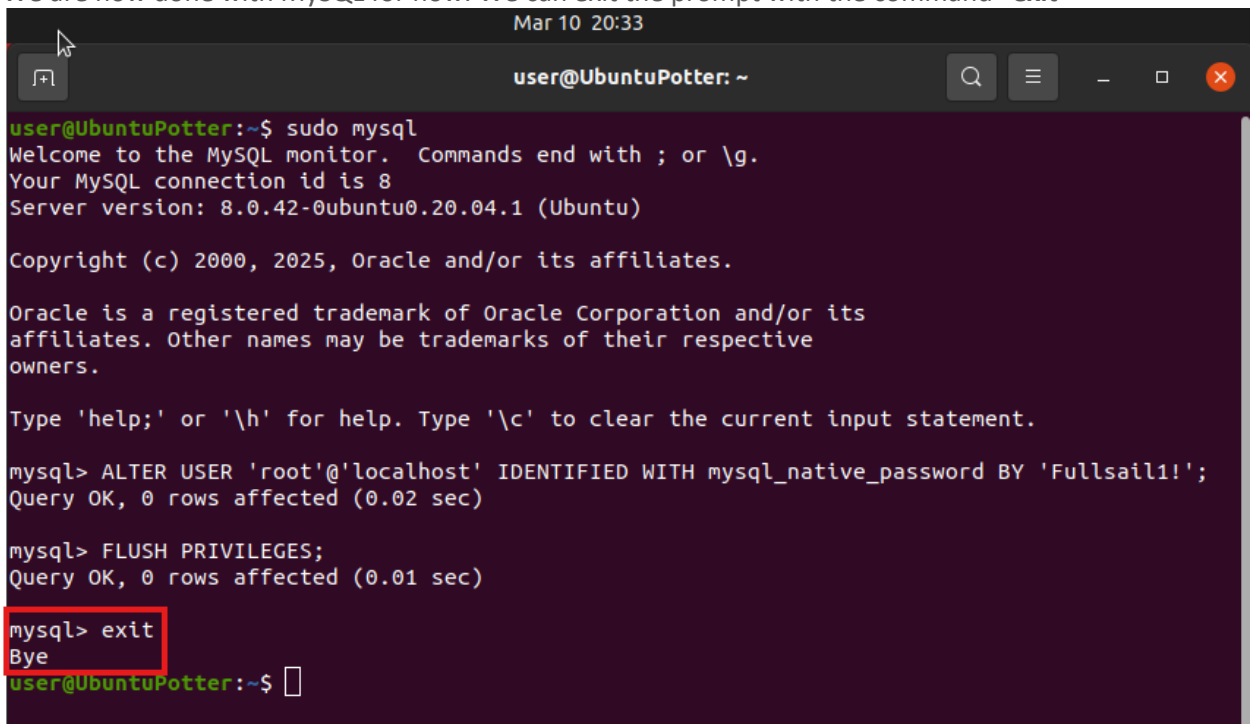
Flush Privileges

We then want to Use a command "**FLUSH PRIVILEGES;**" this command is used to apply the changes we just made. After entering this command you should get this output:



Exit MySQL

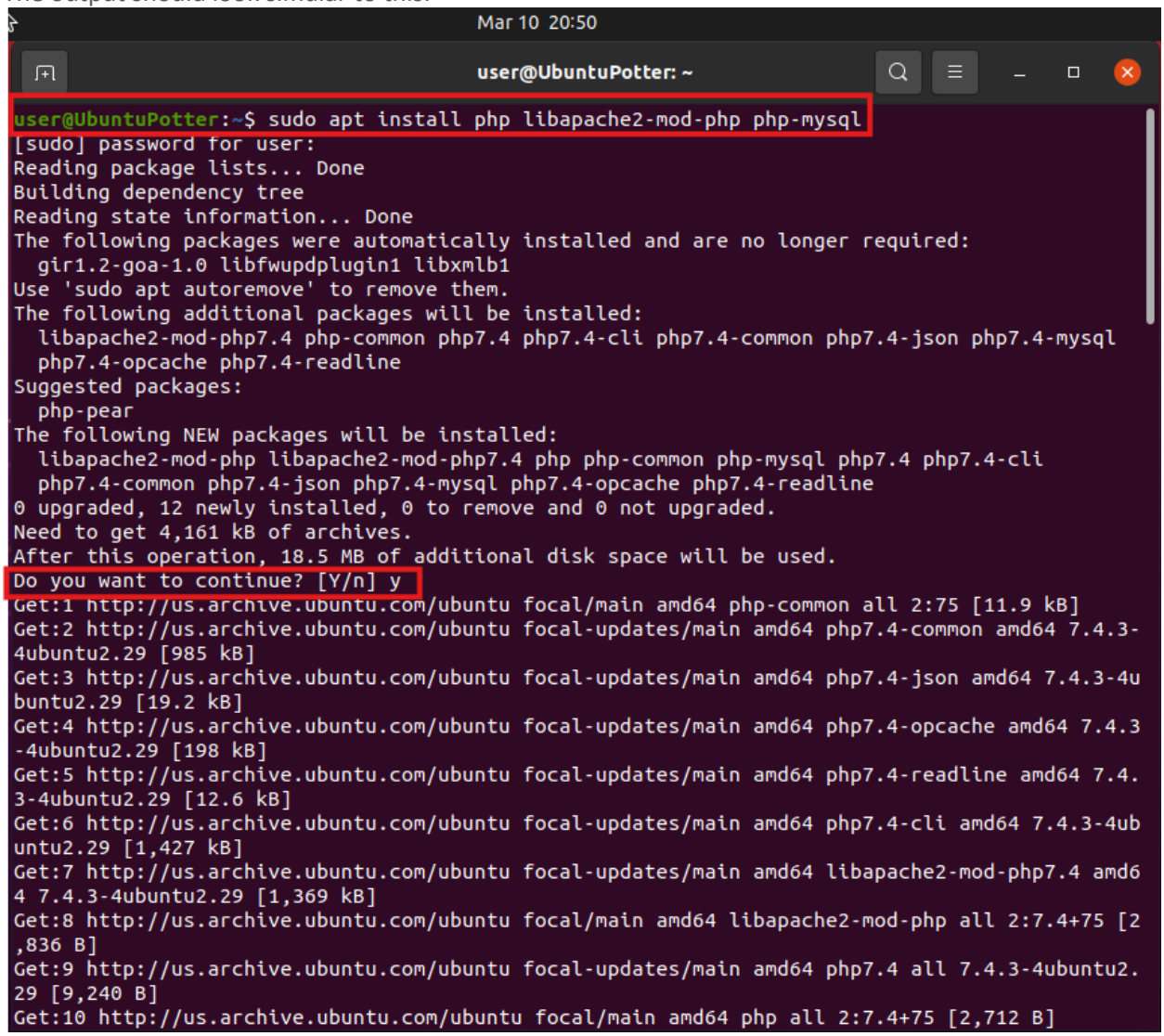
We are now done with MySQL for now. We can exit the prompt with the command "**exit**"



Install PHP

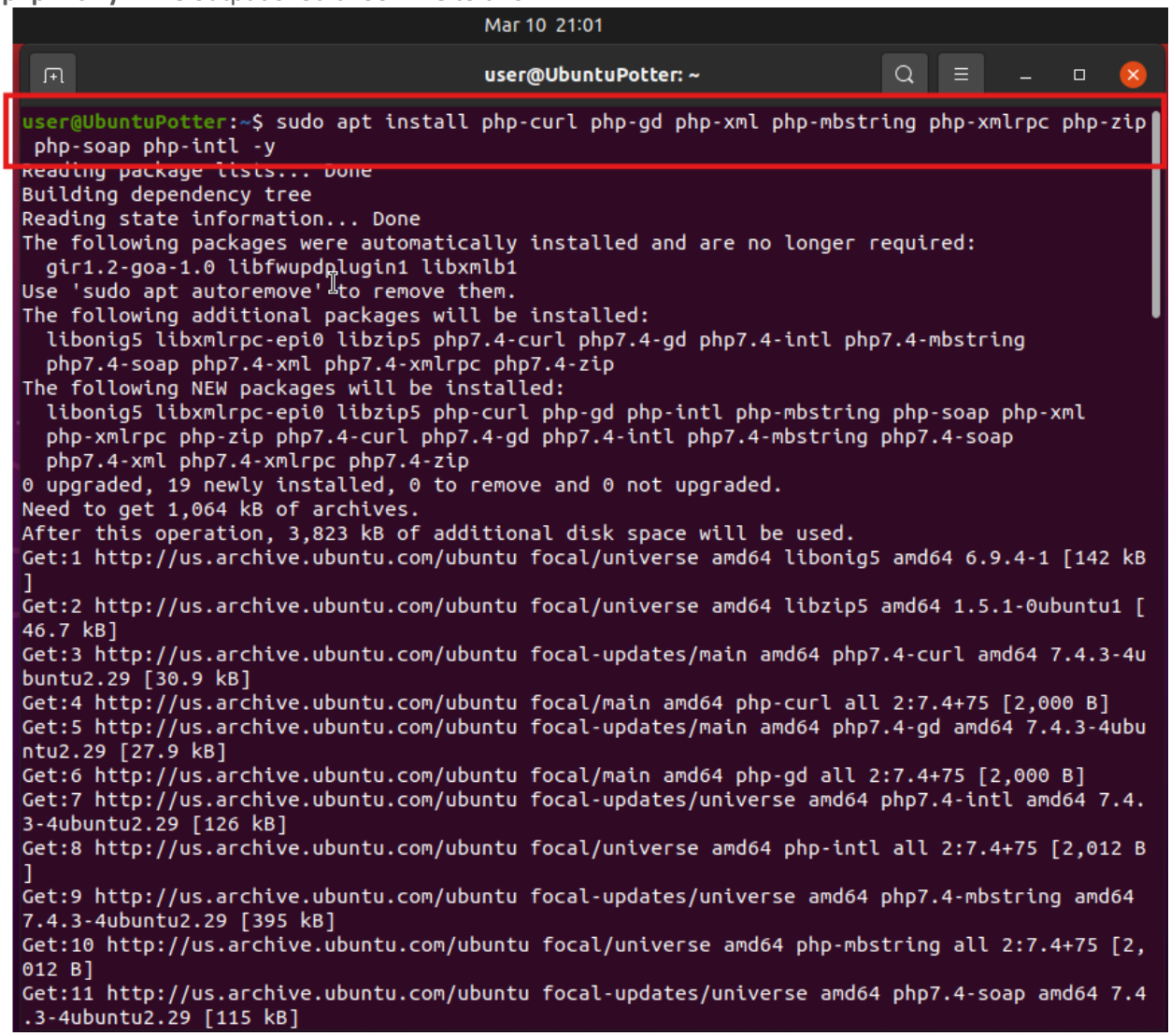
Install Required PHP Libraries

We are now going to install PHP and some other required packages. We will do this by using the command "**sudo apt install php libapache2-mod-php php-mysql**" and when prompted input "**Y**" to confirm the install. The output should look simular to this:



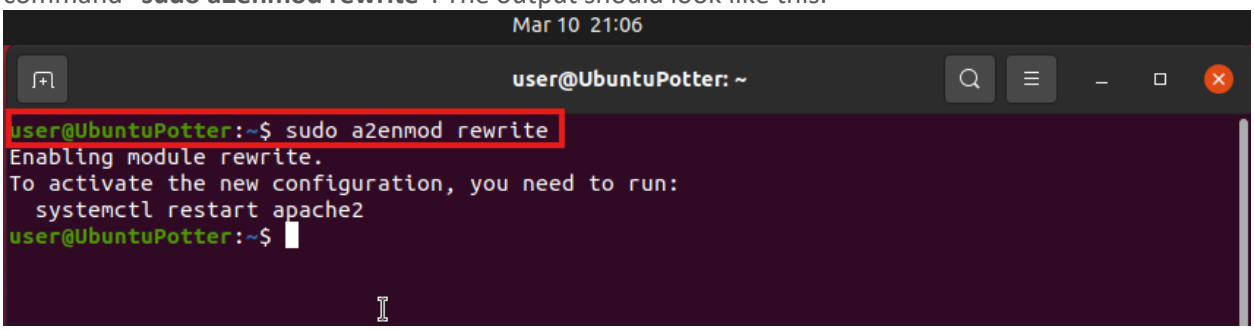
Install Required MySQL Libraries

We now need to install some additional extenision for additional functionality. We need to input the following command "**sudo apt install php-curl php-gd php-xml php-mbstring php-xmlrpc php-zip php-soap php-intl -y**". The output should look like to this:



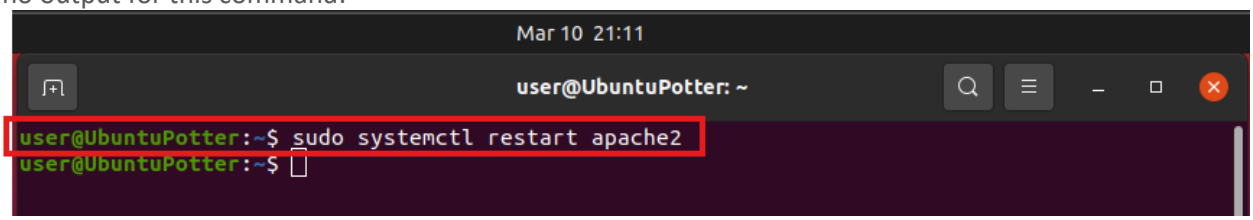
Enable URL Rewrites (clean URLs)

We now want to enable URL to enable them to be more user friendly. We can accomplish this with the command "**sudo a2enmod rewrite**". The output should look like this:



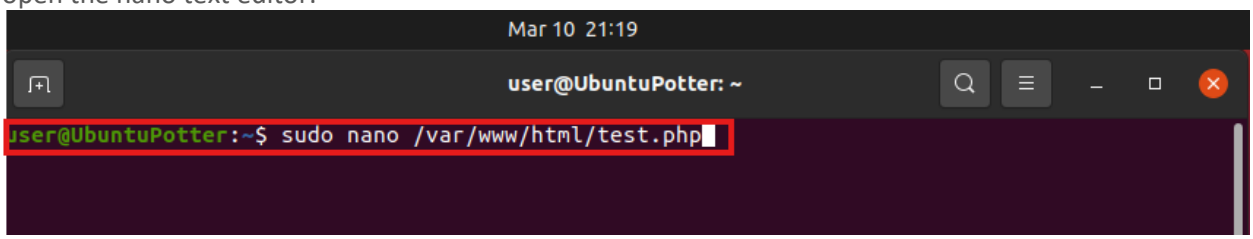
Restart Apache Service

We then need to restart the Apache service using the command "**sudo systemctl restart apache2**" there is no output for this command:

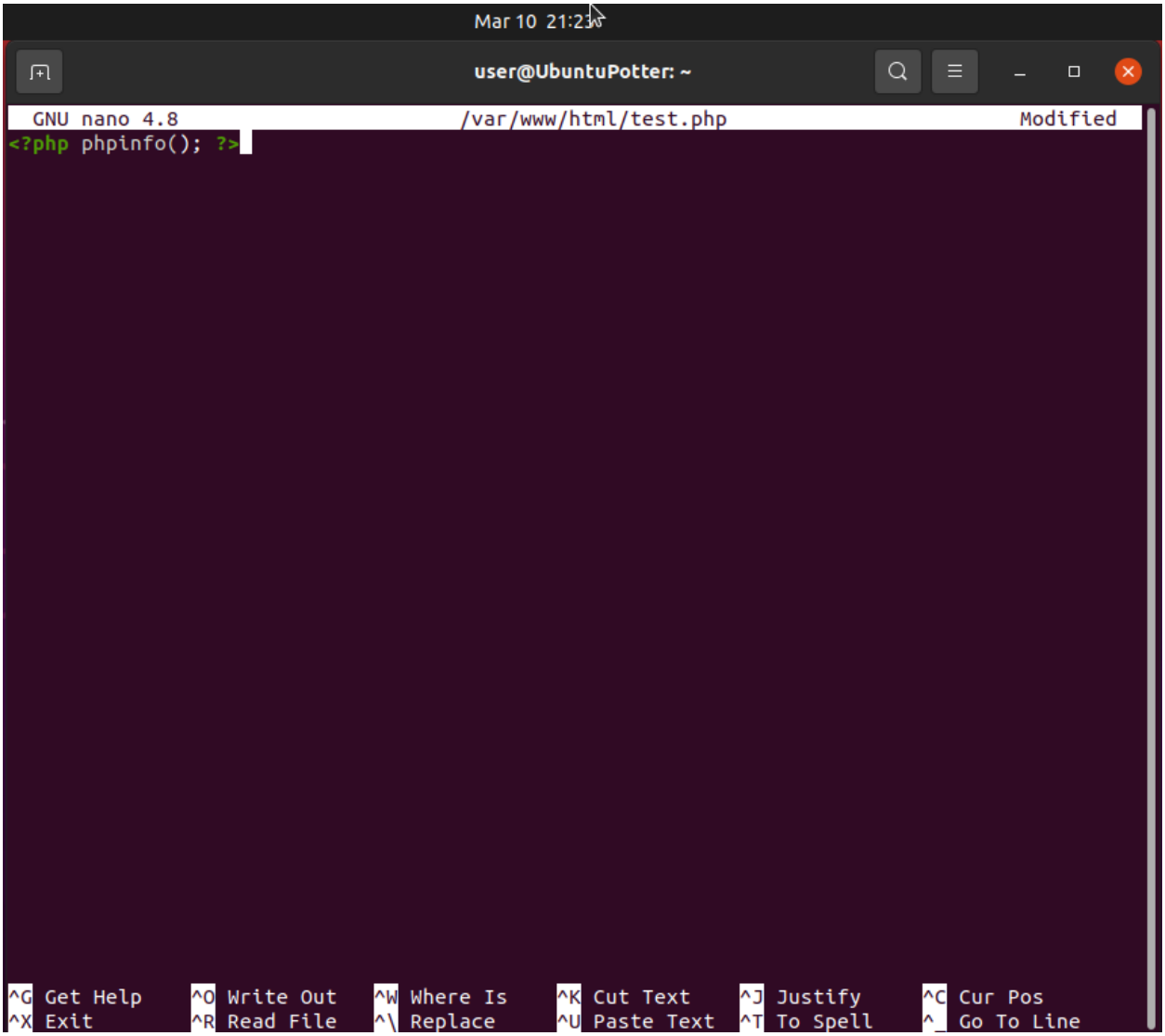


Create a test.php Web Page

Now lets create a test page, we will enter the command "**sudo nano /var/www/html/test.php**" this will open the nano text editor:



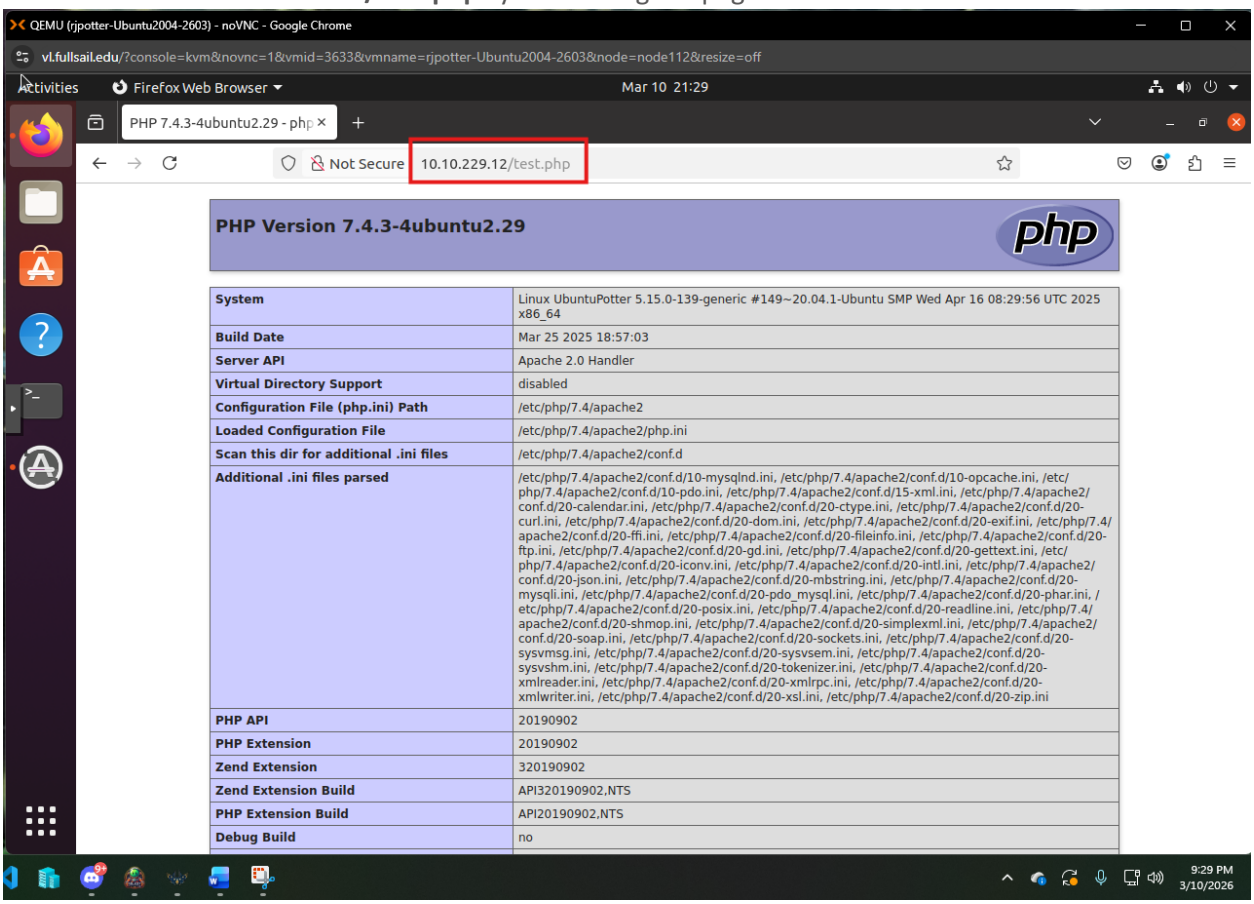
Once in the text editor, we then want to enter the following text "***<?php phpinfo(); ?>***" it should look like this:



On this is done. Press "**Ctrl-X**", Press "**y**", and then press "**Enter**"

Test the test.php Web Page

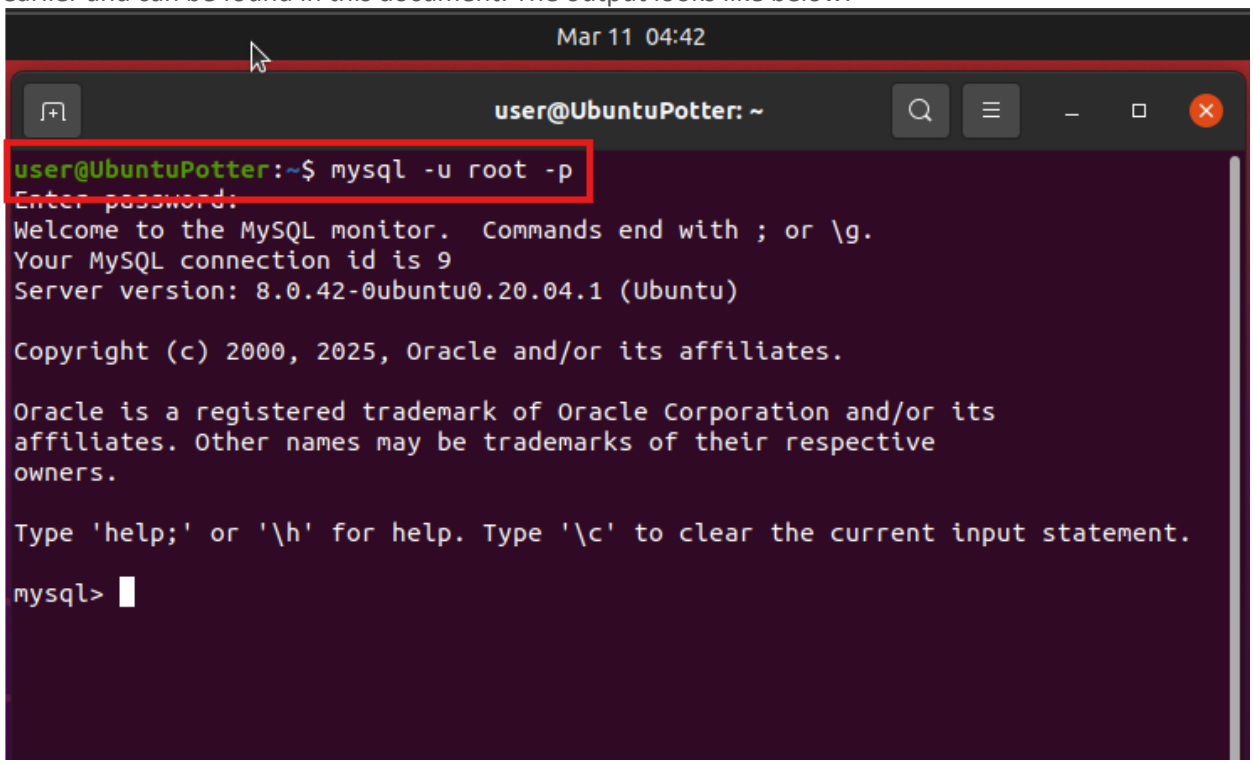
We now want to test the page we just made. We will do this by opening a FireFox Browser and enter in the address bar "**10.10.229.12/test.php**" you should get a page like this:



Database Configuration in MySQL

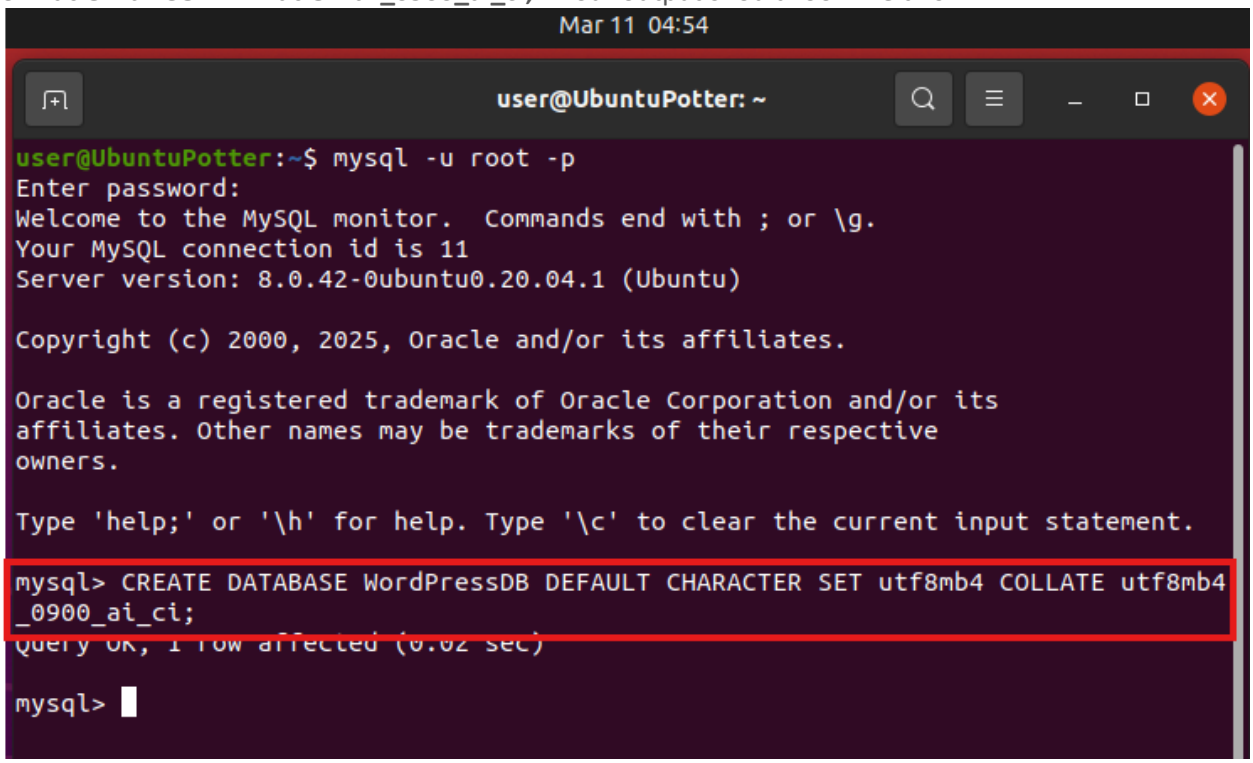
Log into MySQL

In this Section we will be setting up our MySQL for the WordPress site. So First we need to Login to MySQL we do this with the command "**mysql -u root -p**" you will then be prompted for a password that we setup earlier and can be found in this document. The output looks like below:



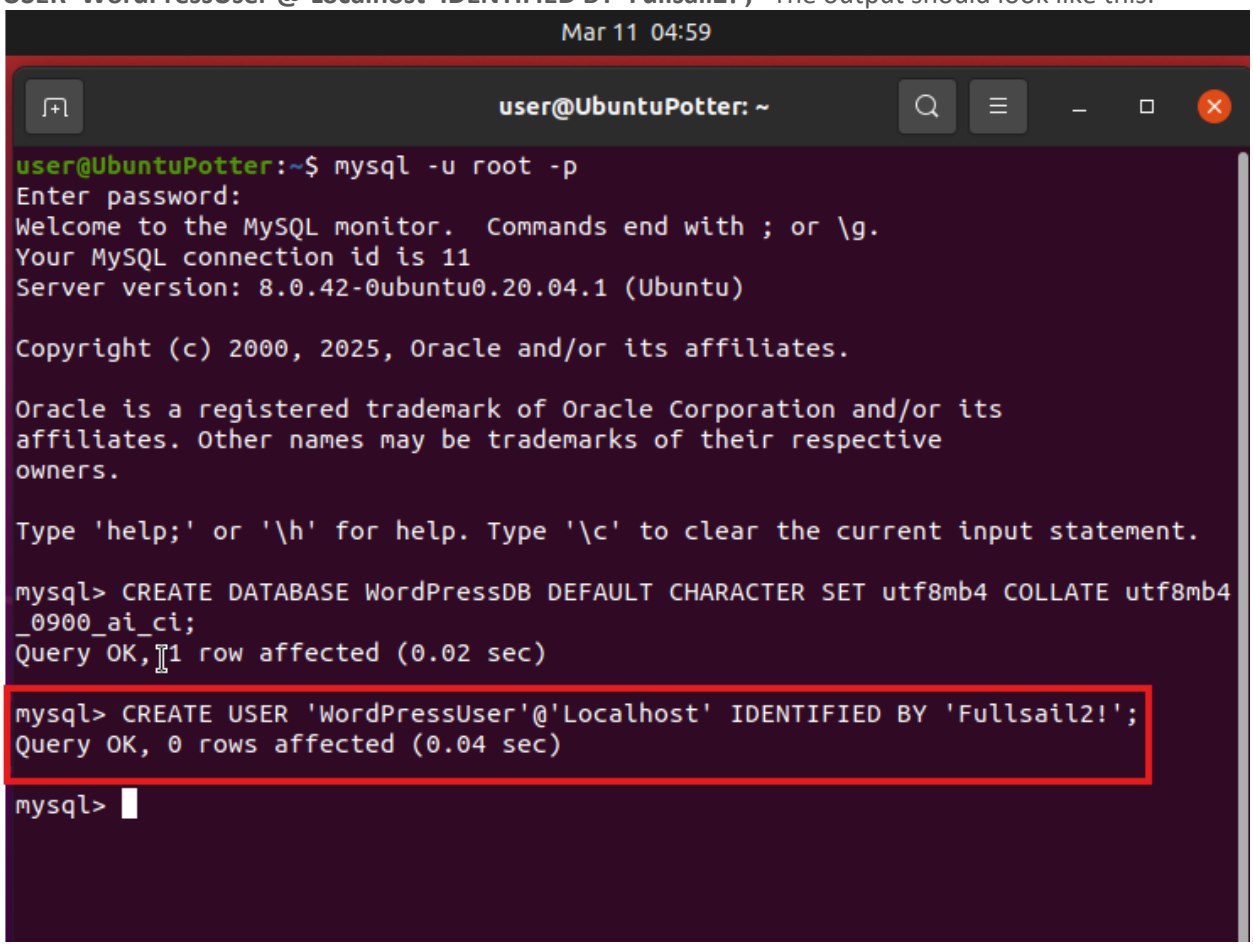
Create MySQL WordPress Database (WordPressDB)

Now Lets create the WordPress Database, this is where the options and data for the website will be stored. We will create the Database using the command "**CREATE DATABASE WordPressDB DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;**". Your output should look like this:



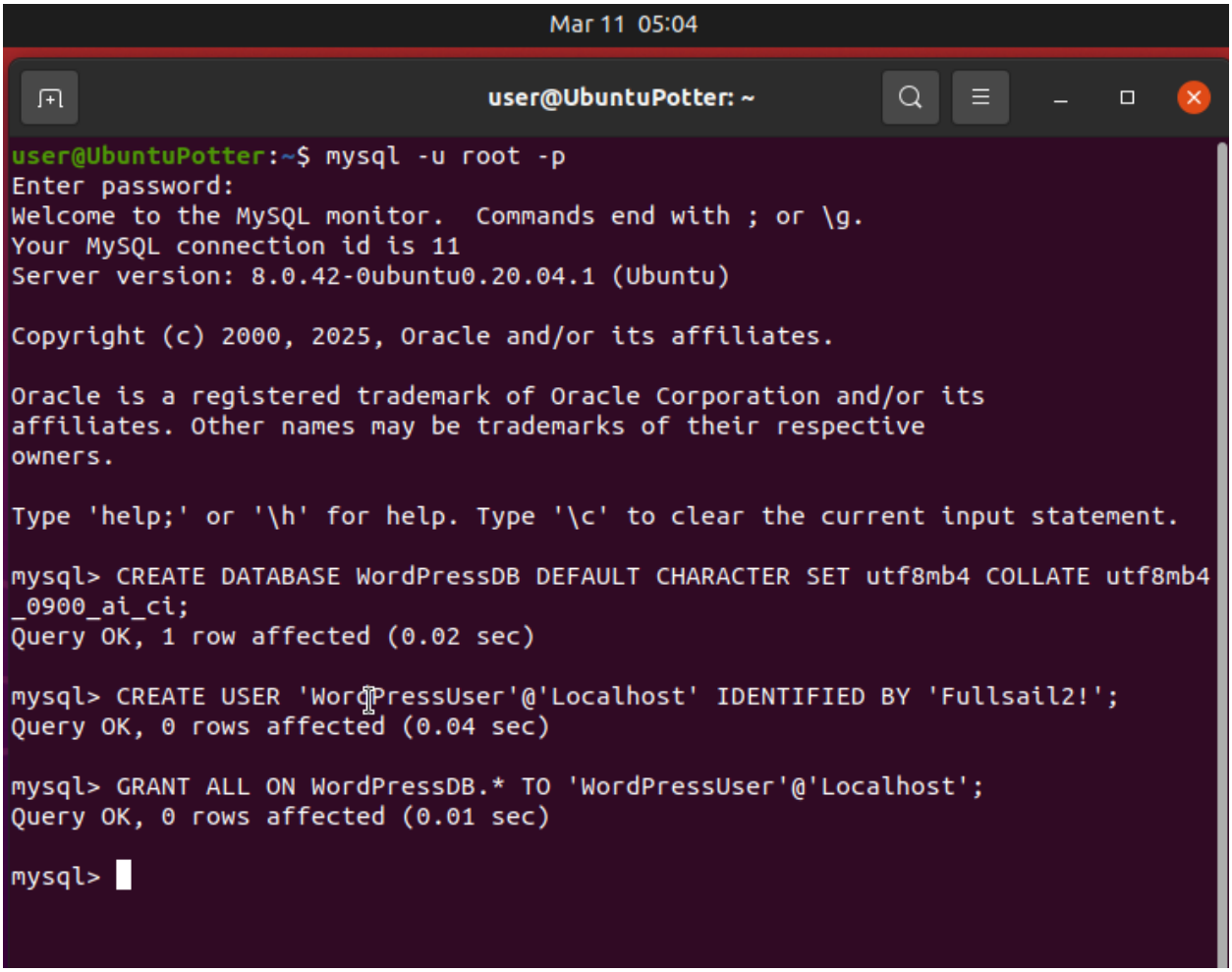
Create MySQL WordPress User (WordPressUser)

We will now create a user account for the wordpress site to use. The command to do this is "**CREATE USER 'WordPressUser'@'Localhost' IDENTIFIED BY 'Fullsail2!';**" The output should look like this:



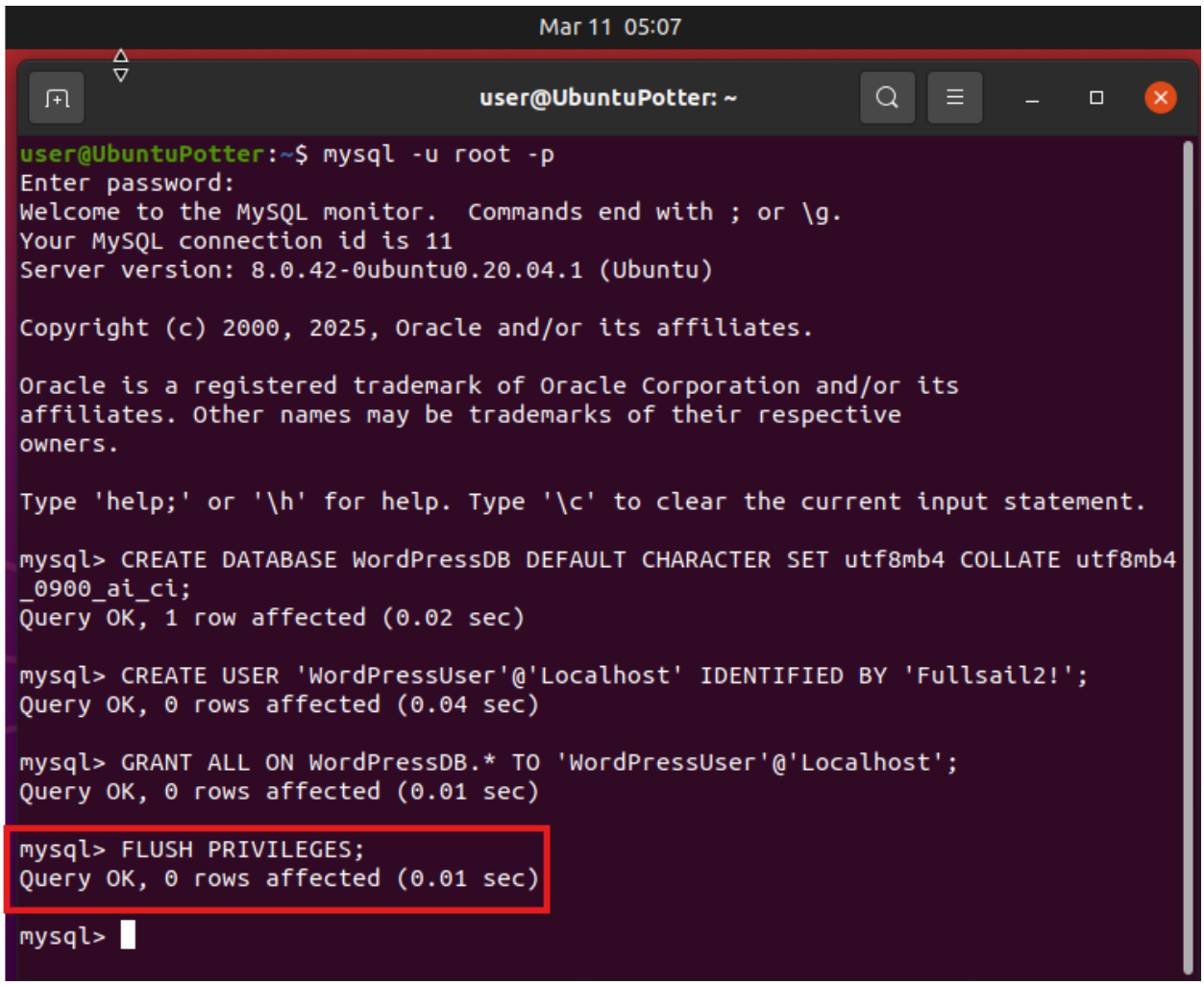
Grant Privileges to the WordPress Database (WordPressDB) to WordPress User (WordPressUser)

We then need to Grant our new user with permissions to make changed to the WordPressDB. We will do this using the command "**GRANT ALL ON WordPressDB.* TO 'WordPressUser'@'localhost';**". The output should look like this:



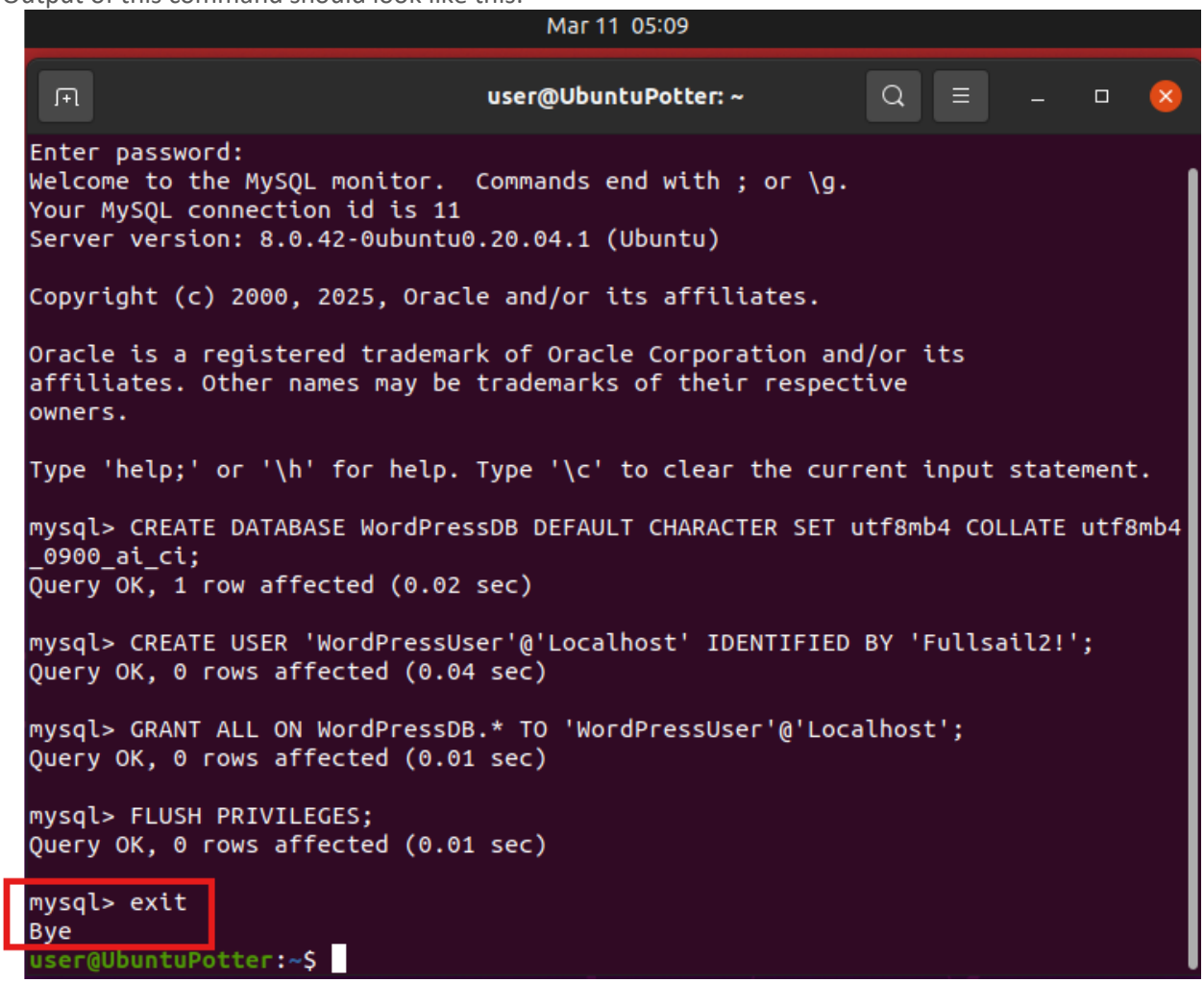
Flush Privileges

Now for all these changes to take effect we need to use the command "**FLUSH PRIVILEGES;**". The output Should look like this:



Exit MySQL

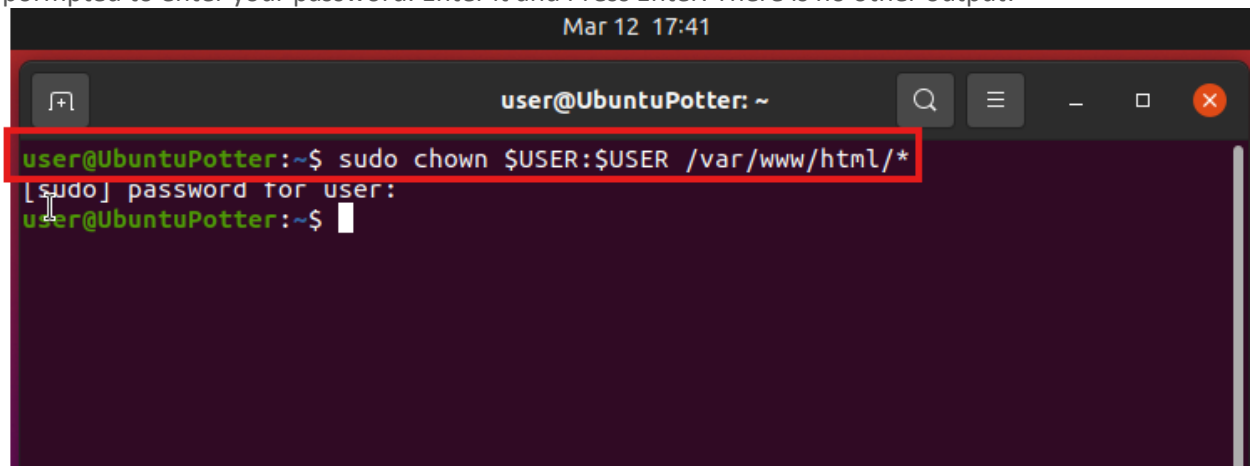
With this final step complete we can now safely Exit the MySQL prompt using the command "**exit**". The Output of this command should look like this:



Install WordPress

Grant Permission to /var/www/html/ Directory to WordPress User

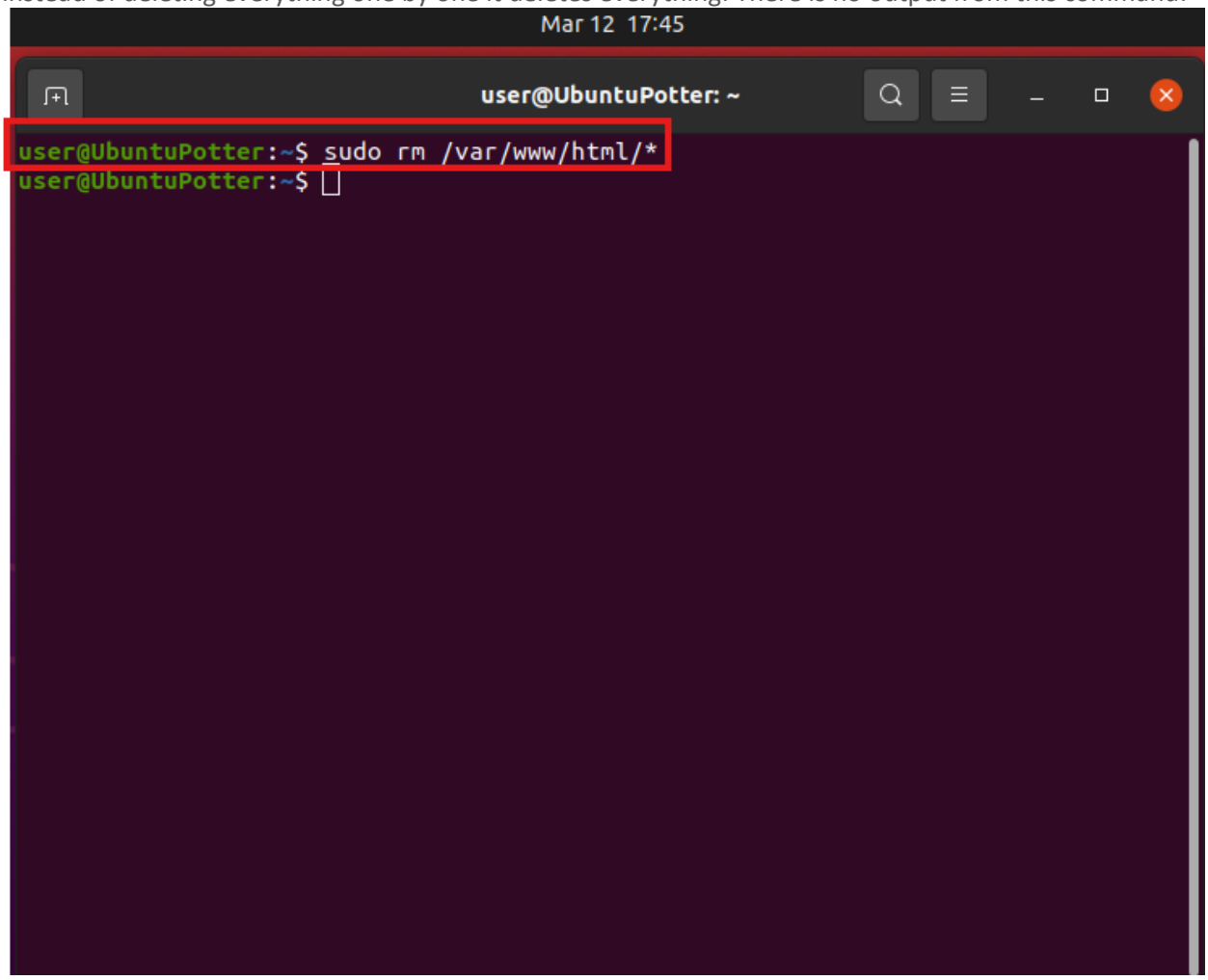
We will now grant permisssions for our WordPressUser to be able to modify files in the the /var/www/html/ directory. We will do this with the command "**sudo chown $USER:$USER /var/www/html/***". You will be pormpted to enter your password. Enter it and Press Enter. There is no other output:



Delete Files from /var/www/html/ Directory

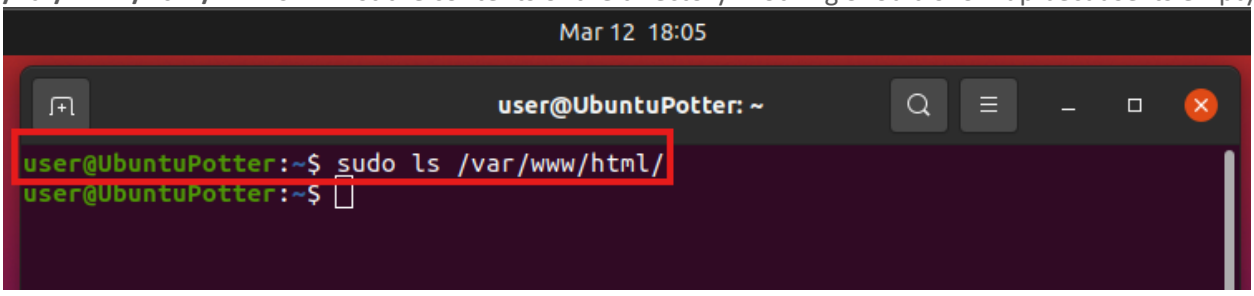
We now want to delete all the contents of this folder including our test file. We will accomplish this with the command "**sudo rm /var/www/html/***". The star at the end is a wildcard and means Everything, so of

instead of deleting everything one by one it deletes everything. There is no output from this command:



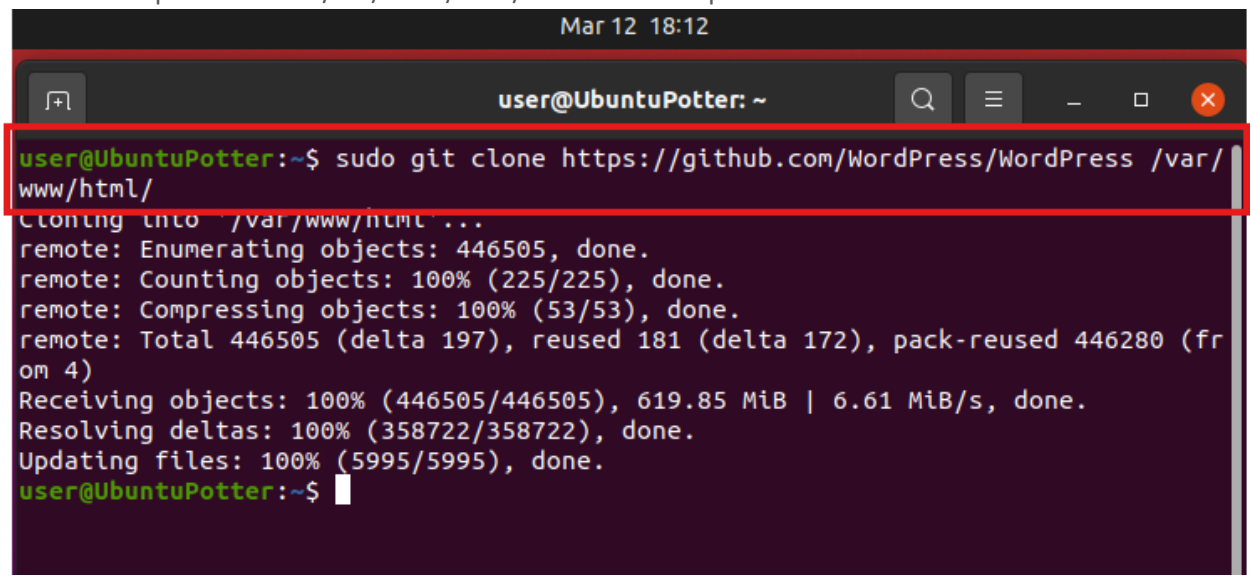
Verify /var/www/html/ Directory is Empty

We will not Verifiy that the Directory after the last step we will do this with the command "**Sudo ls /var/www/html/"**. This will list the contents of the directory. Nothing Should show up because its empty.



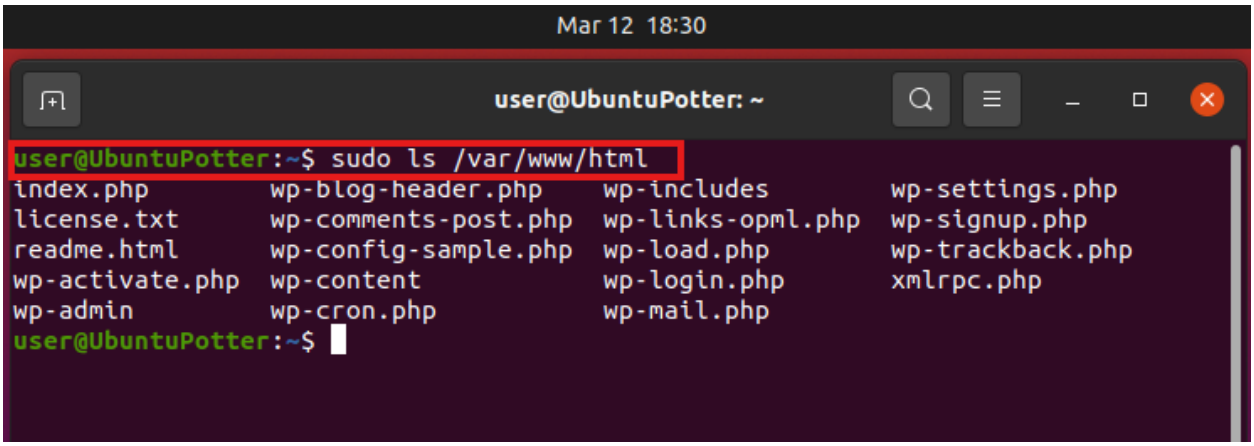
Clone WordPress to /var/www/html/ Directory

We will now Clone Wordpresss from the Github for it using the command "**sudo git clone https://github.com/WordPress/WordPress /var/www/html/**". This will pull the contents of the Github we entered and place it in the /var/www/html/ folder. The output should look like this:



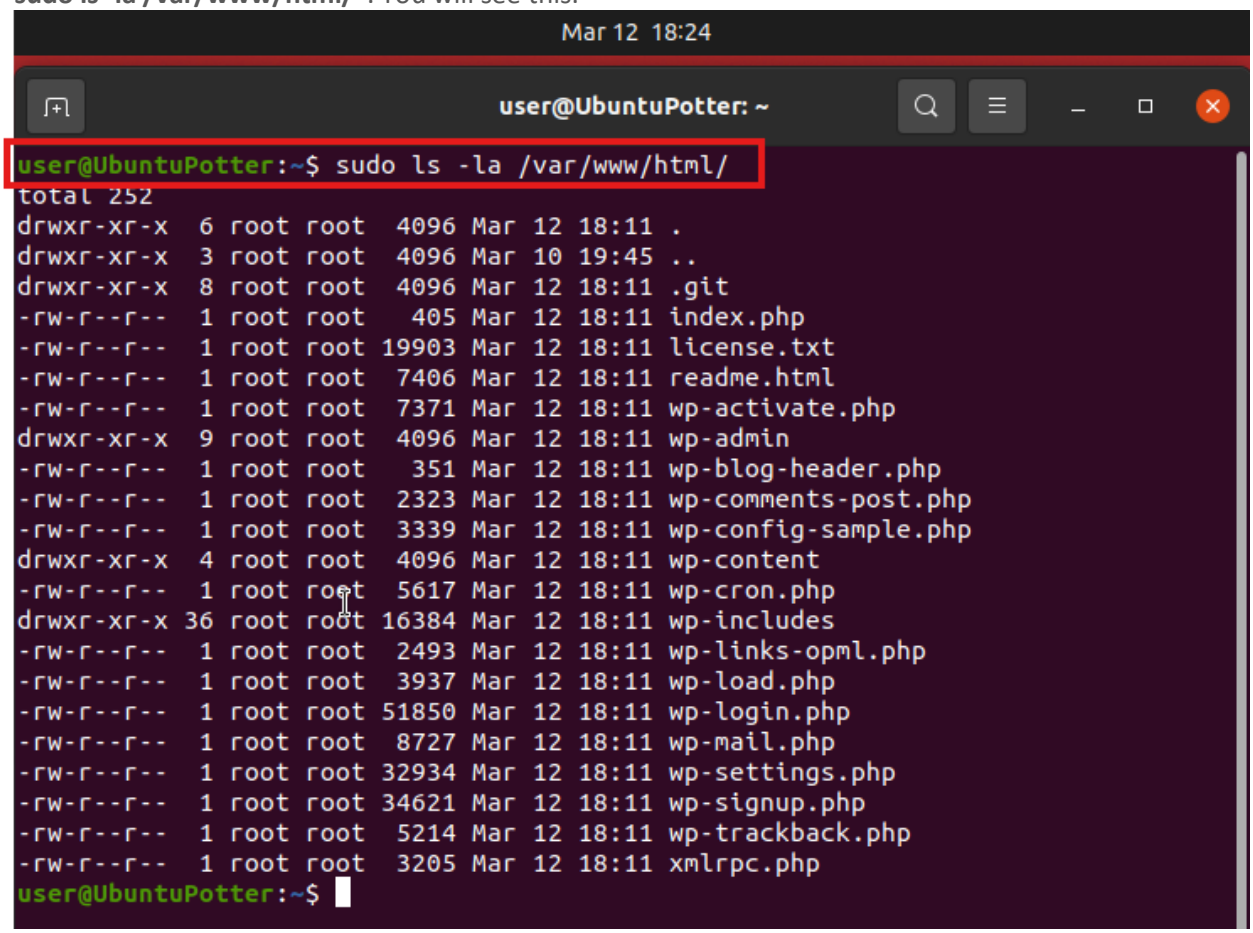
Verify /var/www/html/ Directory Contains WordPress Files

We now want to verify that the process work we will use the command "**sudo ls /var/www/html**" This will show the contents of the folder now that we have cloned WordPress:



Verify Permissions on /var/www/html/ Directory

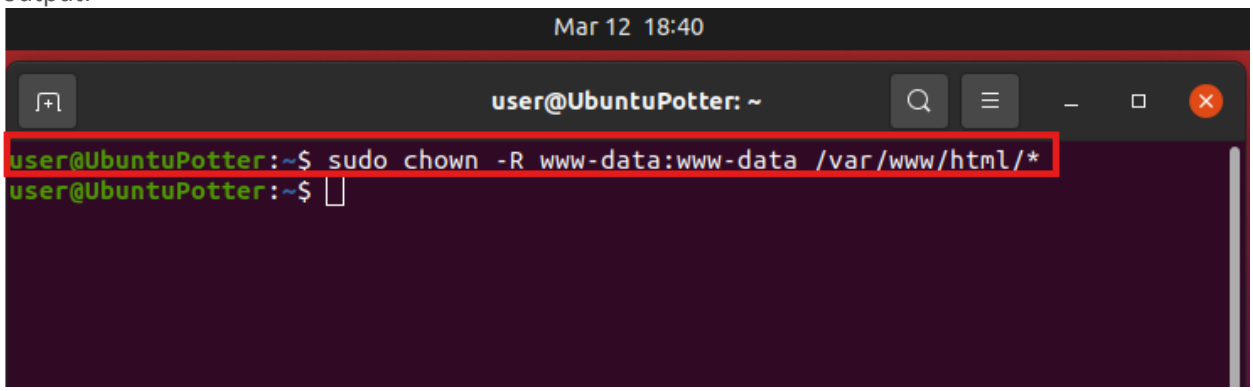
We can now verify the permissions on the files we just downloaded. We will do this with the command "**sudo ls -la /var/www/html/**". You will see this:



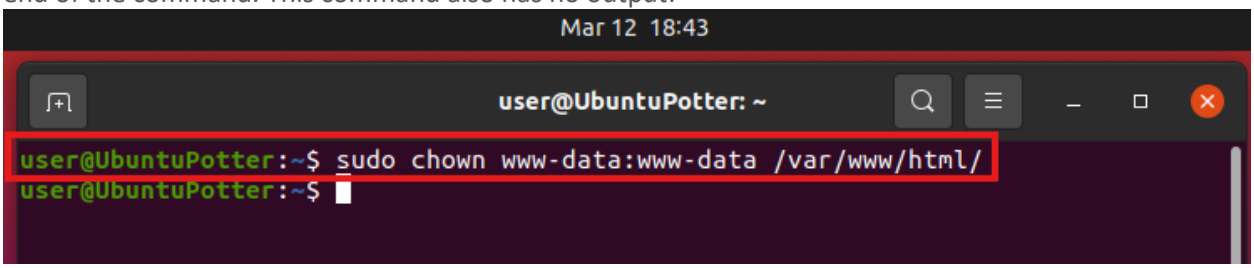
Edit Ownership

Edit Ownership of and the contents of /var/www/html/ Directory

We will now edit the permissions of the contents of /var/www/html/ directory so that the Apache can change things in it with the command "**sudo chown -R www-data:www-data /var/www/html/***". This will change the ownership to the Apache User. If prompted for password enter it and hit enter. There is no other output:

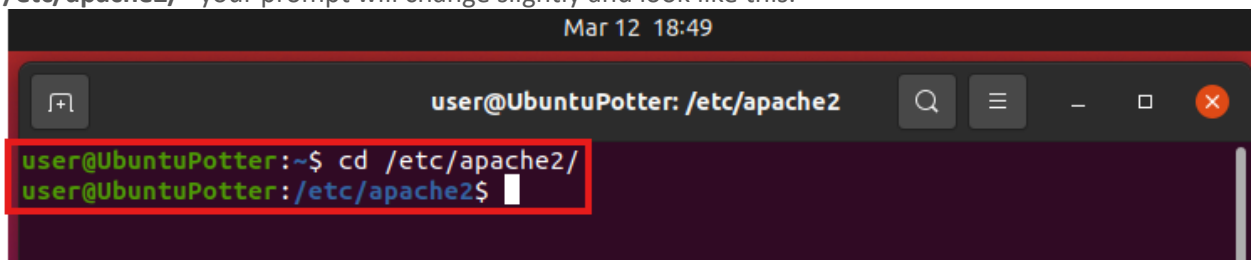


We then want to change the ownership of the directory itself with the command "**sudo chown www-data:www-data /var/www/html/**". This look simular to the last command but notice there is no * at the end of the command. This command also has no output:

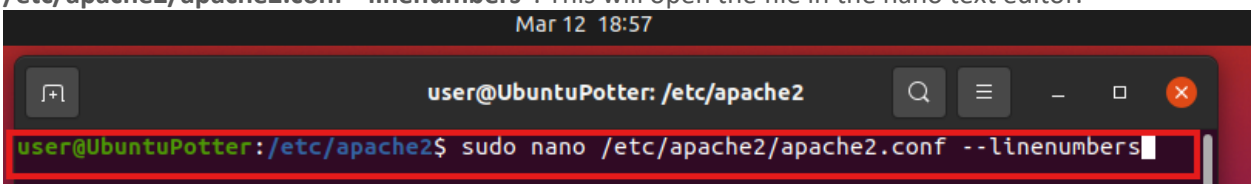


Edit the apache2.conf File

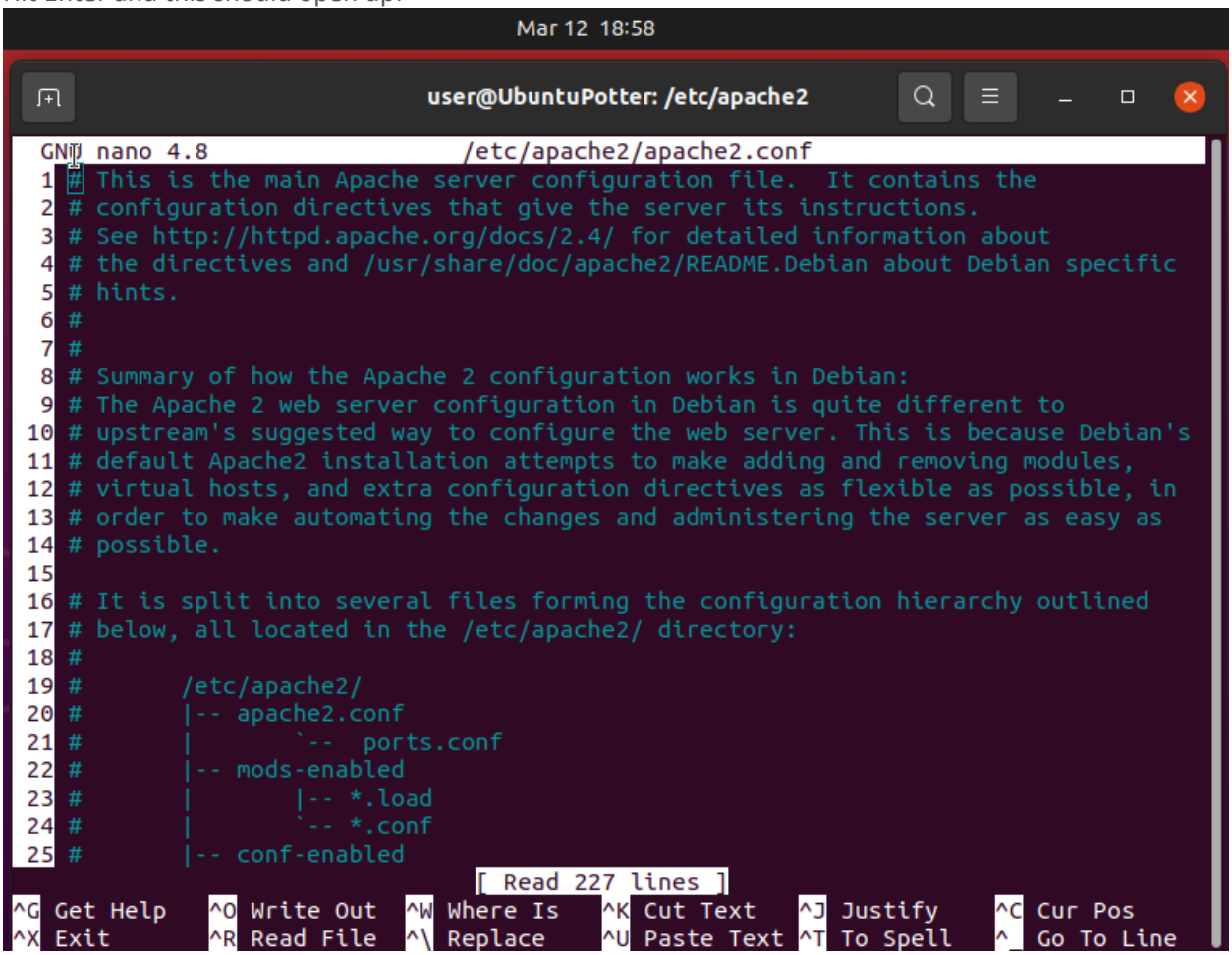
We now need to make a change to the apache2.conf file. To naviage to its location use the command "**cd /etc/apache2/**" your prompt will change slightly and look like this:



We then want to edit the apache2.conf file we will use the command "**sudo nano /etc/apache2/apache2.conf --linenumbers**". This will open the file in the nano text editor.

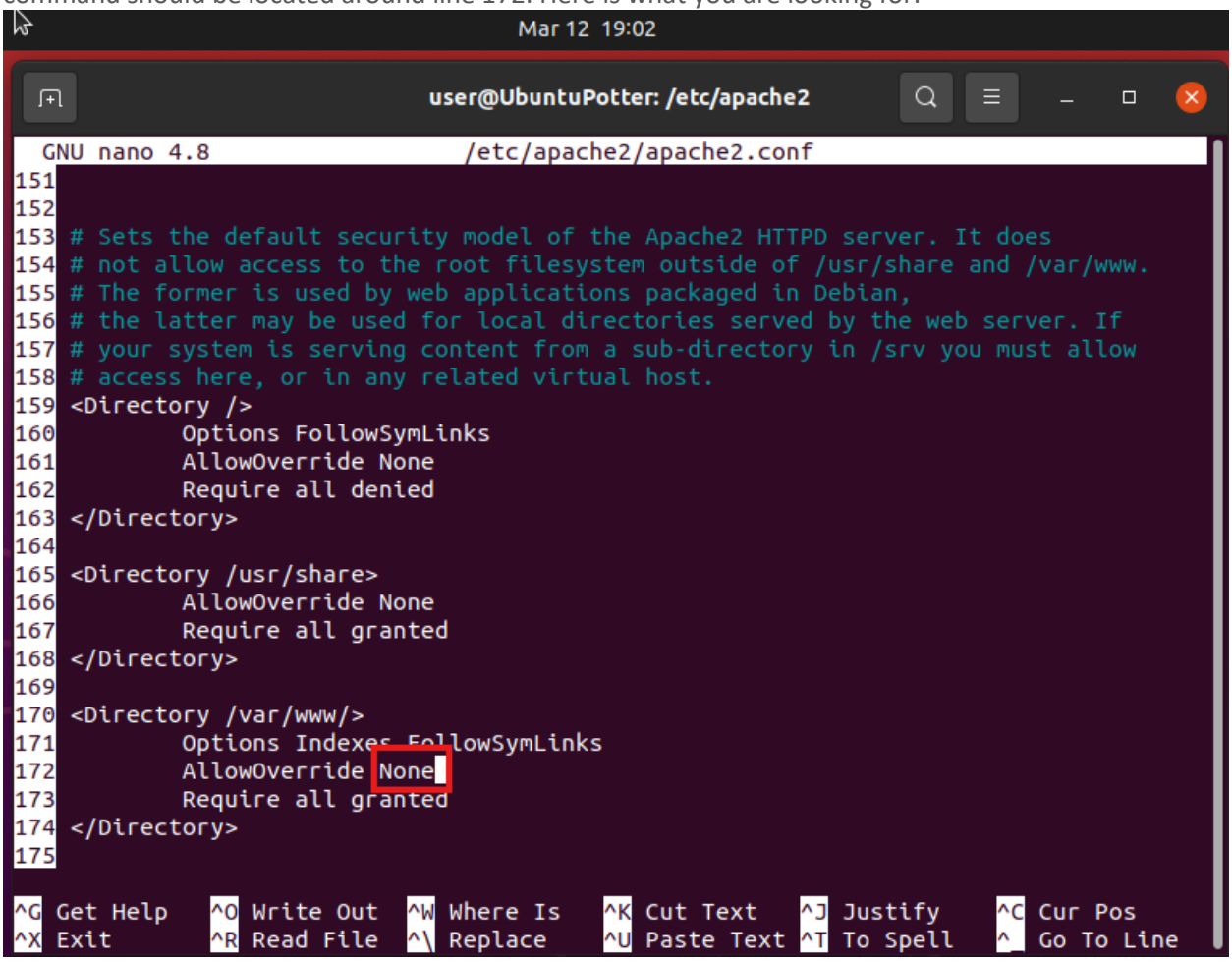


Hit Enter and this should open up:

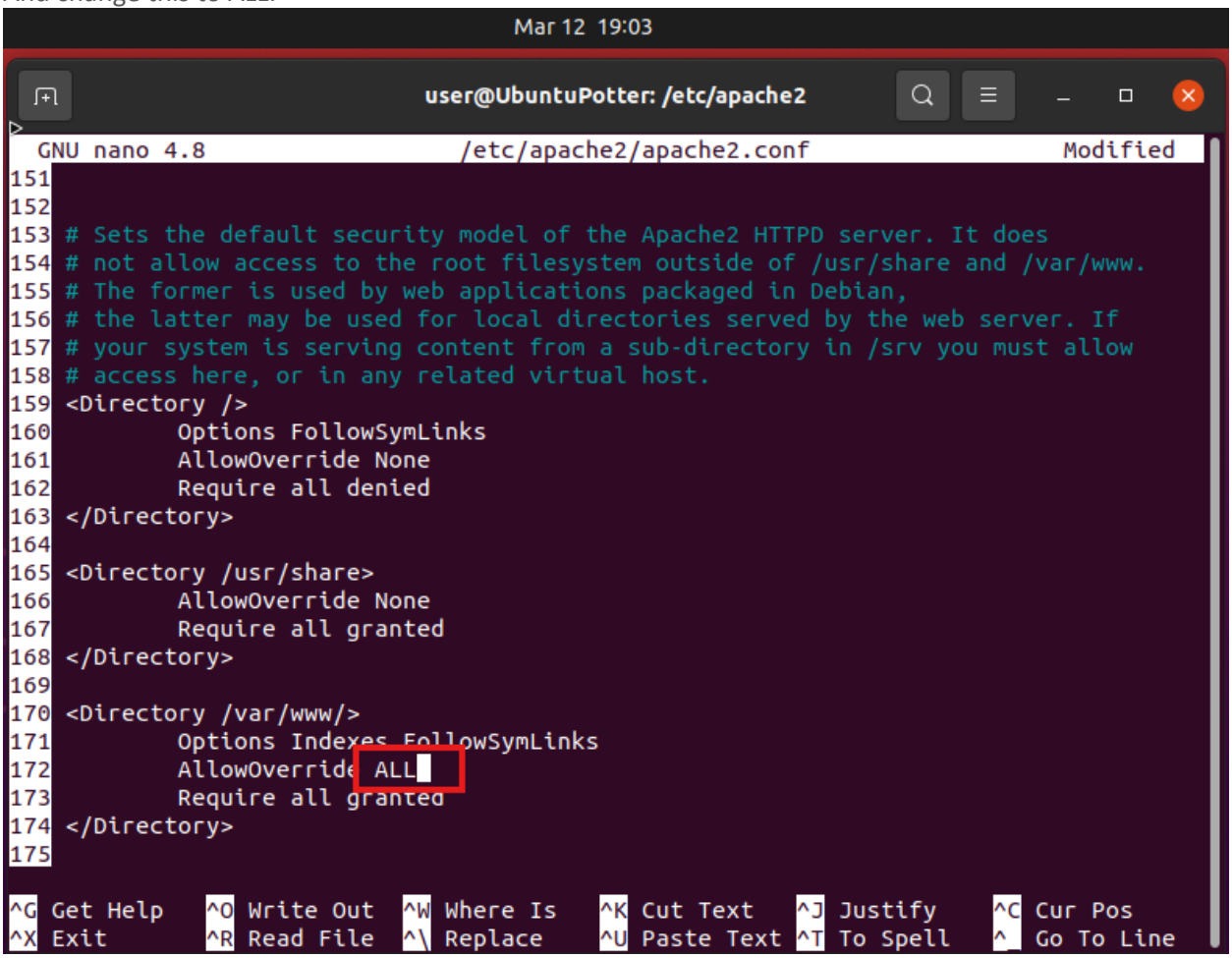


Override All Default Apache Directives

We now want to find the AllowOverride option for the directory /var/www/ and change it to ALL. This command should be located around line 172. Here is what you are looking for:



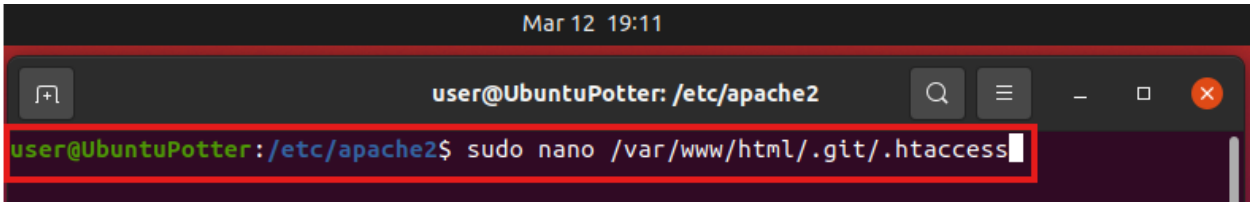
And change this to ALL:



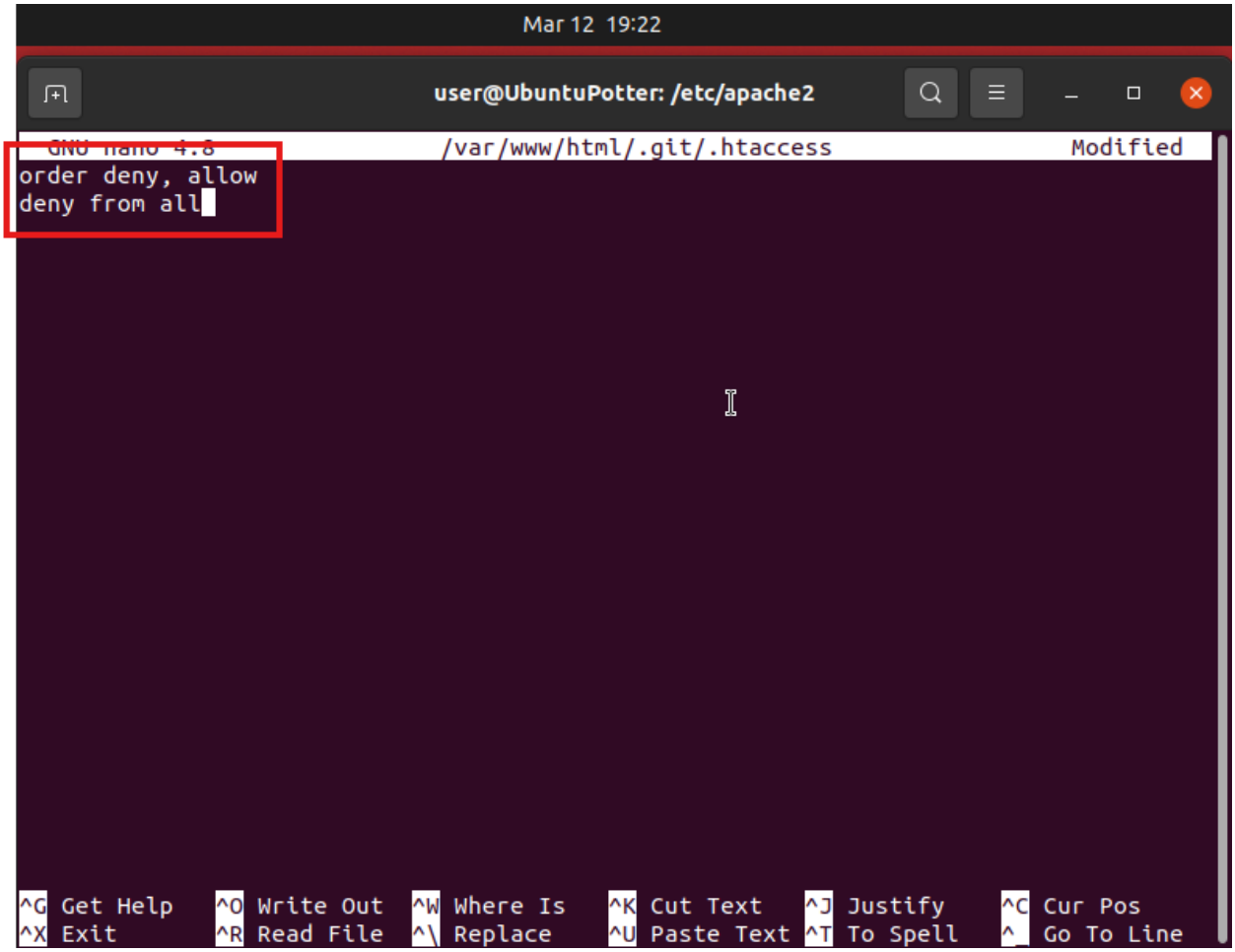
Press **Ctrl+X** Then **Y** then **Enter** to save the change.

Create a .htaccess File in the /var/www/html/.git/ Directory

We then want to create a .htaccess file under the .git folder to secure access to it. We will do this by using the command "**sudo nano /var/www/html/.git/.htaccess**" This will open the Text editor to create our new file:



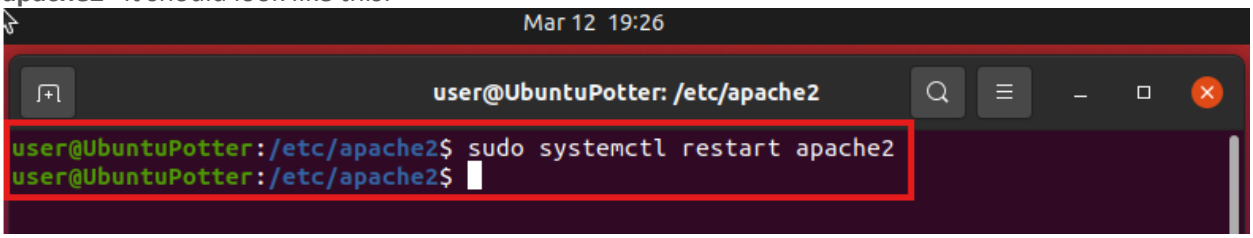
Once in the Text editor enter this text **order deny, allow** and on the next line **deny from all**. Just like below:



Press **Ctrl+X** Then **Y** then **Enter** to save the change.

Restart the Apache Service

The last step of this process is to restart the apache2 service using the command "**Sudo systemctl restart apache2**" It should look like this:



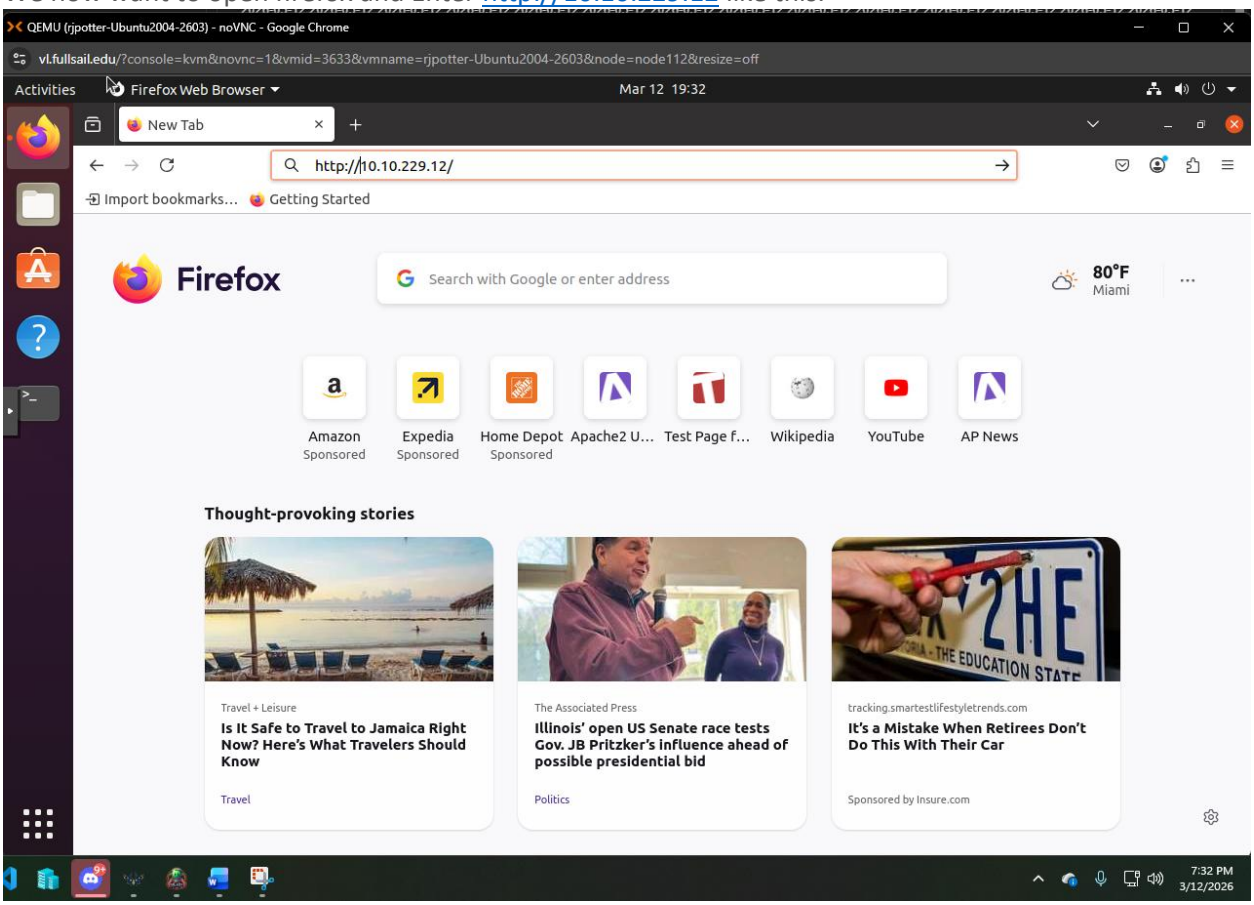
We can now close our terminal as we are done in it for now.

WordPress Configuration

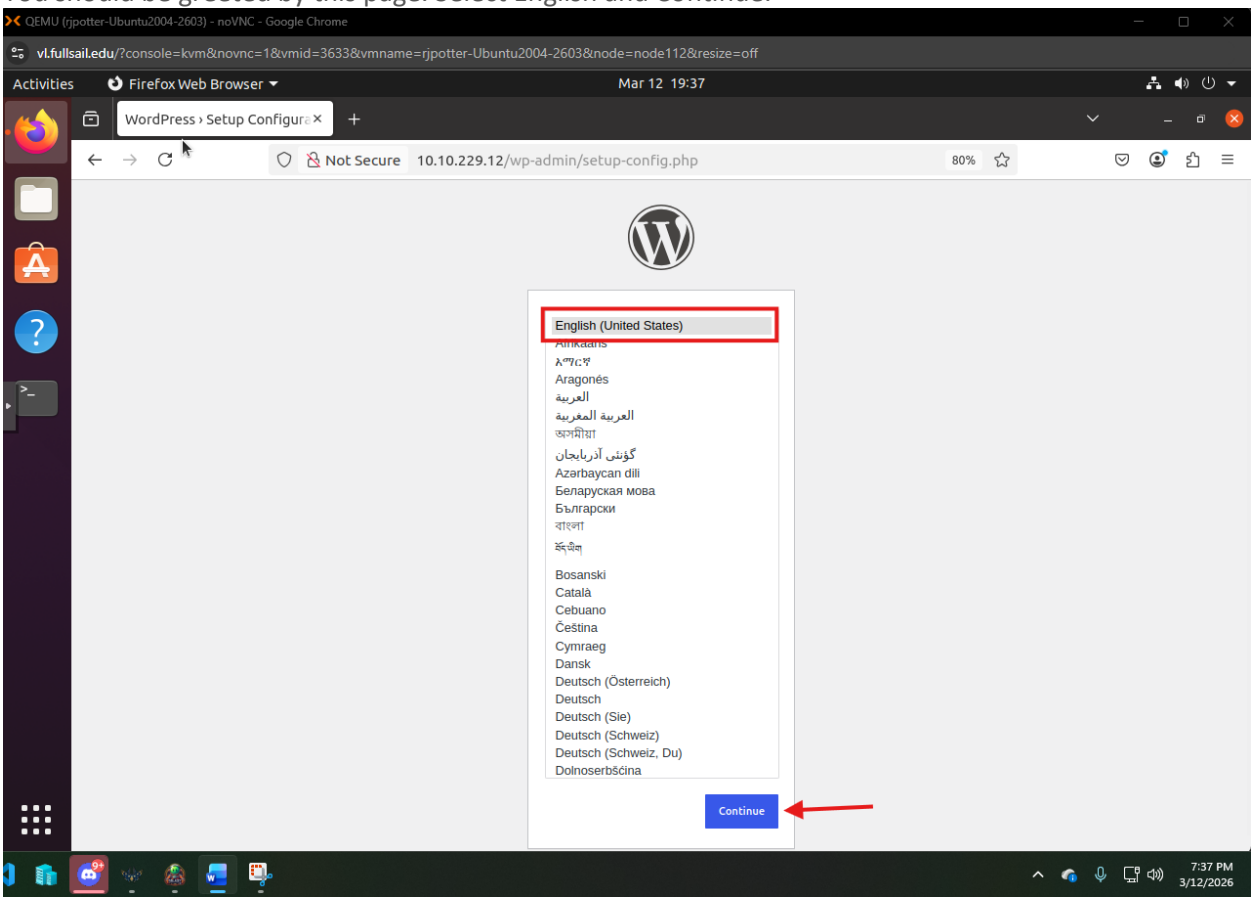
Configure WordPress

WordPress Configuration Selections

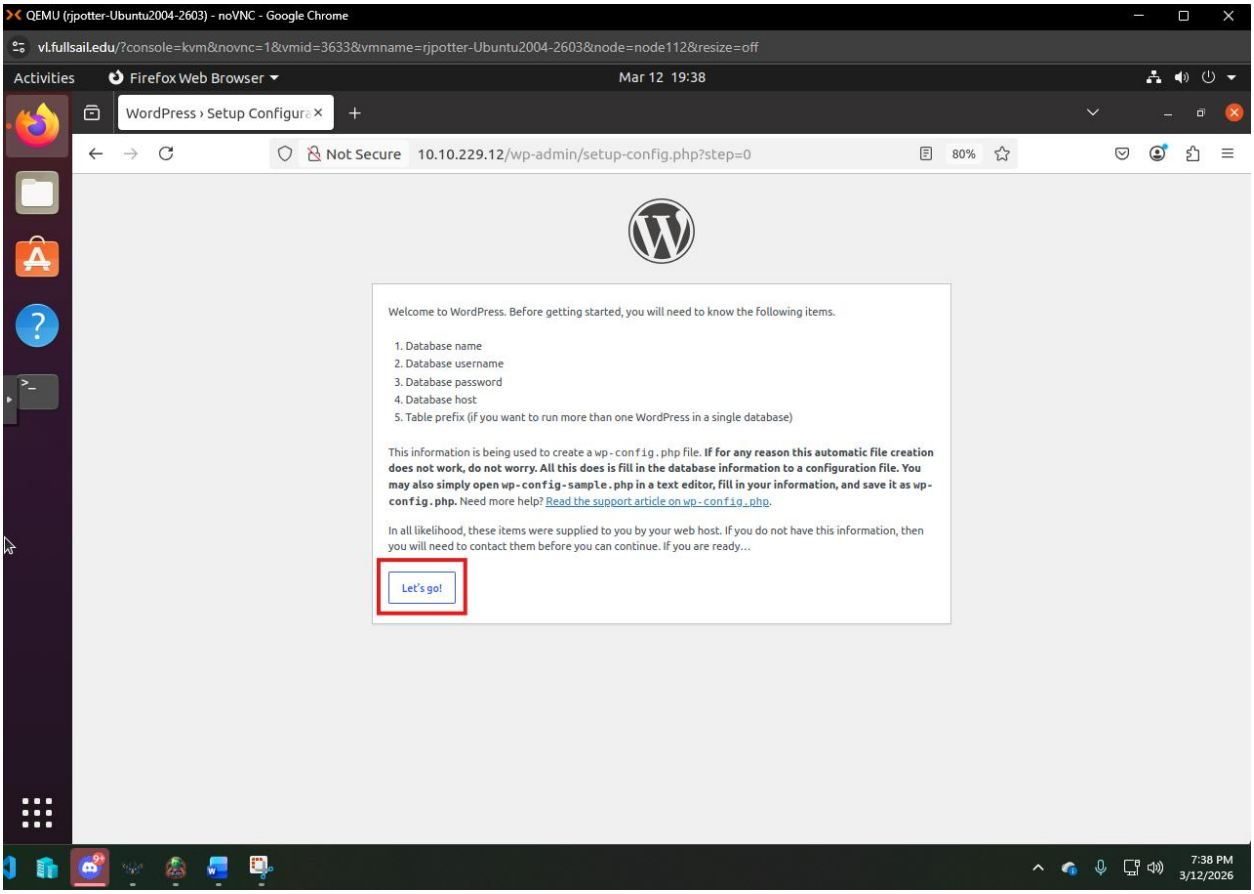
We now want to open firefox and Enter http://10.10.229.12 like this:



You should be greeted by this page. Select English and Continue:



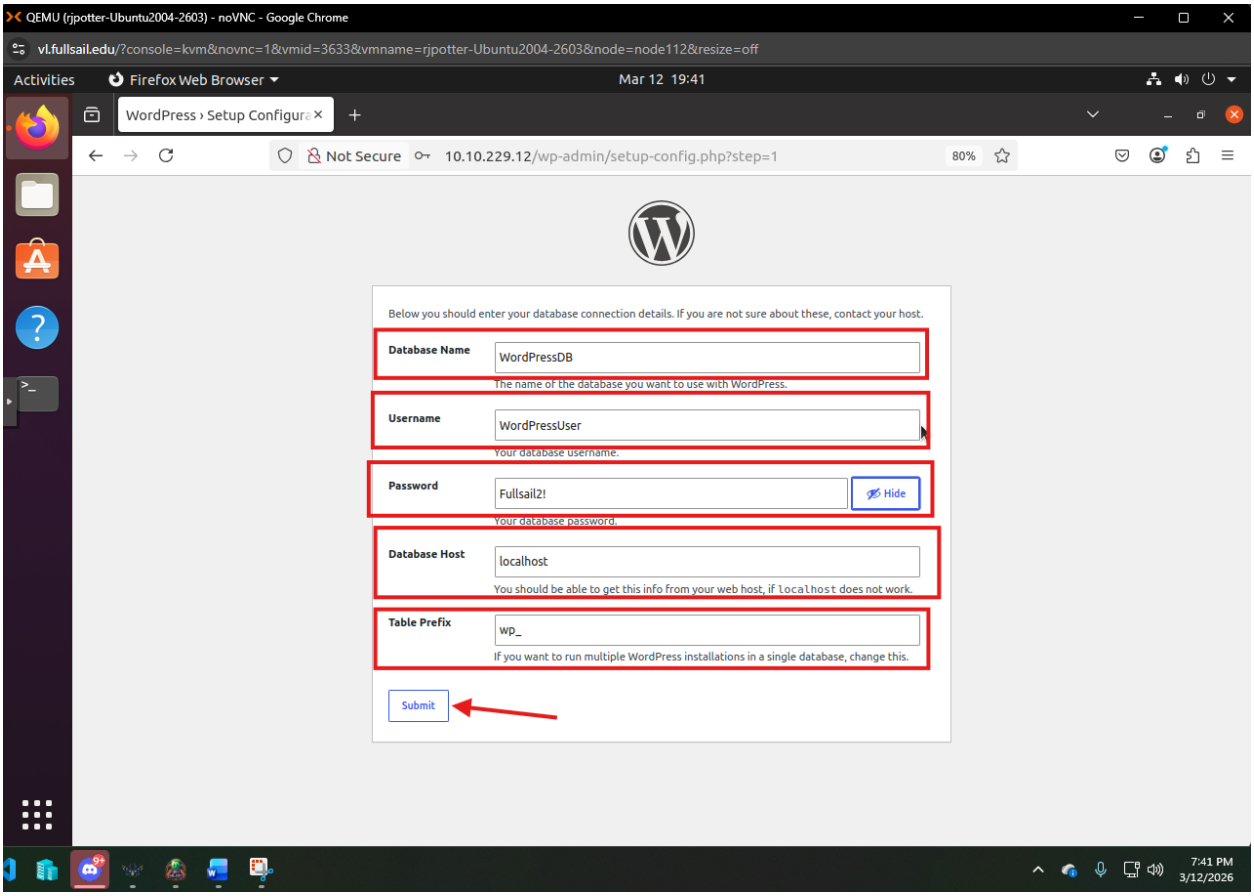
Click “Let’s Go!”:



Enter the info below into the Correct fields:

- **Database Name:** WordPressDB
- **Username:** WordPressUser
- **Password:** Fullsail2!
- **Database Host:** localhost
- **Table Prefix:** wp_

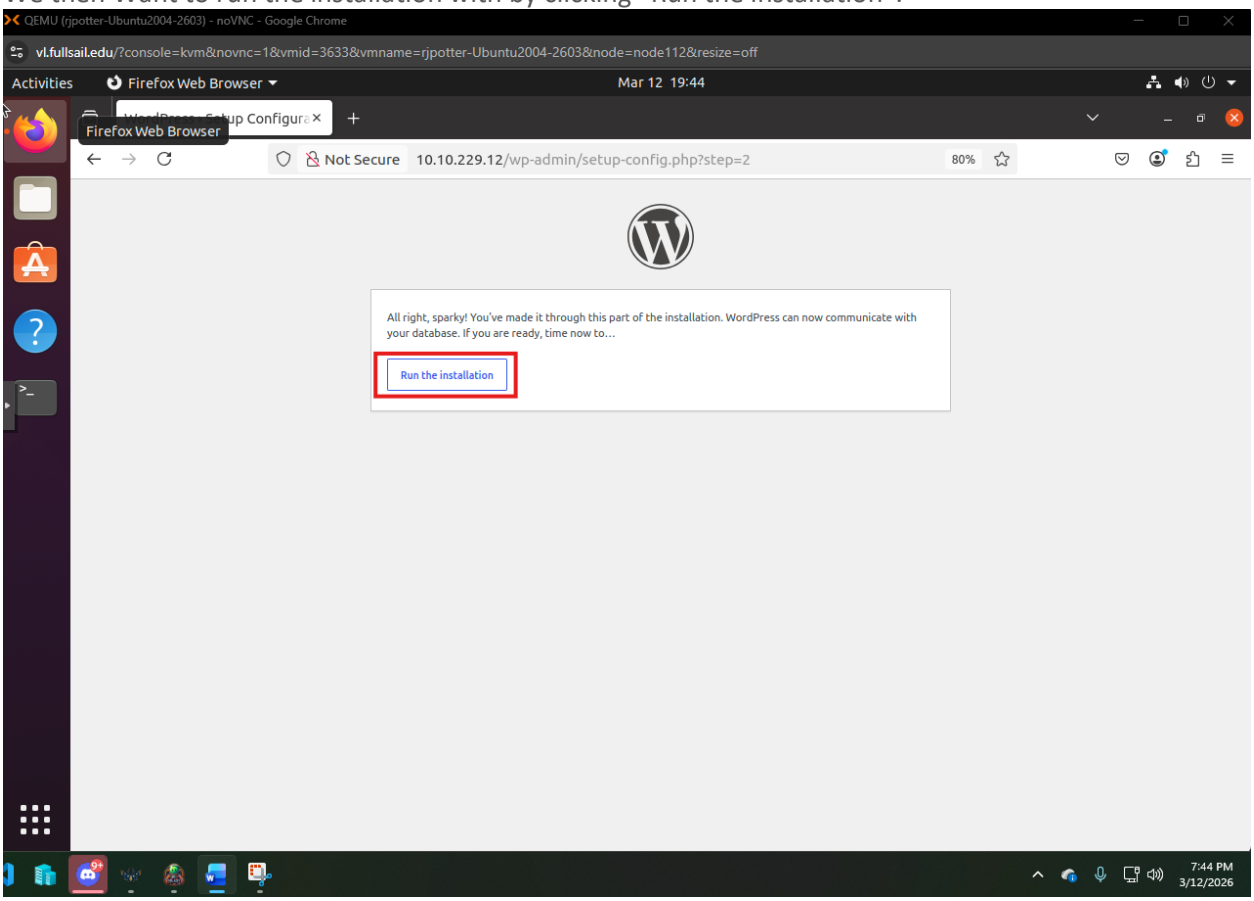
Like this:



Then click Submit.

Run Installation

We then Want to run the installation with by clicking “Run the installation”:

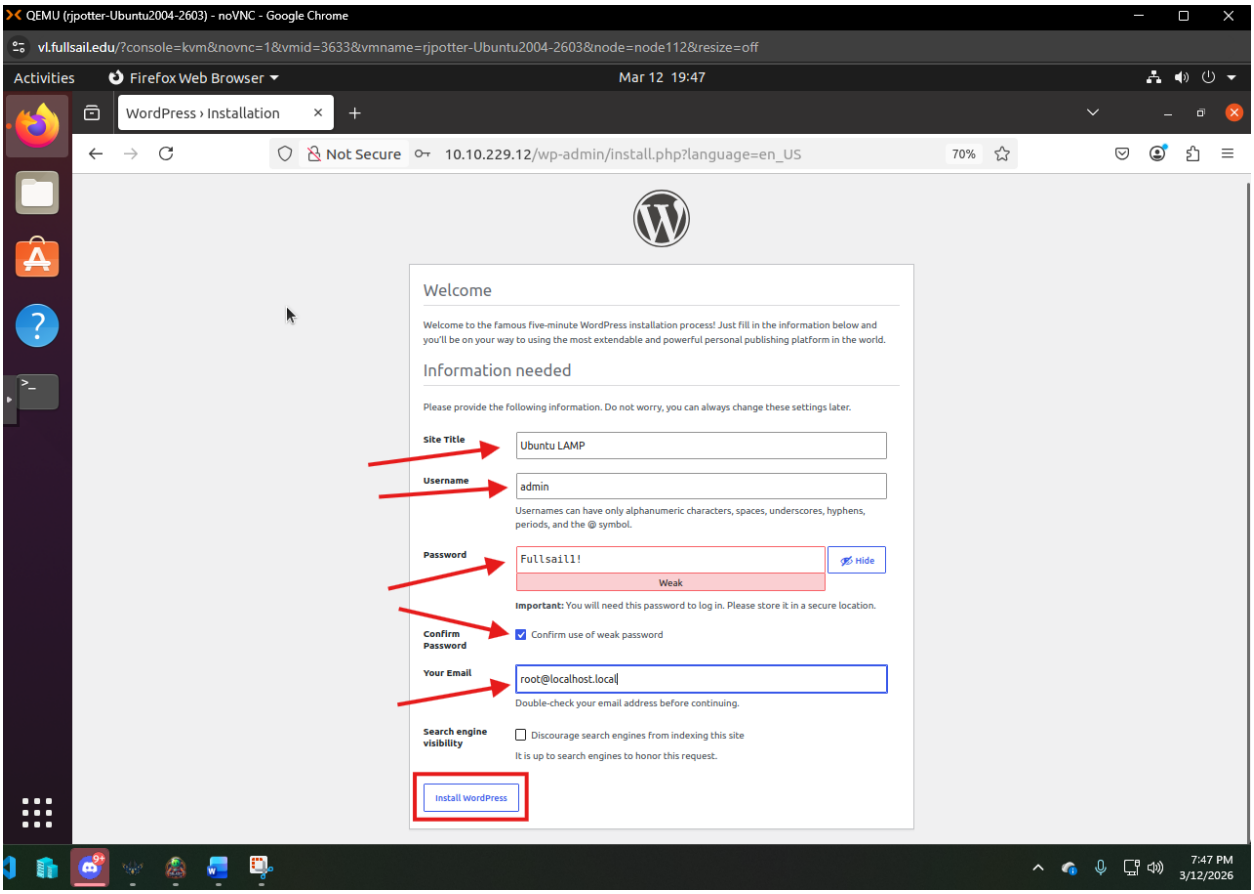


Create an Admin WordPress User

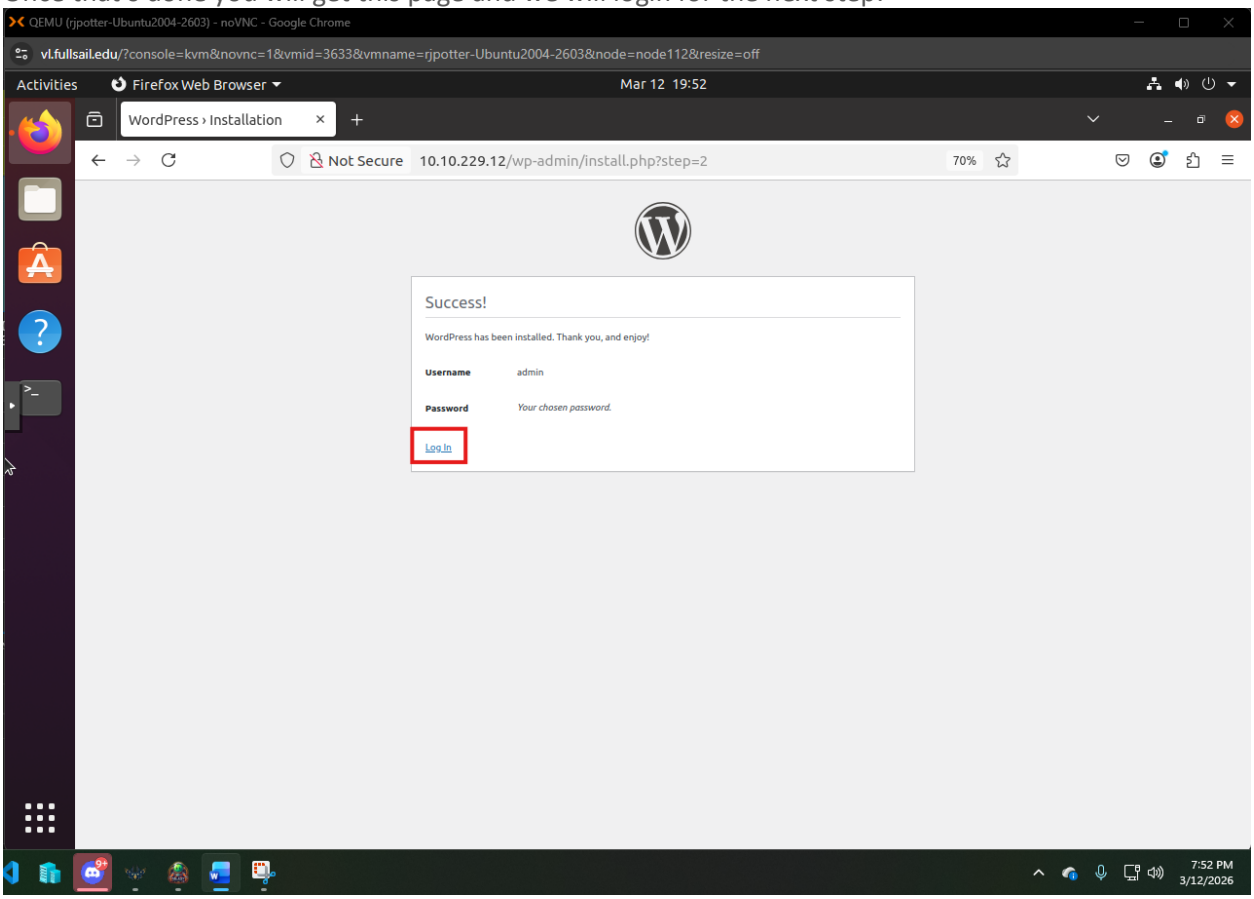
It will then Ask us to fill out all the field for the admin account of the wordpress site use these details:

- **Site Title:** Ubuntu LAMP
- **Username**: admin
- **Password:** Fullsail1!
- **Your email:** root@localhost.local
- **Search Engine Visibility:** leave unchecked

It should Look like this:

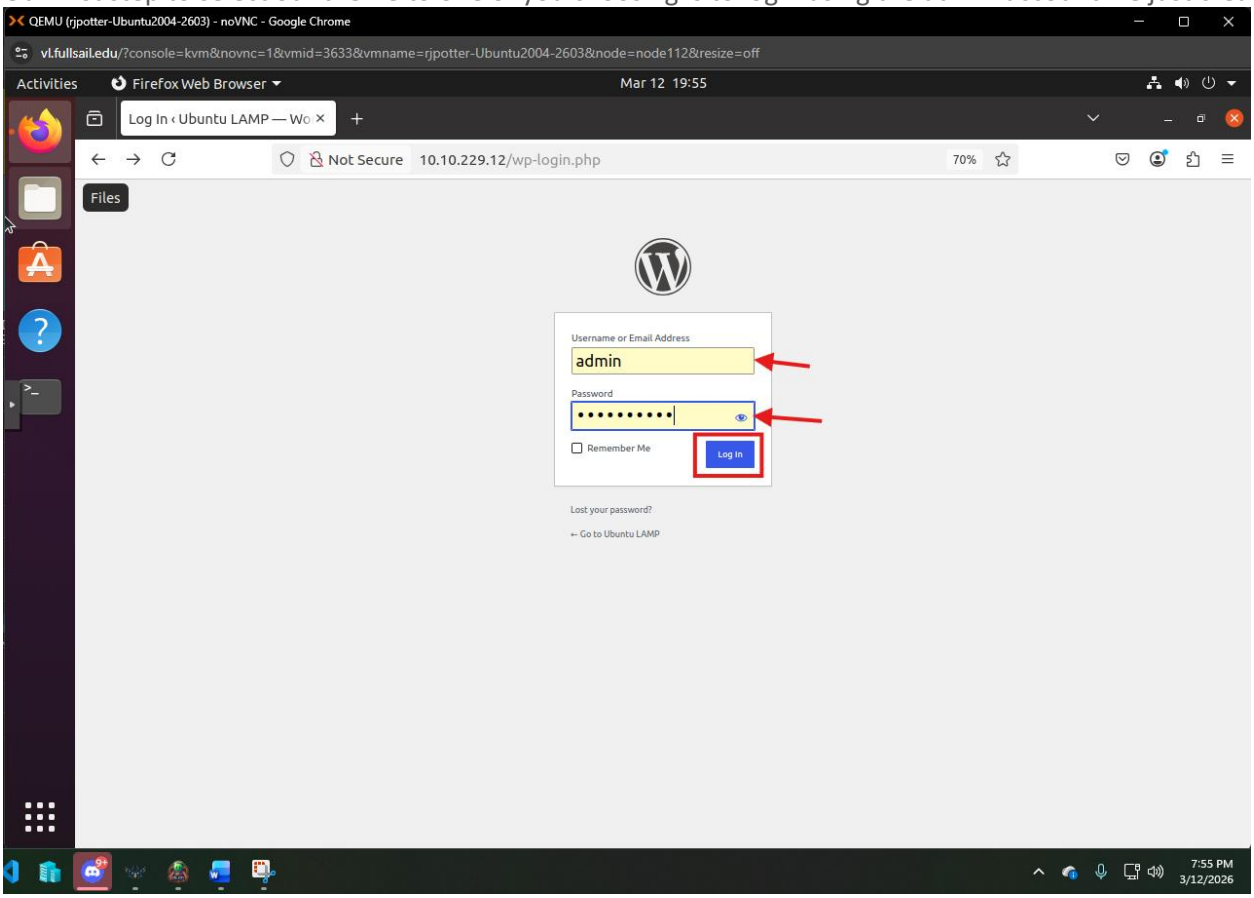


Once that’s done you will get this page and we will login for the next step:

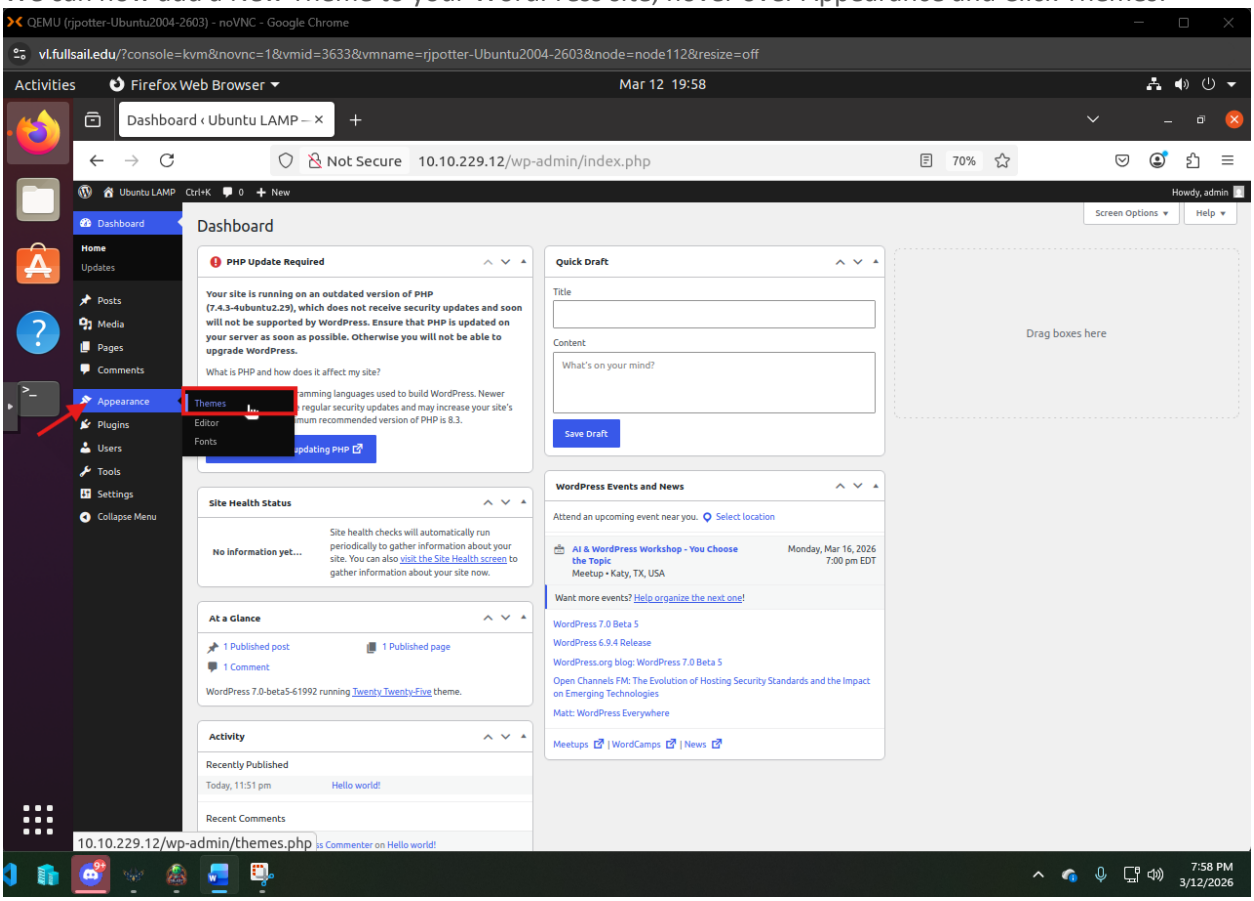


WordPress Site Selections

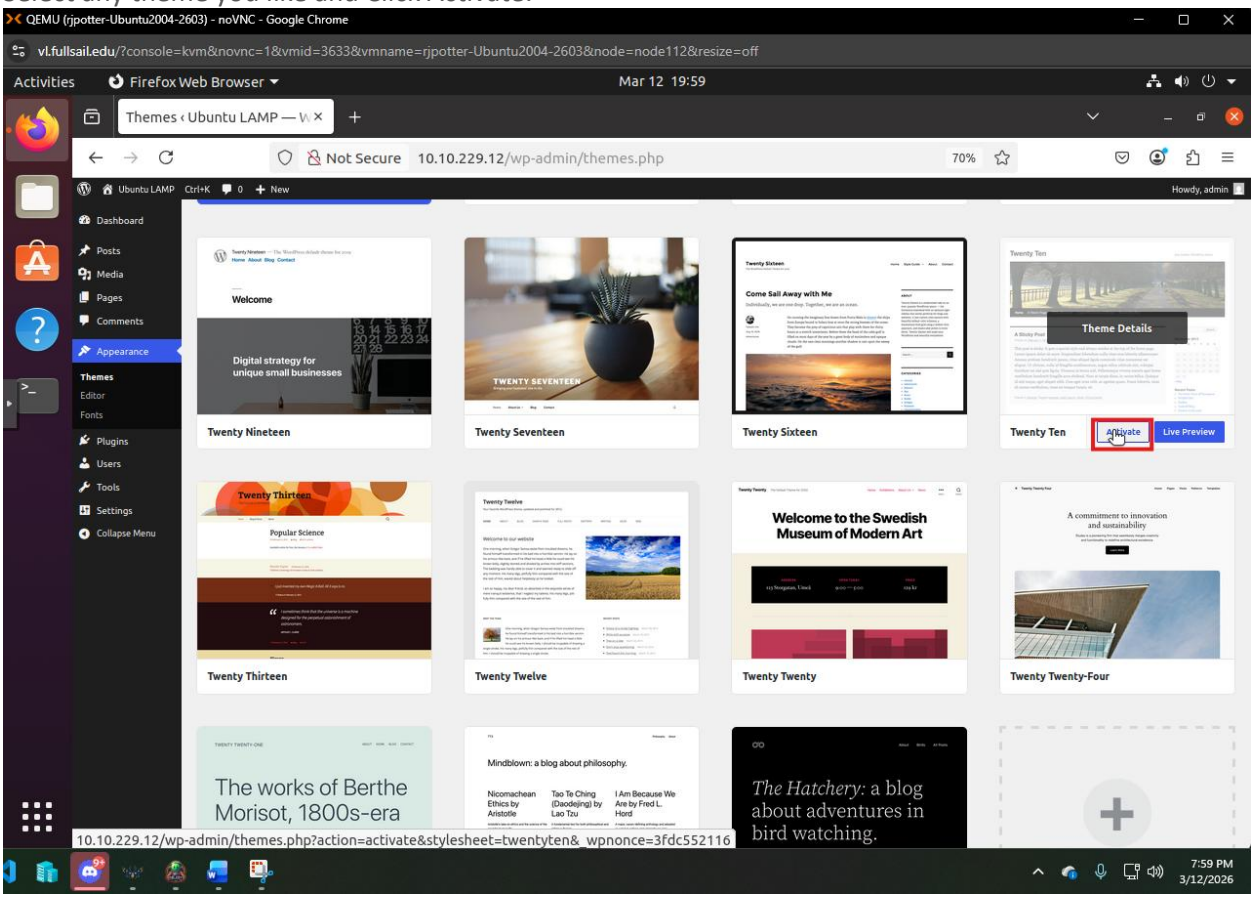
Our first step to select our theme to one of you choosing is to login using the admin account we just created:



We can now add a New Theme to your WordPress site, hover over Appearance and Click Themes:



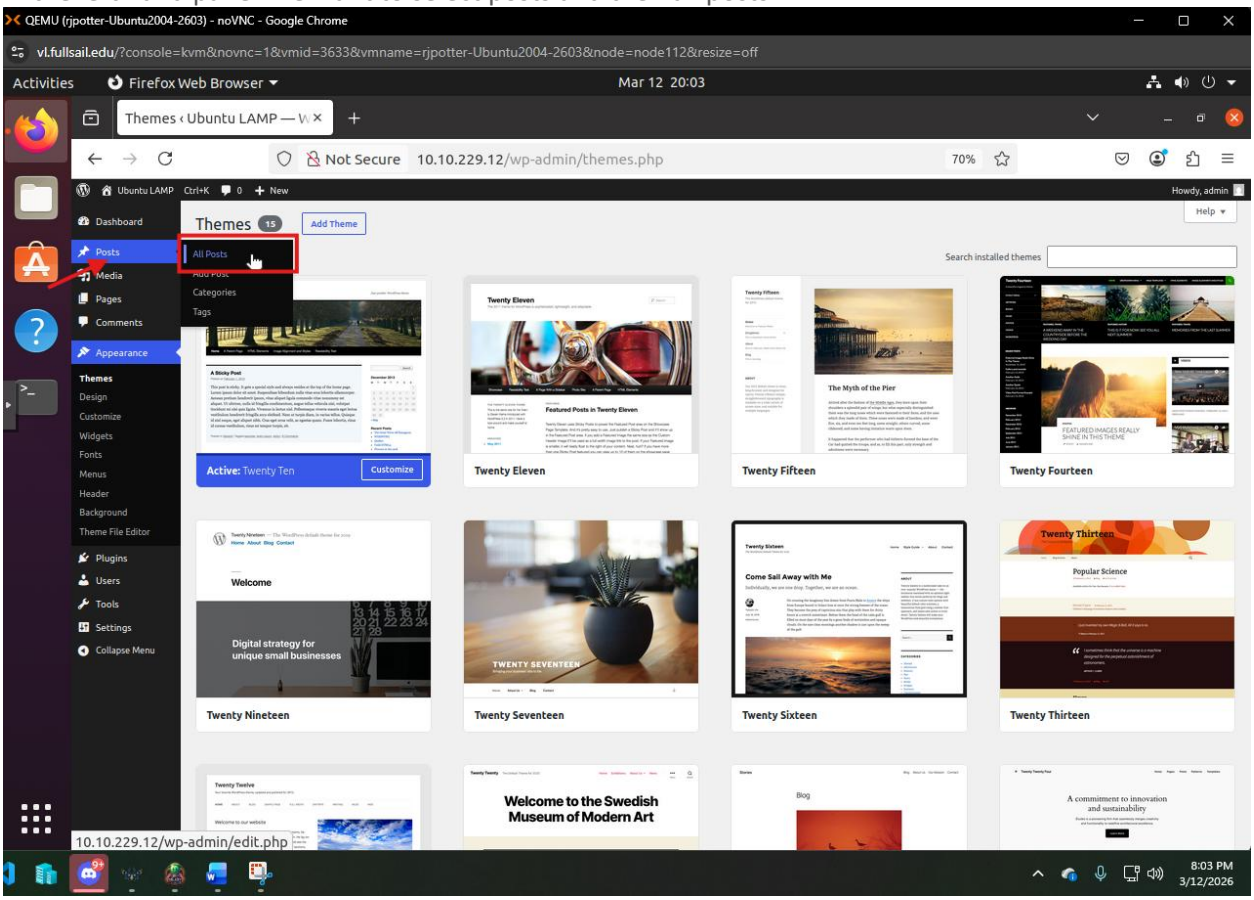
Select any theme you like and Click Activate:



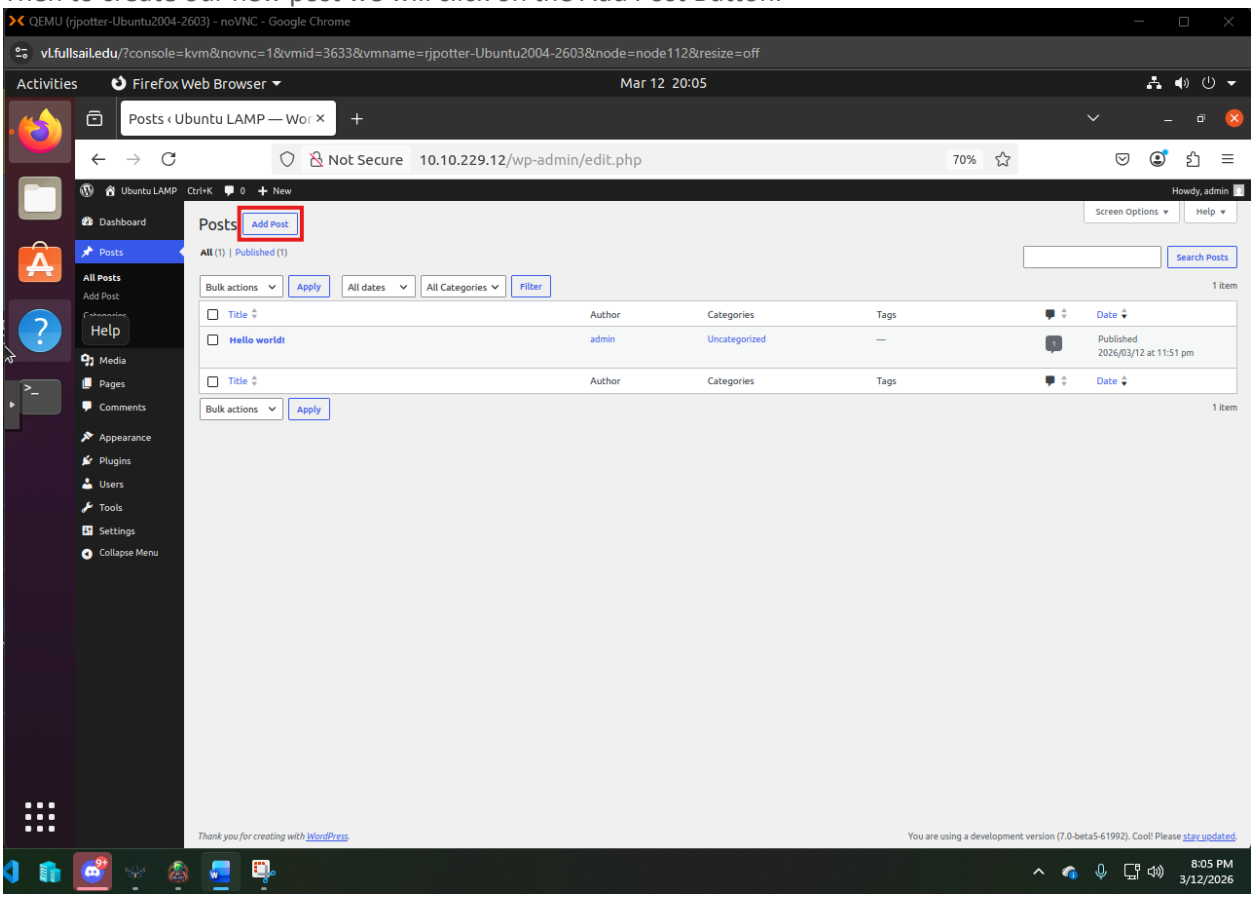
Congratulations! We Set the Theme for the site. Now lets test the site by creating a test post.

Test WordPress Website

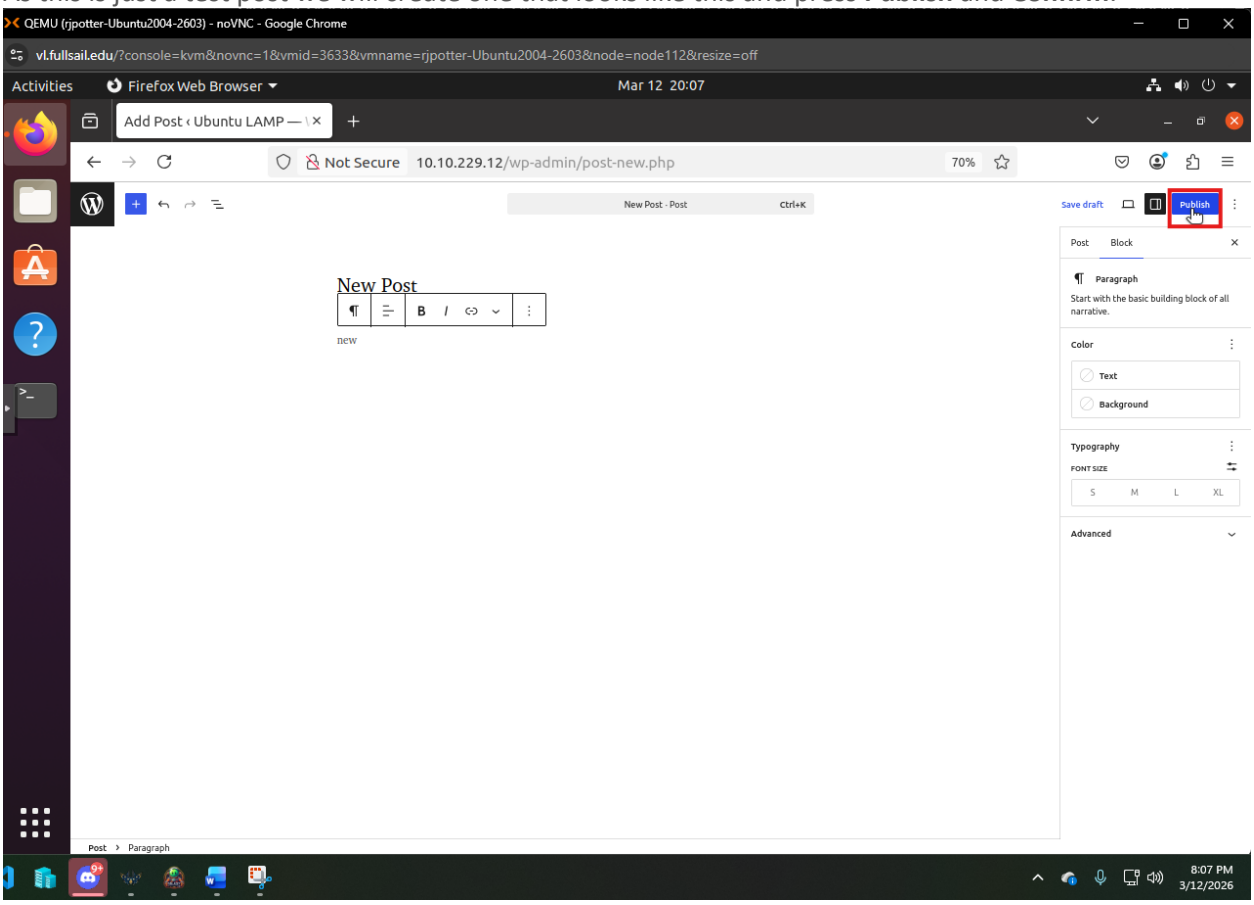
In the left hand panel We want to select posts and then all posts:



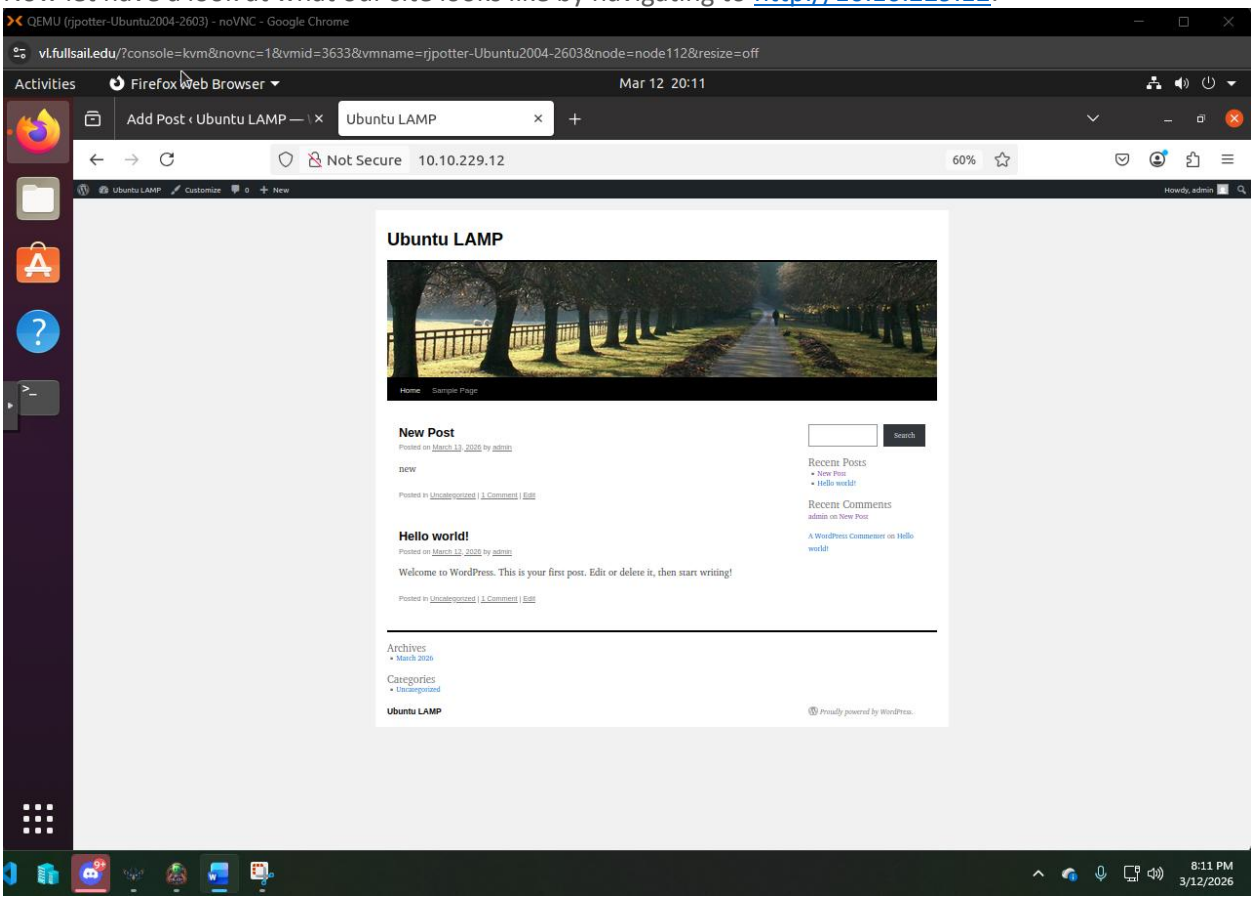
Then to create our new post we will click on the Add Post Button:



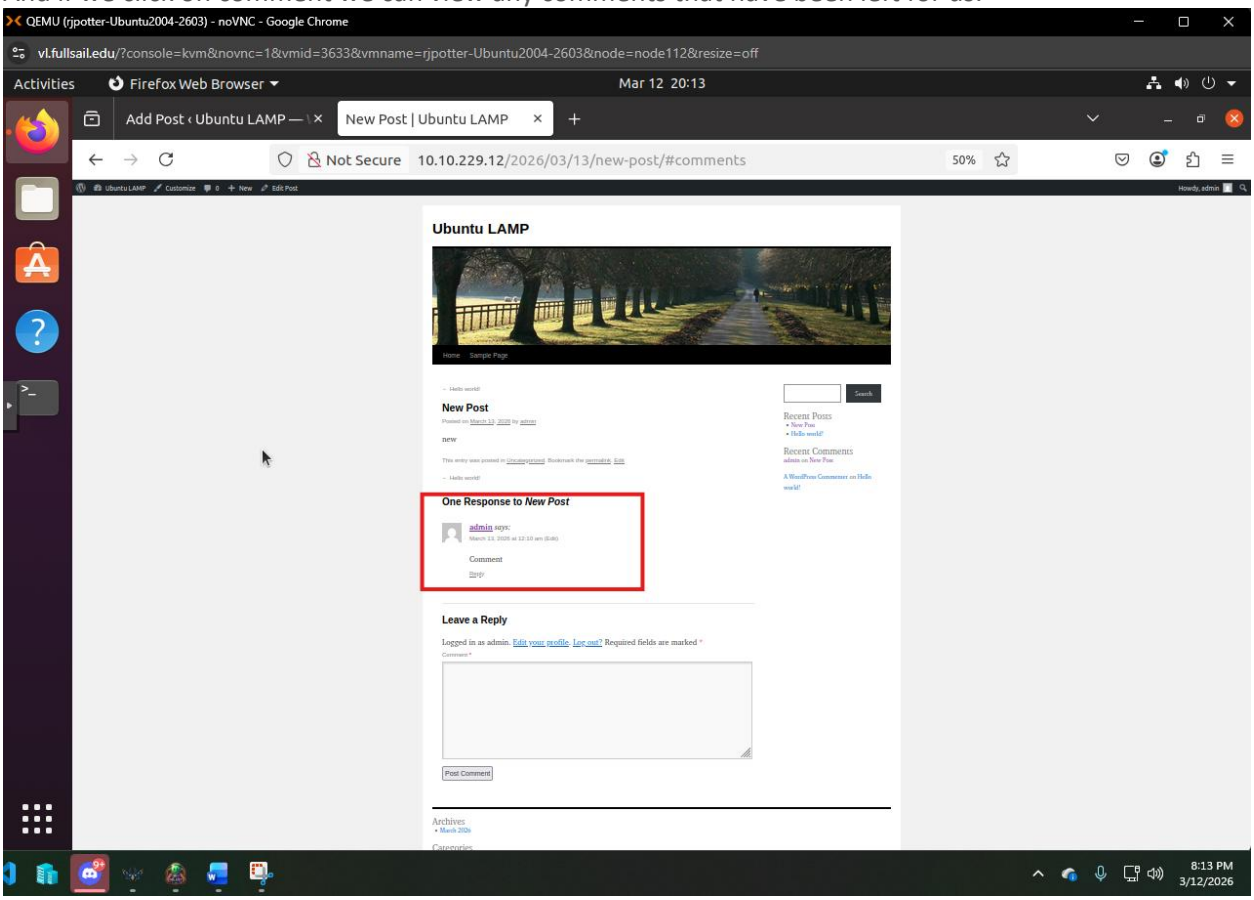
As this is just a test post we will create one that looks like this and press **Publish** and **Confirm**:



Now let have a look at what our site looks like by navigating to http://10.10.229.12:



And if we click on comment we can view any comments that have been left for us:



We now Have a fully operational WordPress Site.

WordPress Security Settings and Configurations

With the initial setup out of the way, we can now begin focusing on securing our website. It is important to consider security from the start, as a weak security posture can lead to several negative impacts on the business. These risks include reputational damage, financial loss, and operational disruption.

We will be implementing three key changes to help harden the environment and establish a baseline level of defense in depth. That said, no security solution is ever 100% perfect. The goal is to make the website a more difficult target, as most attackers will move on if easier opportunities are available.

The first change will be adjusting default file permissions to ensure that only the system, WordPress, or authorized users can make modifications, preventing unauthorized changes. The second change will be relocating and securing the wp-config.php file, which contains sensitive configuration data. The third and final change will be implementing an application-layer firewall to provide additional protection for the website and its data.

Defence In Depth:

Defense in depth is the concept of using multiple layers of security instead of relying on just one control to protect a system. The idea is simple: if one layer fails, there are still other layers in place to stop or slow down an attacker. A good way to think about this is like securing your home. You might have a lock on your front door, but you also have windows, maybe a fence, and possibly a security system. Even if someone gets past one of those, they still have to deal with the others. This layered approach makes it much harder for someone to successfully break in.

When applying this to a website, you don't depend on a single security measure. Instead, you combine different types of controls that each protect a specific part of the system. For example, setting proper file permissions helps prevent unauthorized changes to your files. Securing the wp-config.php file protects sensitive information like database credentials. Adding an application layer firewall helps block malicious traffic before it can reach your website. Each of these controls works together as part of a larger security strategy.

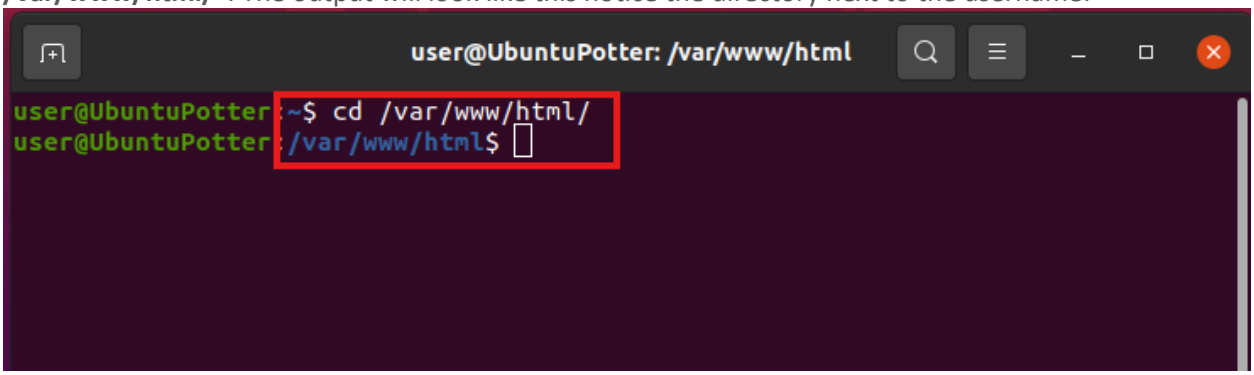
It is also important to understand that no security setup is ever 100% perfect. Attackers are constantly finding new ways to get around defenses. Because of this, the goal of defense in depth is not to be unbreakable, but to make your system harder to attack and less attractive as a target. Most attackers are looking for the easiest way in, so when they encounter multiple layers of security, they are more likely to move on to something else.

If you would like to read more you can Visit:
https://www.fortinet.com/resources/cyberglossary/defense-in-depth

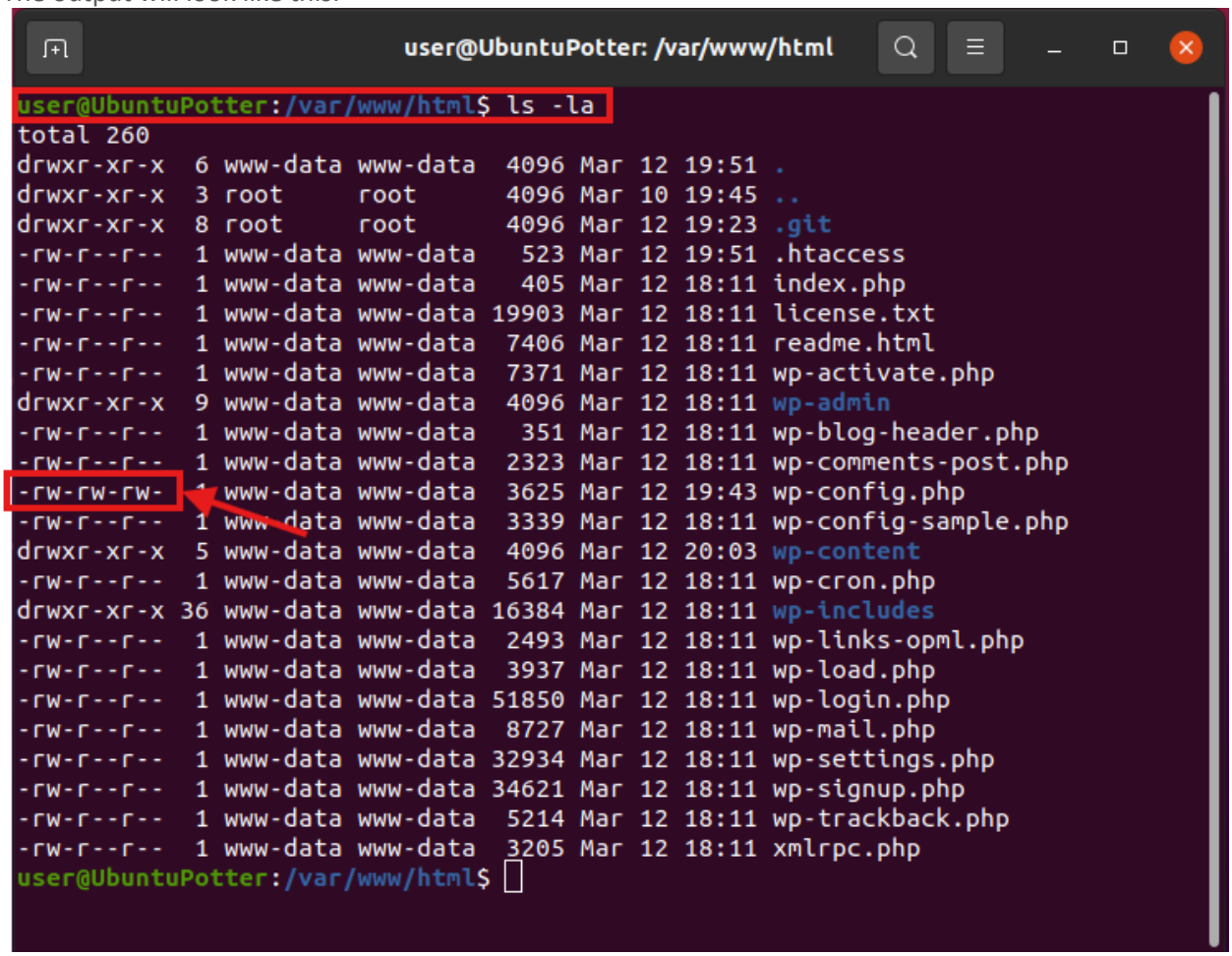
File Permissions

Vulnerability

So here we can see that all of the permissions that our current configurations has. This can be viewed by first navigating to the folder that contains our wordpress site. The command to do this is "**cd /var/www/html/**". The output will look like this notice the directory next to the username:



This will put us in the directory and we can view the current configurations. With this command "**ls -la**". The output will look like this:



Now here we have a problem with the current configuration ANYONE Can read and Write in our WP-Config file. Let me Explain a little about what we are looking at.

Linux file permissions are how the system controls who can access or change files and folders. Every file has three groups of permissions: the owner (the person who created it), the group (a set of users), and everyone else. Each of these groups can be given three types of access: read, write, and execute. Read means you can view the file, write means you can change it, and execute means you can run it like a program.

You can see these permissions by using the ls -la command. This command lists all files, including hidden ones, and shows details about them. At the beginning of each line, you will see a string of letters and dashes that represent the permissions.

For example, you might see something like this: **-rwxr-xr--**. The first character tells you what type of file it is. A dash means it is a regular file, and a "**d**" means it is a directory. The next nine characters are split into three groups of three. The first group shows what the owner can do, the second shows what the group can do, and the third shows what everyone else can do.

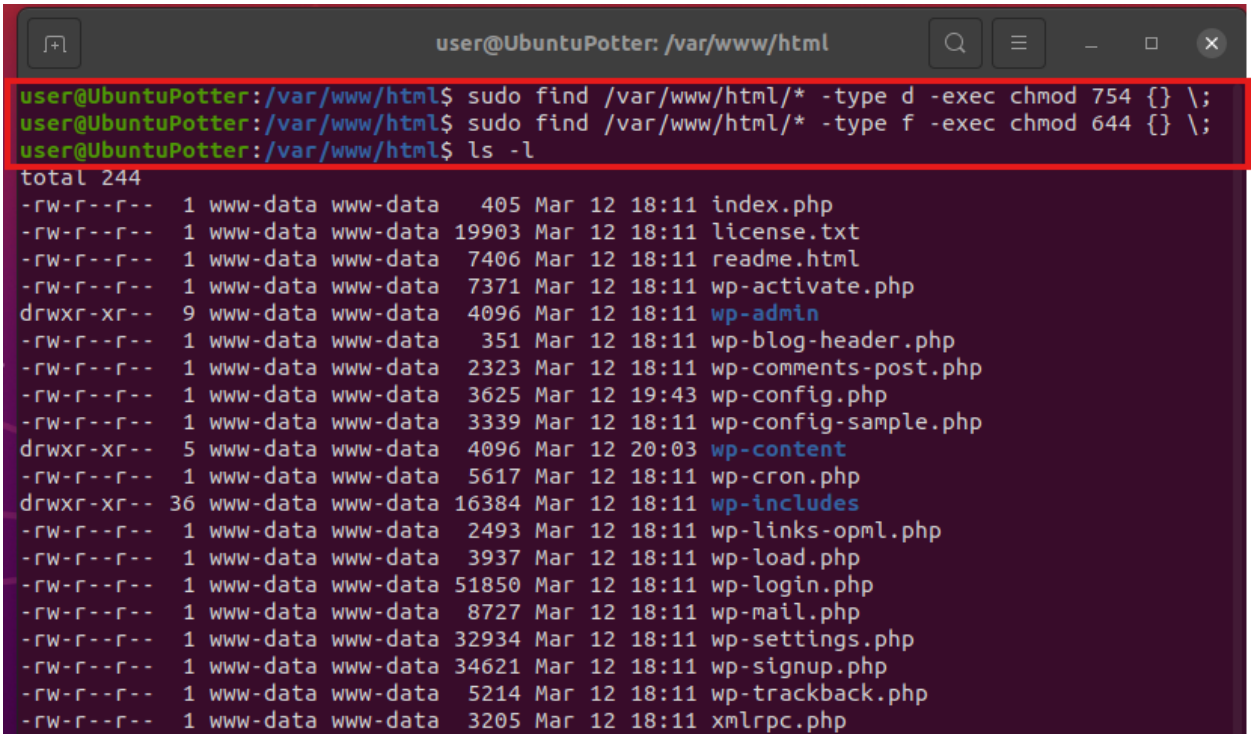
In this example, the owner has full access (read, write, and execute), the group can read and execute but not make changes, and everyone else can only read the file. This setup helps make sure that only the right people can make changes, which is important for keeping a system secure and preventing unauthorized access.

Do you want to know more? Here is a good reference to learn more about it:
https://www.redhat.com/en/blog/linux-file-permissions-explained

Configuration

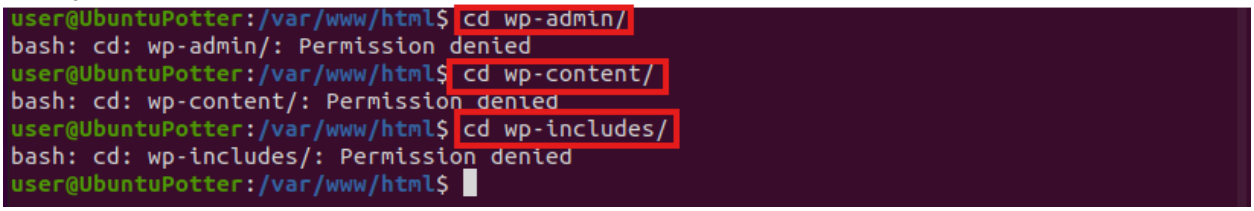
WordPress Makes the following recommendations for file permissions on files and folders. They recommend that Files should be set to 644 and what Folders should be set to 754. This will give users the minimal amount of privilages required and in order to maintain a functional website.

So first we will set the permissions of all the Folders. The command we will use here is "**sudo find . -type d -exec chmod 754 {} \;**" This will change the permissions of out folders it should look like this:
The next command will be to set the permissions of the files in the same directory. The command for this is "**sudo find . -type f -exec chmod 644 {} \;**" and then "**ls -l"** to see the new permissions. The output will look like this:



Validation

Now, Let out verify that the file permissions have been fixed. We will use the commands
“**cd wp-admin/**”
“**cd wp-content/**”
“**cd wp-includes/**”

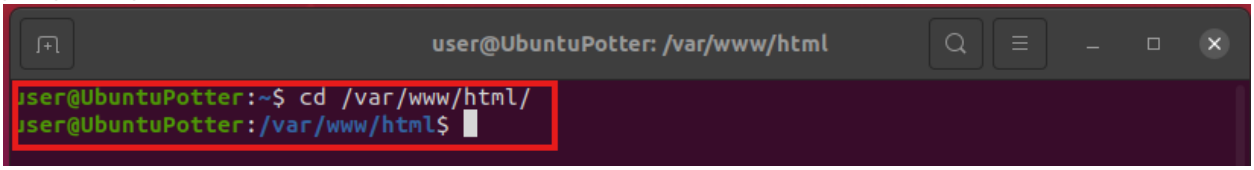


We as a normal user should now be denied access but wordpress will still be able to access these files and the site will contine to work because it is the owner and has full access.

Securing wp-config.php

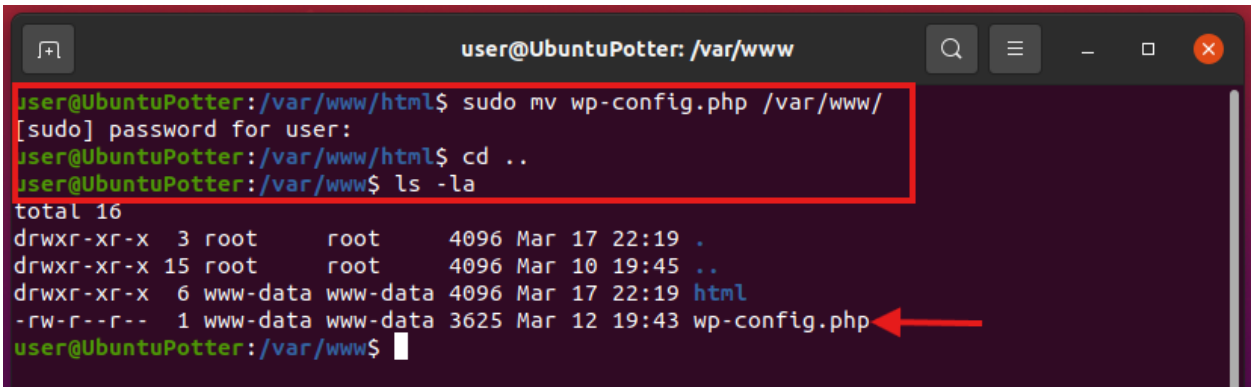
Vulnerability

The Next step in hardening our WordPress is to move the configuration file Up one level to remove it from the web facing directory. This will make the file harder for a bad actor to find the config file to make changes to it. Out first step is to go back to the WordPress root directory. We will use the command "**cd /var/www/html**" It will look like this:



Configuration

Now to move the file to the level above this one we will use the command "**sudo mv wp-config.php /var/www/**" and to confirm it was moved we will use the "**cd ..**" command to go to /var/www/ directory and then use the command "**ls -la**" and we can see that the wp-config.php file is now in this directory. As seen below:

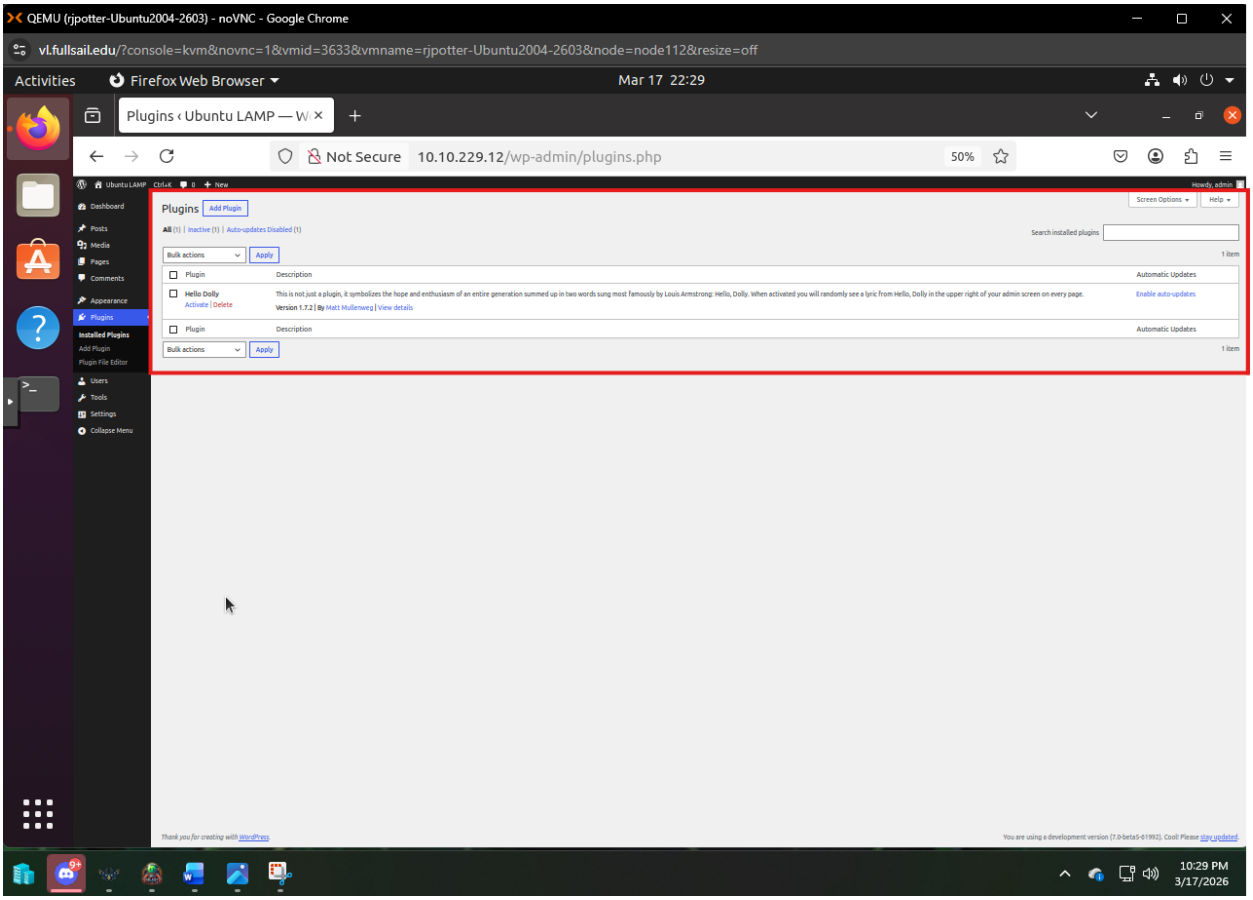


Validation

So as we can see the wp-config.php file is not in the /var/www directory. This will make it harder for an unskilled attacker to get access to it from the website and and should deter bad actors.

Firewall (Shield)

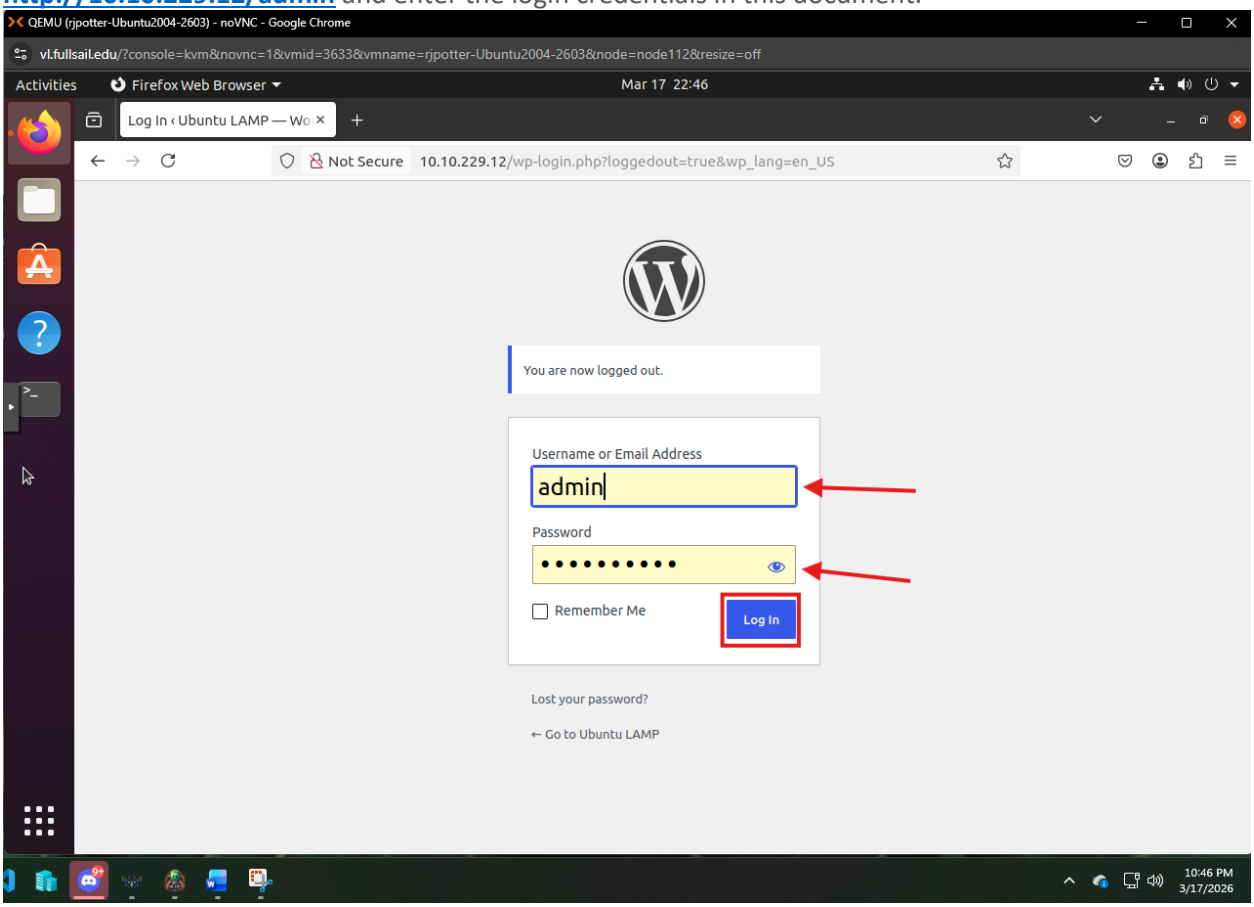
Vulnerability



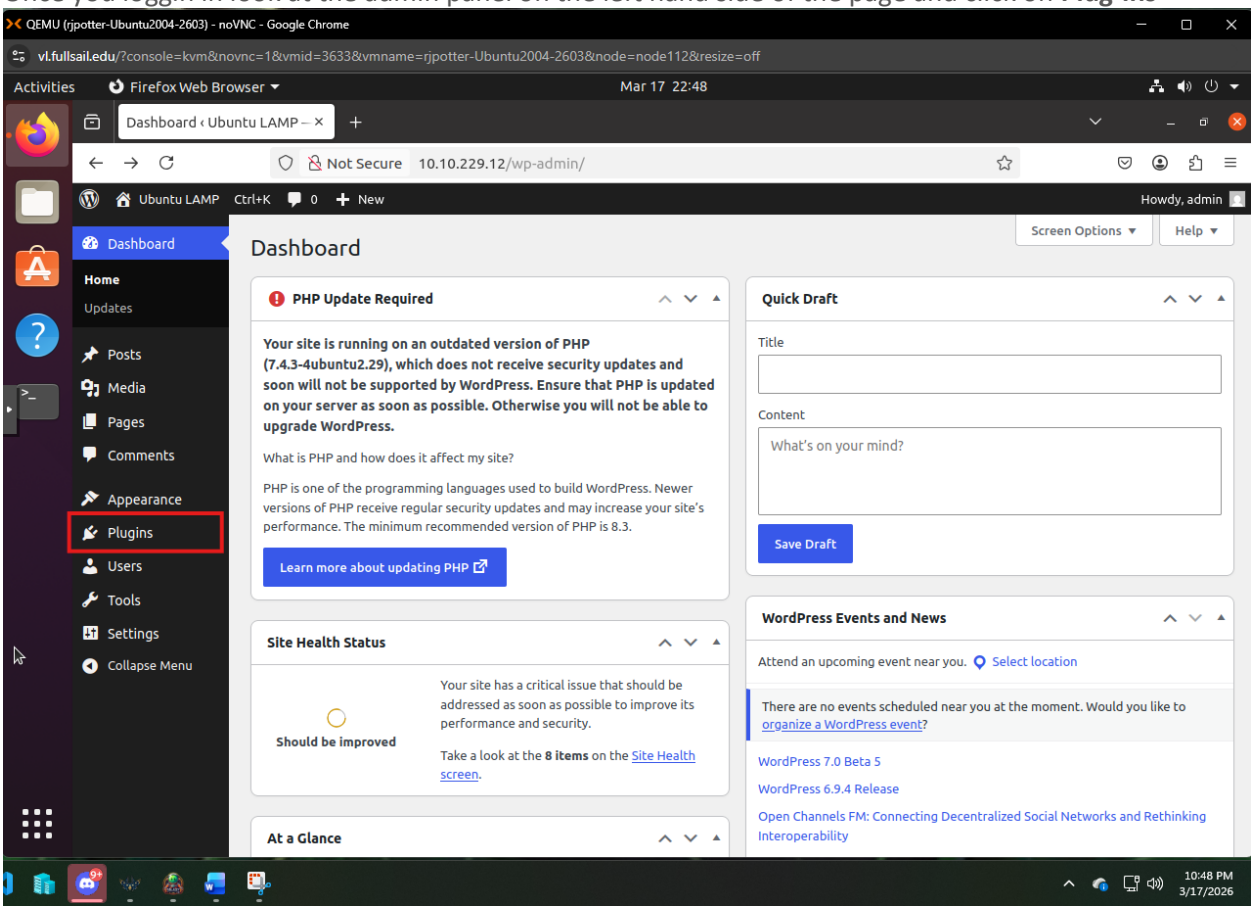
So currently on our WordPress site we have no firewall currently setup so we have no contol over network traffic other than our existing hardware firewall. By adding WordPress firewall we can add an additional layer of protection for our setup.

Configuration

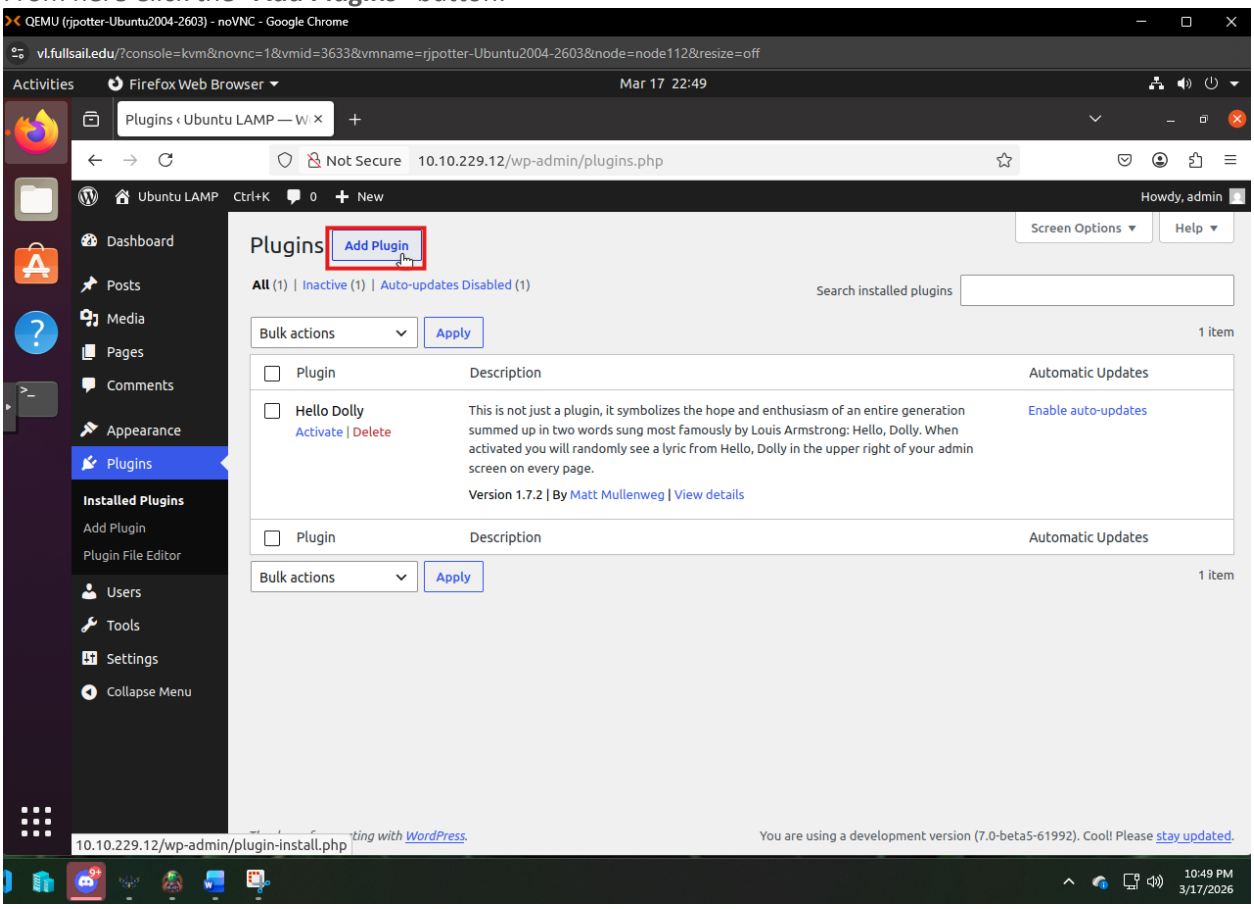
Our First step in the process is to access the admin login page. Open the Firefox browser and navigate to **http://10.10.229.12/admin** and enter the login credentials in this document.



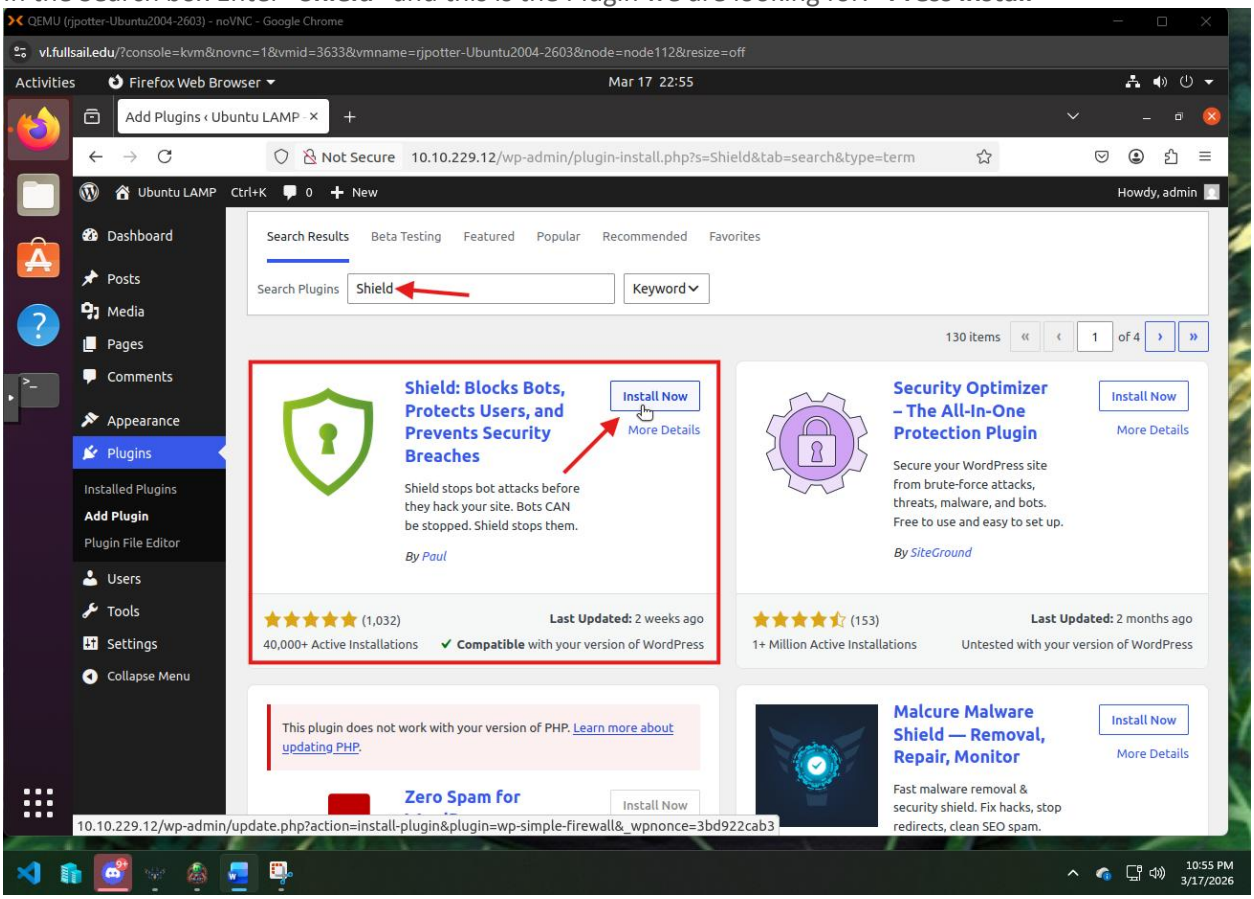
Once you loggin in look at the admin panel on the left hand side of the page and click on **Plug-ins**



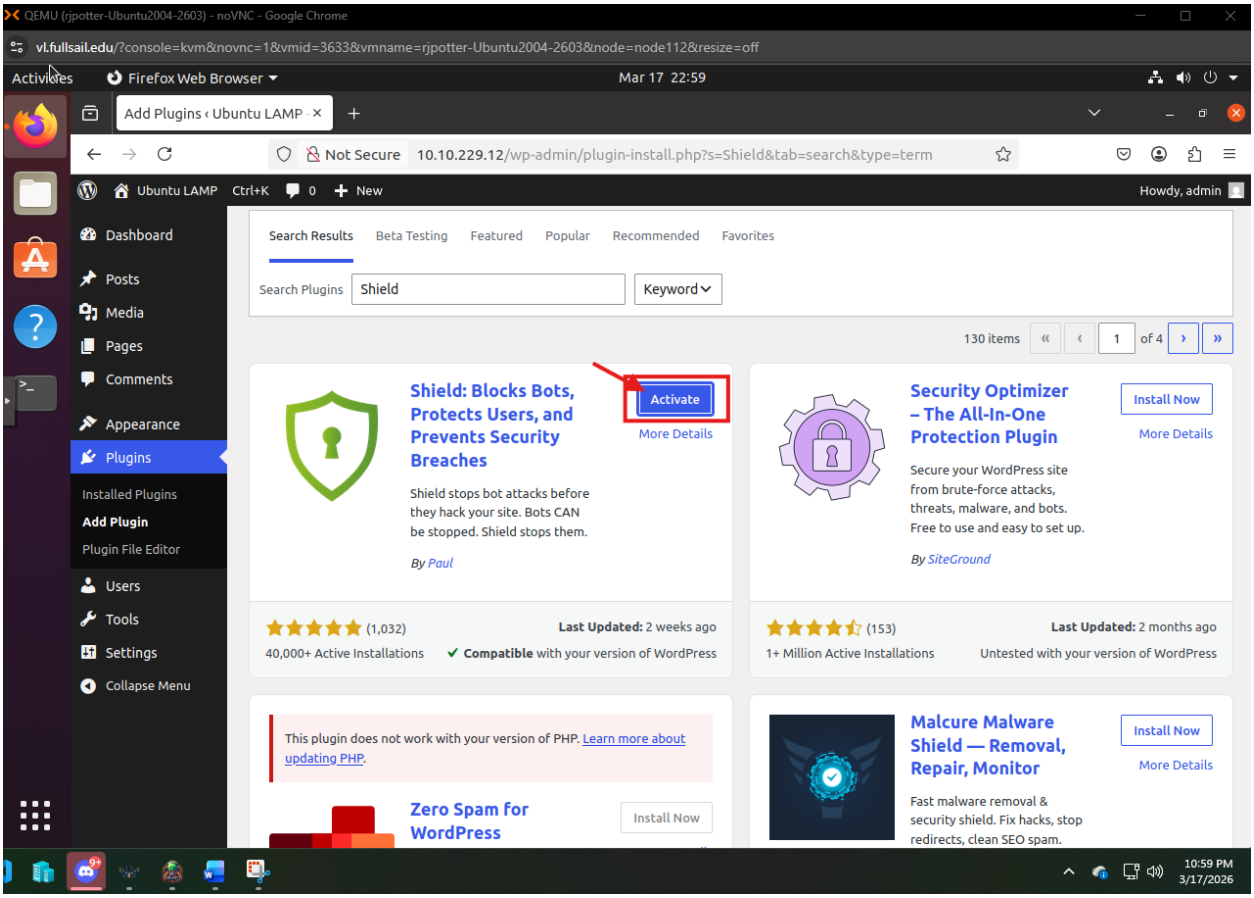
From here Click the “**Add Plugins”** button:



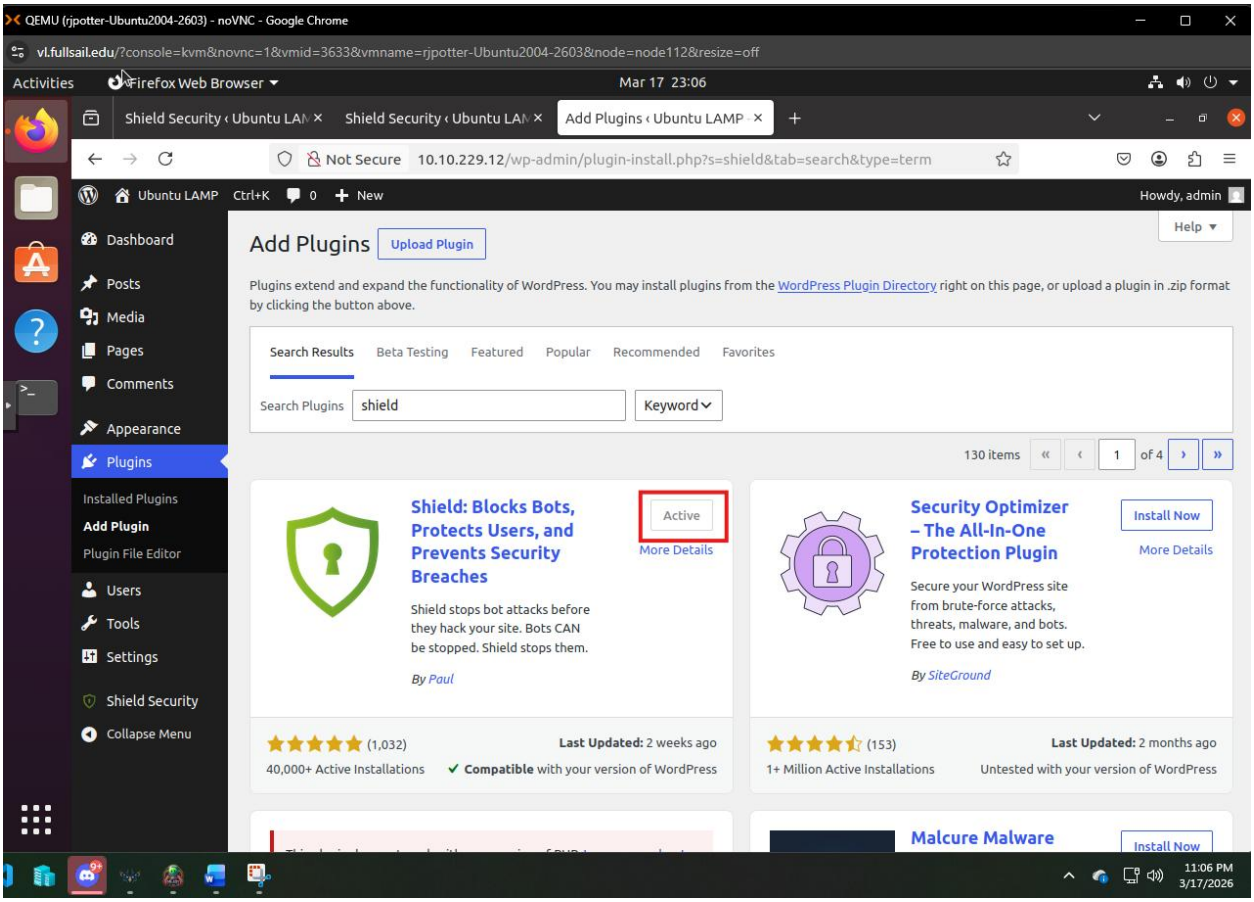
In the Search box Enter “**Shield**” and this is the Plugin we are looking for: “**Press install”**



Once its installed, Press “**Activate**”:

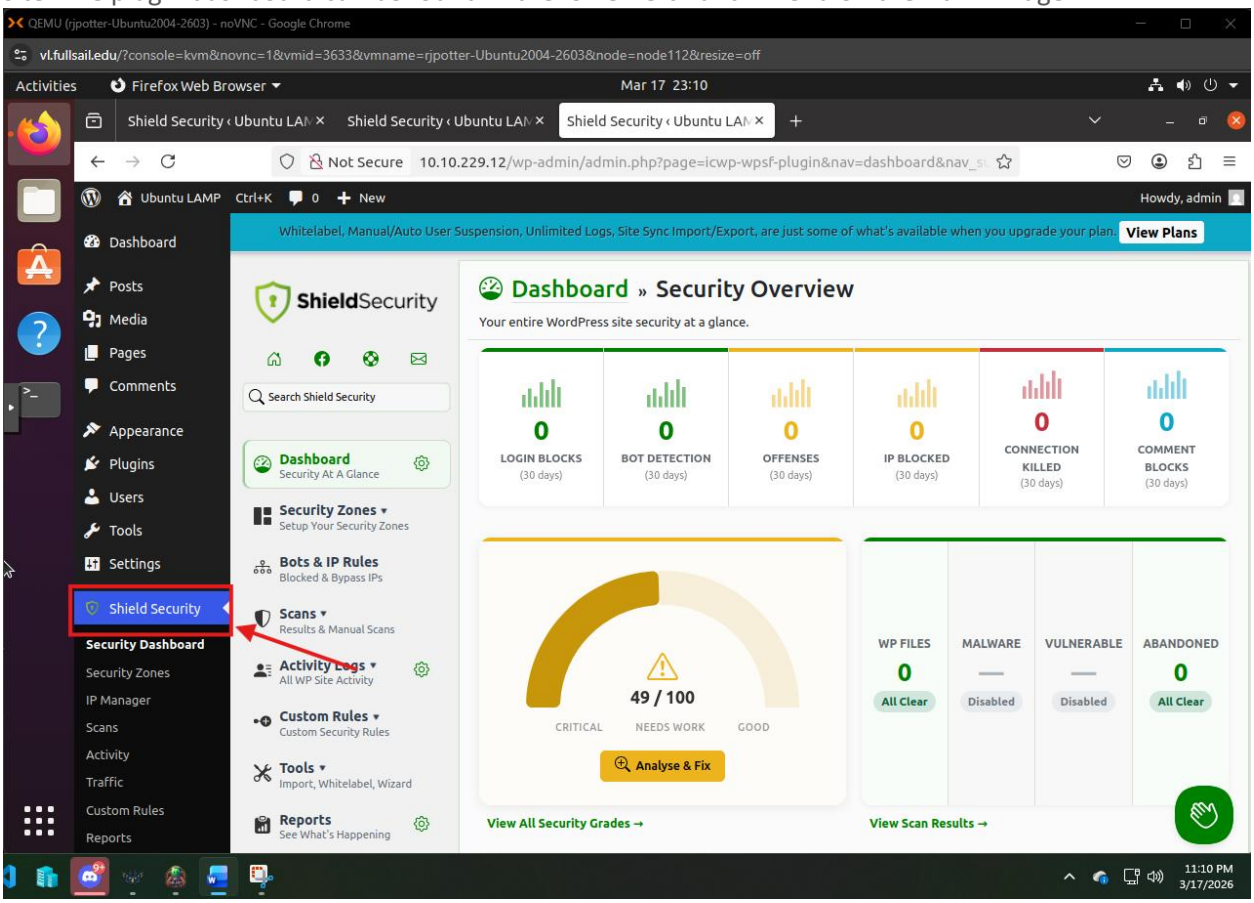


It should now show As active:



Validation

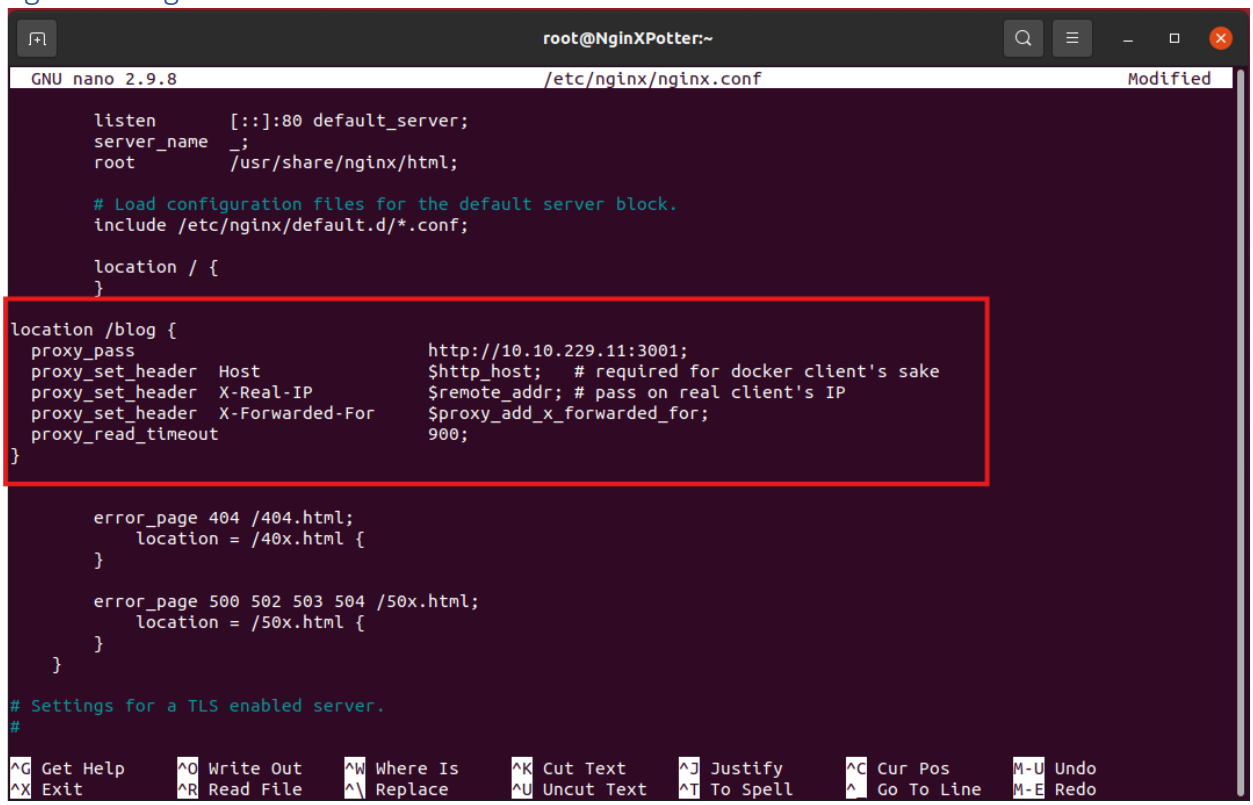
We can now View our dashboard and see that the security is no active and see any activity. This serves multiple purposes as it now protects us from bots as well as provides an additional source of loggin traffic to our site so we know what kind of visitors we get and if there is any malcious activity tword our site. The plugin dashboard can be found in the lower left hand menu on the Admin Page:



t

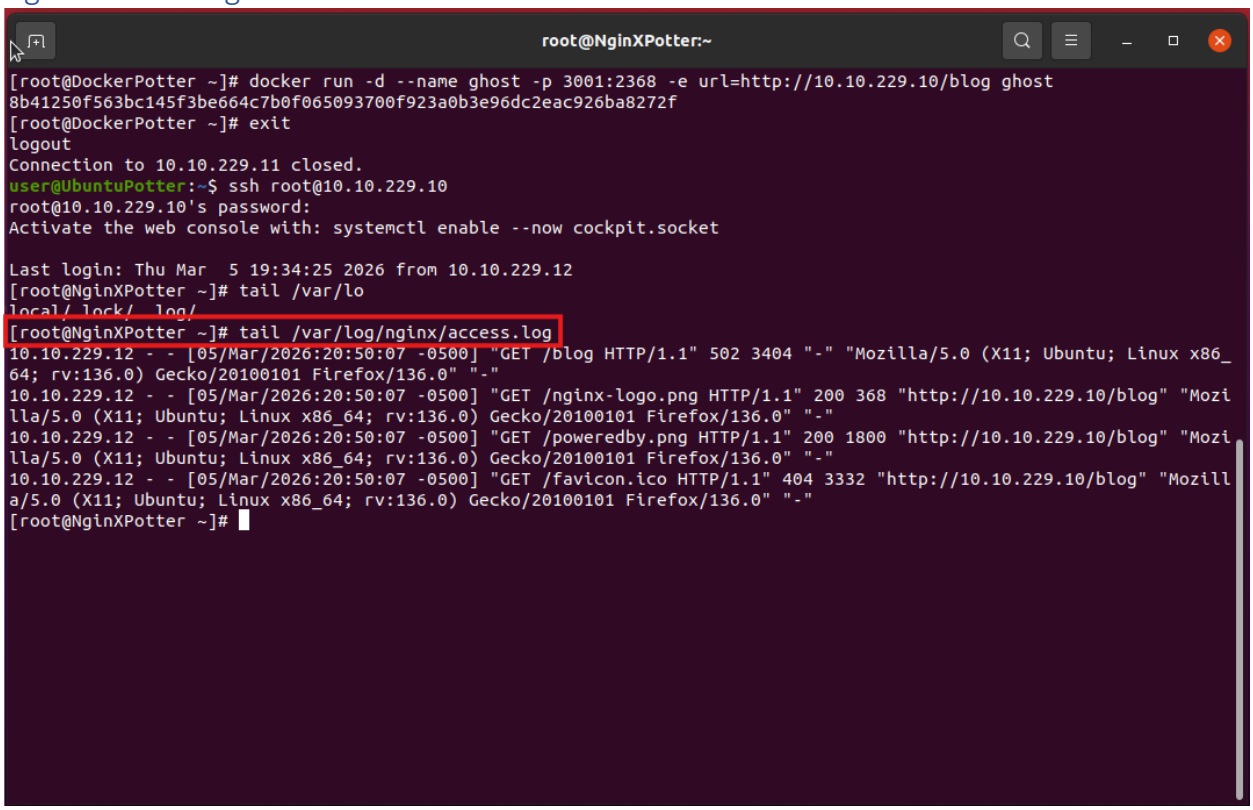
Appendix A

NginX Config File



Appendix B

NginX Access Log File



NginX Error Log File